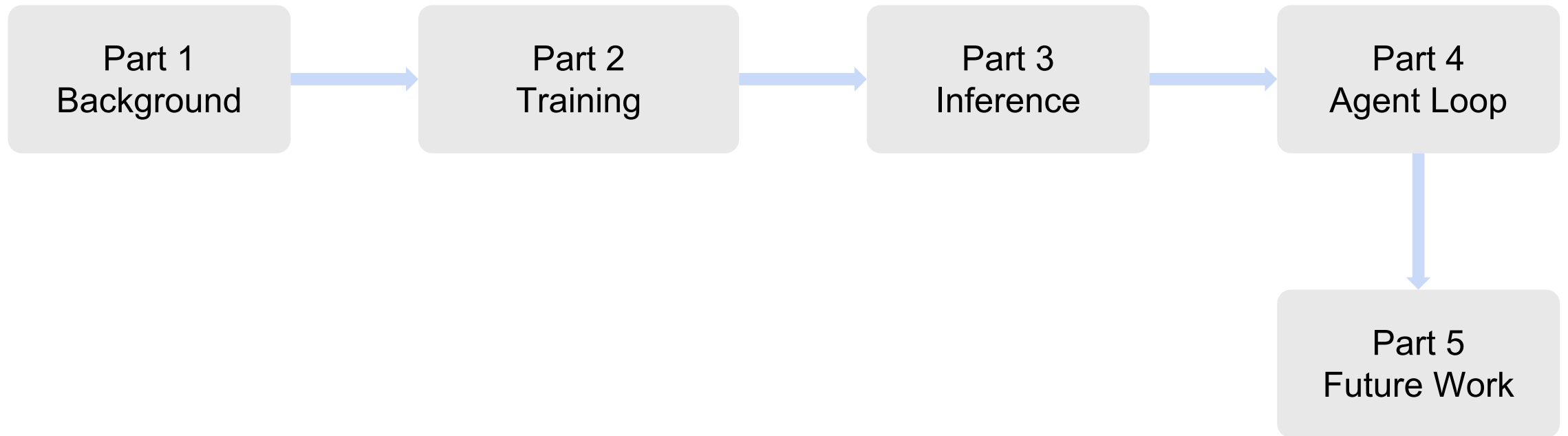


HPC101: Algorithm-Infra Co-Design for AGI

Presenter: Bohan Zhuang

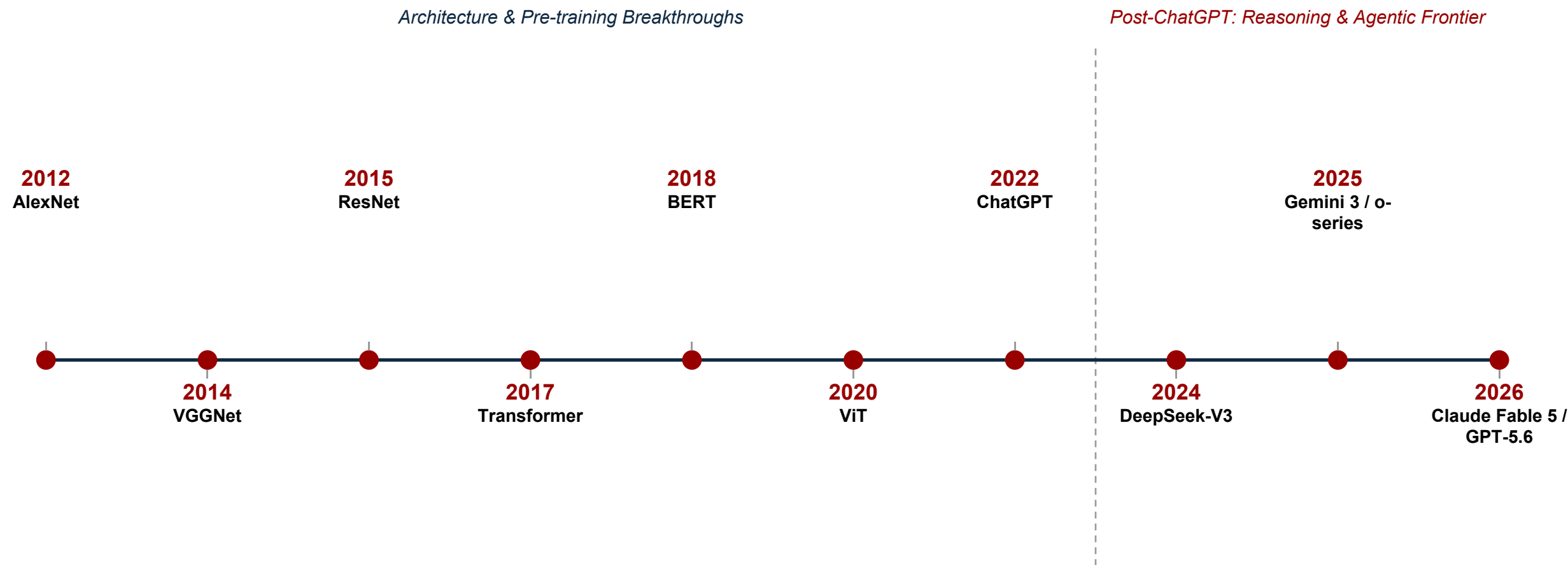
PI of [ZIP Lab](#) @ Zhejiang University

Overview



Part 1 - Background of Large Foundation Models

Background - Evolution of Foundation Models



From architecture races (CNN → Transformer) to scale races (pre-training) to agentic, self-evolving races — each era compounds on the last.

Background - The Industry Is Token Maxing

3.2 Quadrillion

tokens / month
Google, I/O 2026

15B / minute

OpenAI API throughput (~21.6T/mo)
up 50x since 2023

1.5Q → 120Q /mo

industry agent+LLM tokens, 2026→2030
24x projected growth

100B

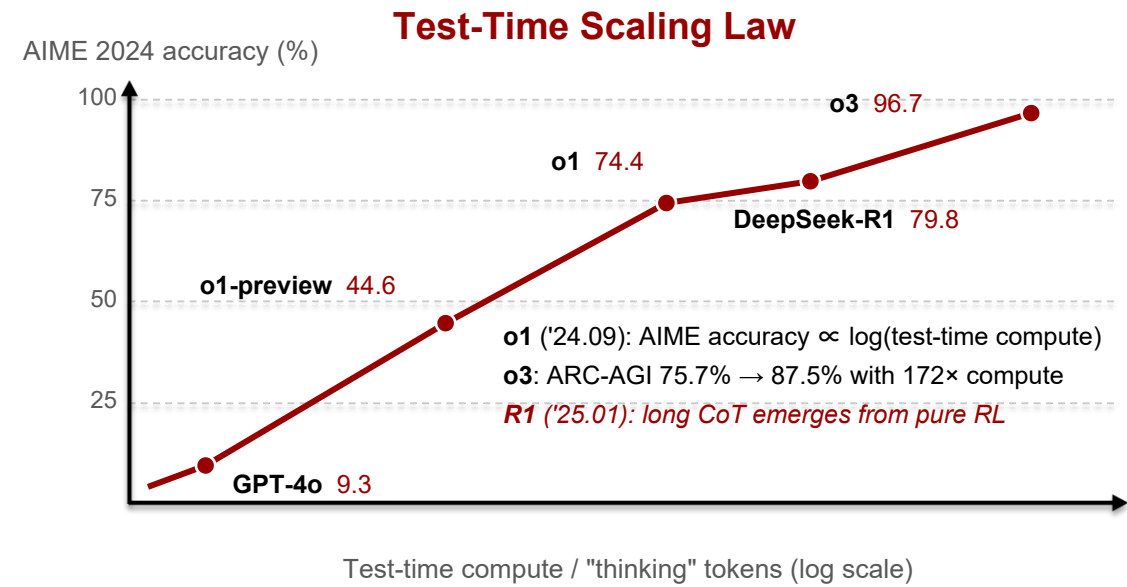
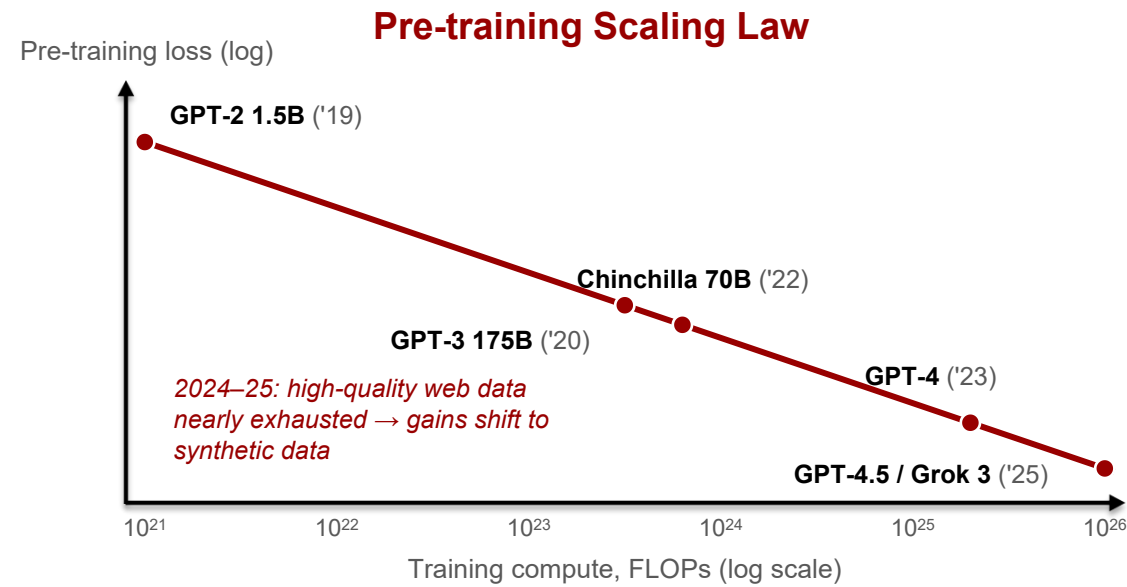
tokens / month

Case in point (OpenAI, June 2026):

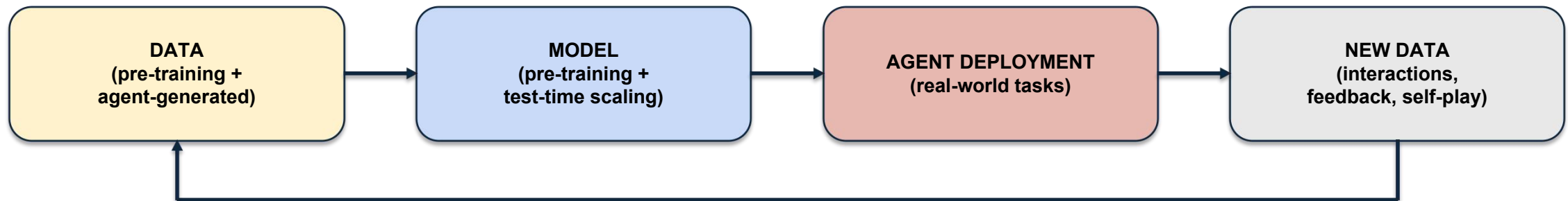
its single largest enterprise customer now burns 100 billion tokens per month — the industry is maxing out tokens, and throughput, not model count, is the new usage currency.

Everyone is token maxing — token efficiency & productivity are the new moat.

Background - Scaling Laws and the Data Flywheel

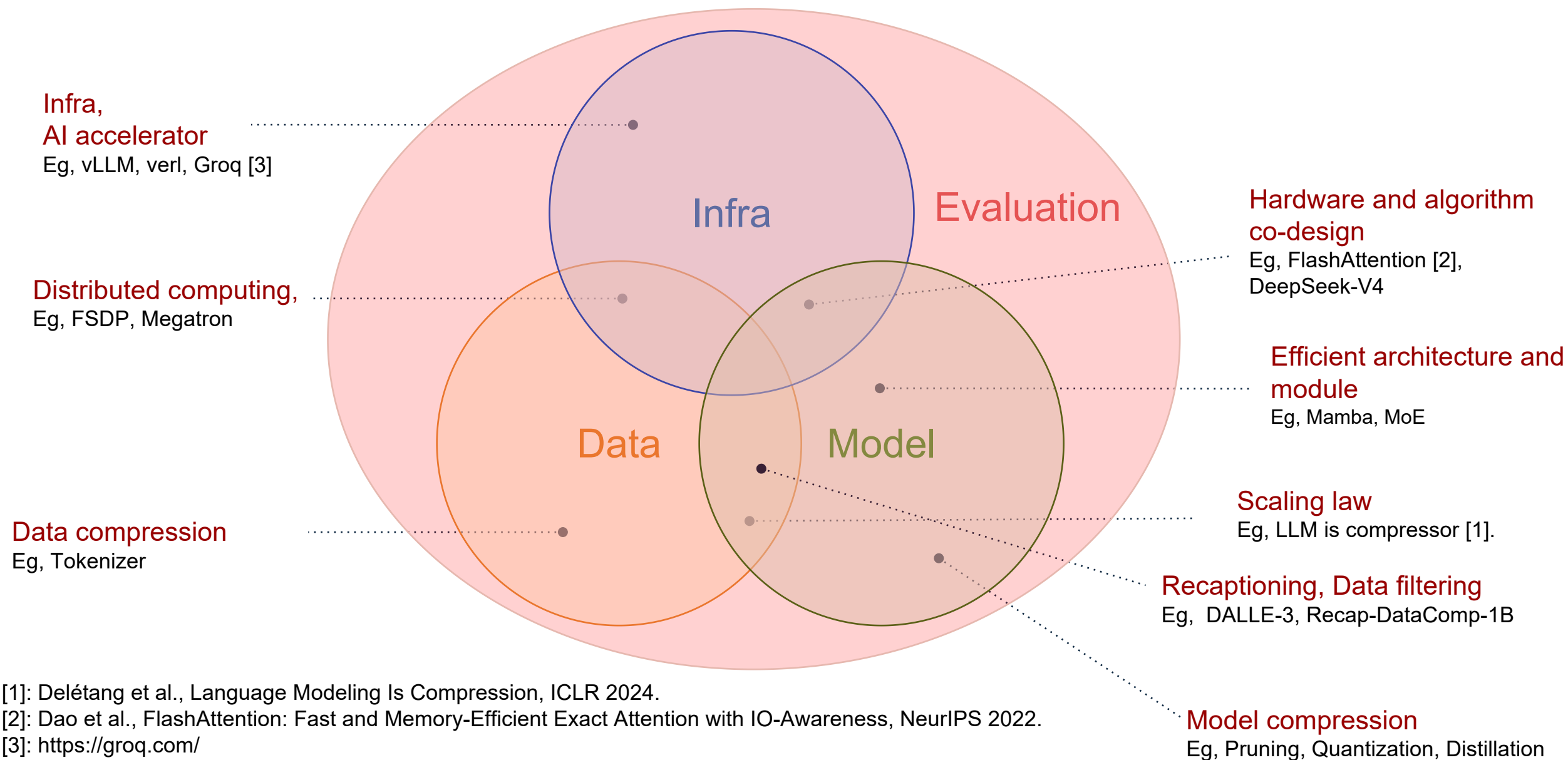


Self-Evolving Data Flywheel

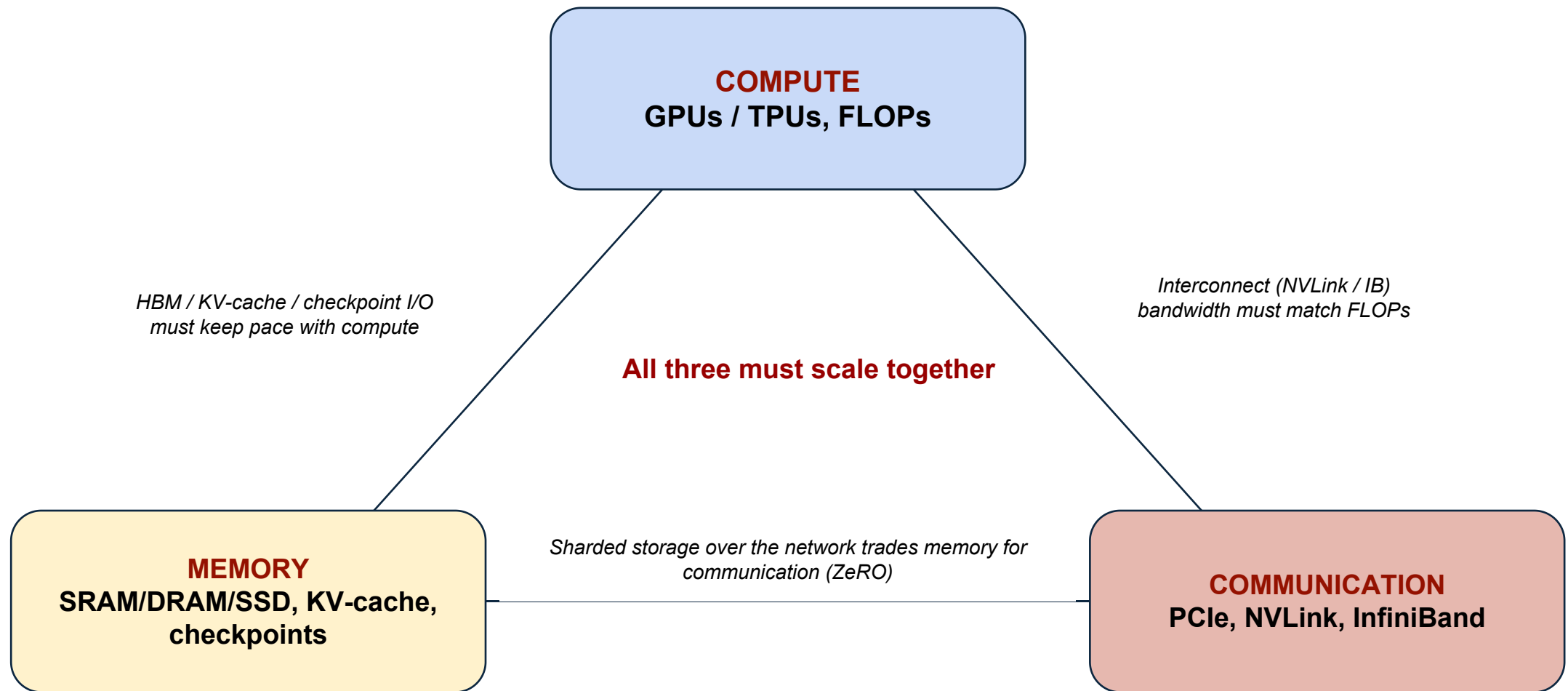


More agent interactions → more data → better models → more capable agents: pre-training, test-time, and the data flywheel compound into accelerating growth.

Background - The Four Elements of Efficiency



The Triangle in ML Sys -- Compute, Memory, and Communication



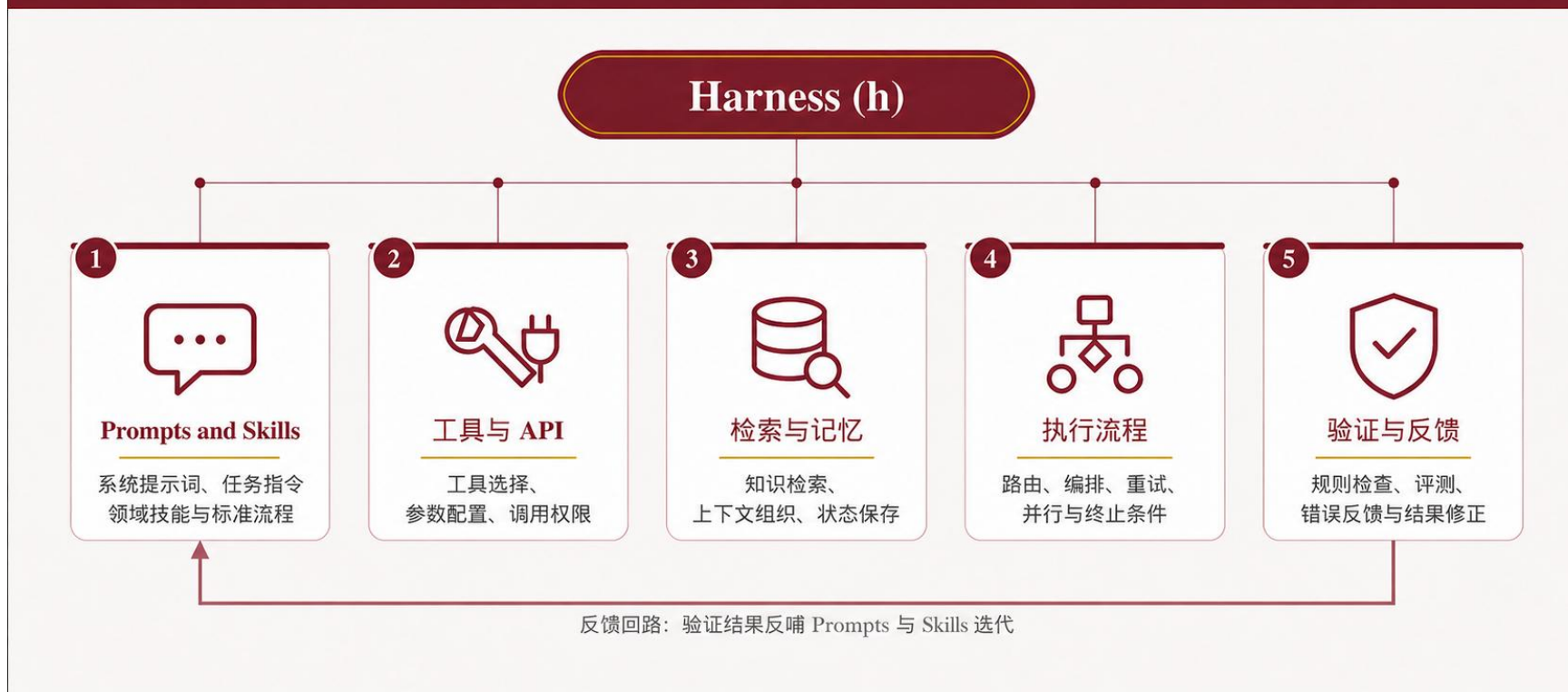
A bottleneck in any one of compute, storage, or communication throttles the whole system.

What are Agent and Harness?

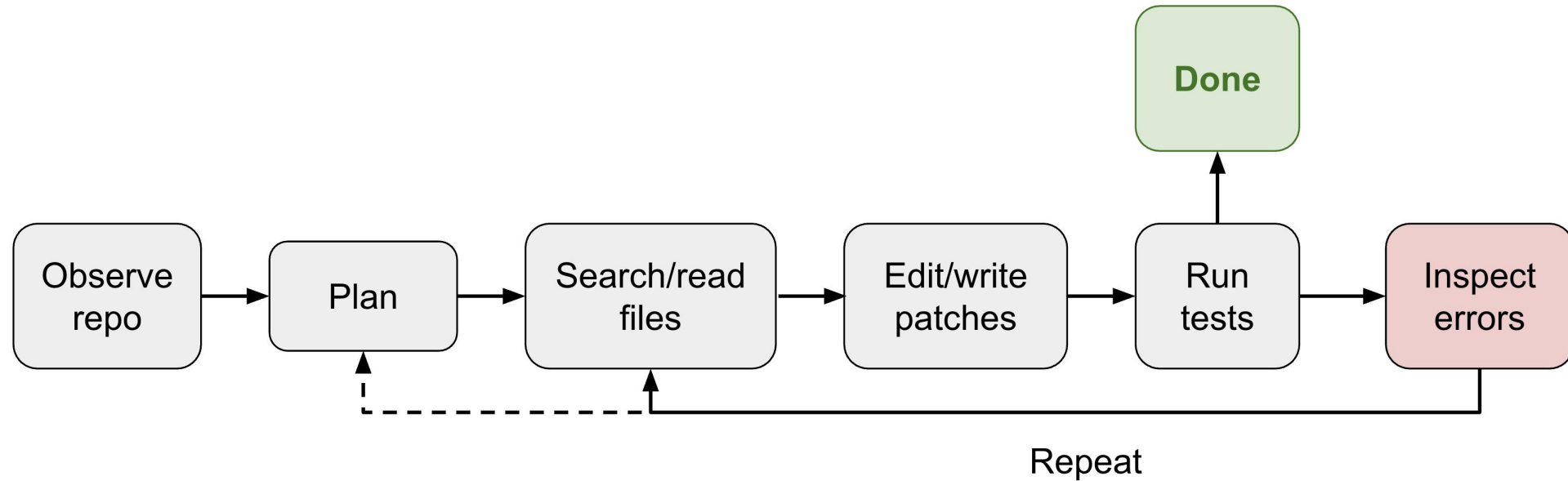
- **Agent = Model + Harness**
- **Code is a universal language for defining programs and systems**
- **A Harness is code that programs how prompts, tool calls, subagents, control flow, memory, and workflow logic work together.**

Harness 包含什么？

围绕模型构建的可观察、可调试、可快速迭代的系统配置



Case study: Coding Agent Harness



The core interface of mainstream coding agents across Claude Code, Codex, OpenCode, and Cursor-style agents

Bi-level Optimization: Harness-Level & Model-Level

$$\begin{aligned} \max_{\theta} \quad & J(\theta, h^*(\theta)) \\ \text{s.t.} \quad & h^*(\theta) = \arg \max_h J(\theta, h) \end{aligned}$$

θ

θ : black-box model 参数 (慢, 隔很久 update 一次)

h

h : white-box harness 配置 (快, 频繁迭代)

$J(\theta, h)$

$J(\theta, h)$: 部署后的 system 在任务上的期望 reward

🕒 异步实现: harness 快, model 慢



快

每一步:

$$h \leftarrow \arg \max_h J(\theta, h) \quad (\theta \text{ 冻结})$$



慢

每 $T \gg 1$ 步:

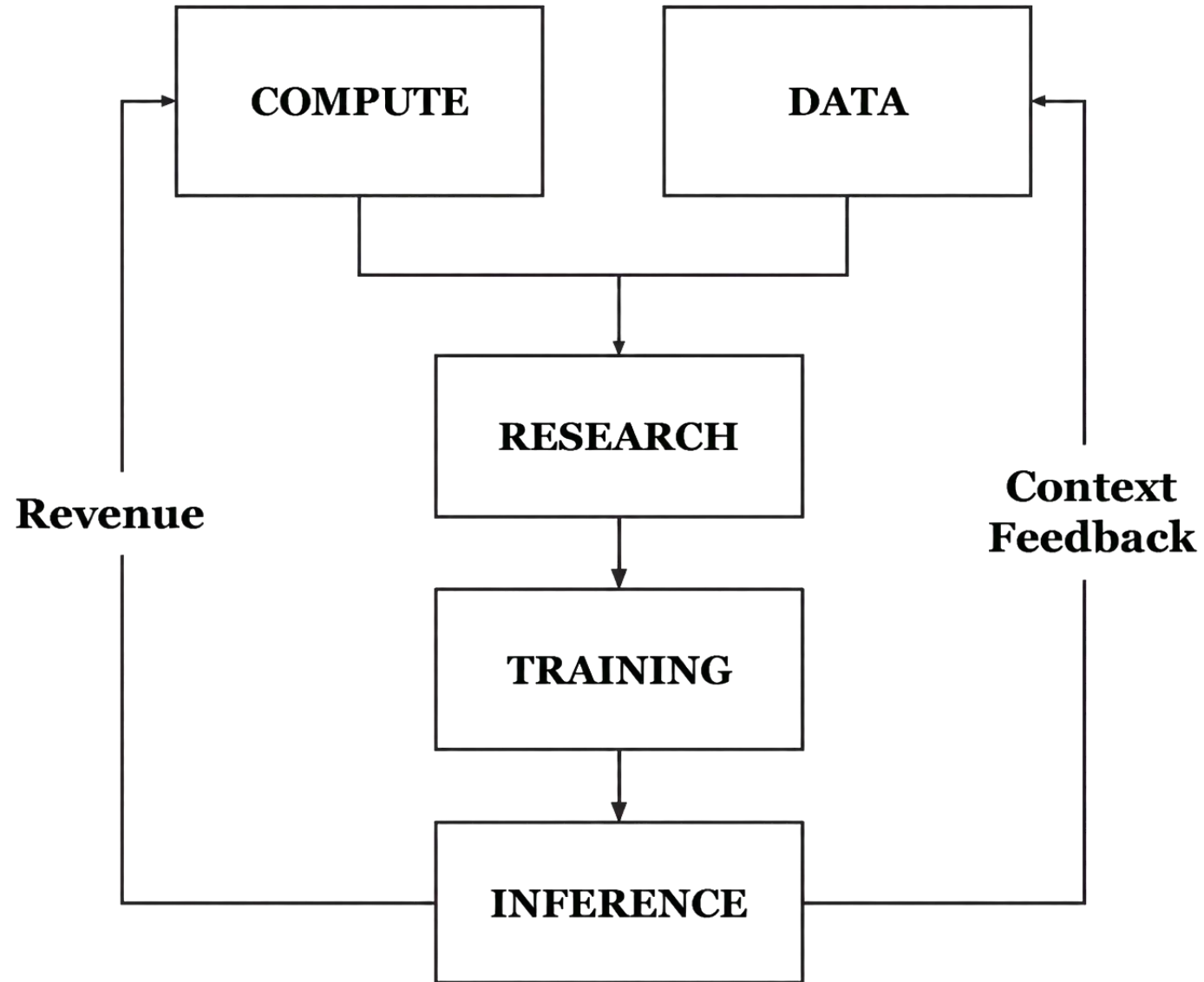
$$\theta \leftarrow \theta + \eta \nabla_{\theta} J(\theta, h)$$



inner loop (harness) 反复运行直至收敛; outer loop (model) 间隔较久才更新一次。

- Harness-level (white-box) optimization is fast and cheap; model-level (black-box) updates are slow and expensive

An Interesting Loop



Task Categories

Text-Only LLM

Coding & Chatbot

- DeepSeek-V3/V4

Multimodal Understanding

Perception & reasoning

- GPT-5.6 (vision)
- Gemini 3
- Qwen3-VL
- Claude Sonnet 5
- LLaVA-Next

Multimodal Generation

Image & video

- Nano Banana pro
- Nano Banana pro
- FLUX.2
- Midjourney v7
- Seedance 2.0
- Kling 3.0
- Wan 2.6
- Hunyuan-World

From pure text to multimodal understanding and generation — the task frontier keeps expanding outward.

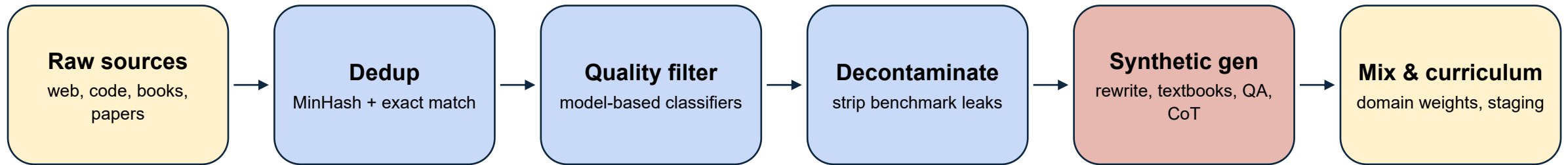
Unified (Omni) Model

<https://research.nvidia.com/labs/cosmos-lab/cosmos3/>

Part 2 – Training

Understanding Training from the Beauty of Data, Algorithm & Infra Co-design

Training - Data Curation Pipeline



Pre-training data mix at the frontier

Synthetic data >90%

<10%

curated real data

Data curation is now model-in-the-loop: LLMs filter, rewrite, and generate most of what the next model learns.

Training - Data Curation

- Data efficiency is highly important.
- Data quality, diversity and quantity matter.
- Tokenizer is very important.

GPT-4o & GPT-4o mini

GPT-3.5 & GPT-4

GPT-3 (Legacy)

Graphic User interface (GUI) 123@f !automation requires agents with the ability to understand and interact with user screens. However, using general purpose LLM models to serve as GUI agents faces several challenges: もちろん、こちらに簡単な日本語の段落を書いてみます; 2) understanding the semantics of various elements in a screenshot and accurately associating the intended action with the corresponding region on the screen. OmniParser closes this gap by 'tokenizing' UI 春の訪れとともに、公園の。structured elements in the screenshot that are interpretable by LLMs. This enables the LLMs to do retrieval based next action prediction given a set of parsed interactable elements.

Clear

Show example

Tokens

146

Characters

646

Graphic User interface (GUI) 123@f !automation requires agents with the ability to understand and interact with user screens. However, using general purpose LLM models to serve as GUI agents faces several challenges: もちろん、こちらに簡単な日本語の段落を書いてみます; 2) understanding the semantics of various elements in a screenshot and accurately associating the intended action with the corresponding region on the screen. OmniParser closes this gap by 'tokenizing' UI 春の訪れとともに、公園の。structured elements in the screenshot that are interpretable by LLMs. This enables the LLMs to do retrieval based next action prediction given a set of parsed interactable elements.

Text

Token IDs

Optimizers: from Adam to Muon

AdamW - the default workhorse

Adaptive per-parameter step size from gradient moments

$$m \leftarrow \beta_1 m + (1 - \beta_1)g \quad v \leftarrow \beta_2 v + (1 - \beta_2)g^2$$

$$\theta \leftarrow \theta - \eta \cdot m / (\sqrt{v} + \varepsilon), \text{ decoupled weight decay}$$

Robust across scales - the default since GPT-3

Muon - matrix-aware optimization

orthogonalizes the momentum update via Newton–Schulz iteration

≈2× compute efficiency: AdamW-level loss at ~52% of the training FLOPs

MuonClip stabilized Kimi K2: 1T-param MoE, 15.5T tokens, zero loss spikes

A better optimizer is free compute: Muon reaches AdamW-quality loss with roughly half the training FLOPs.

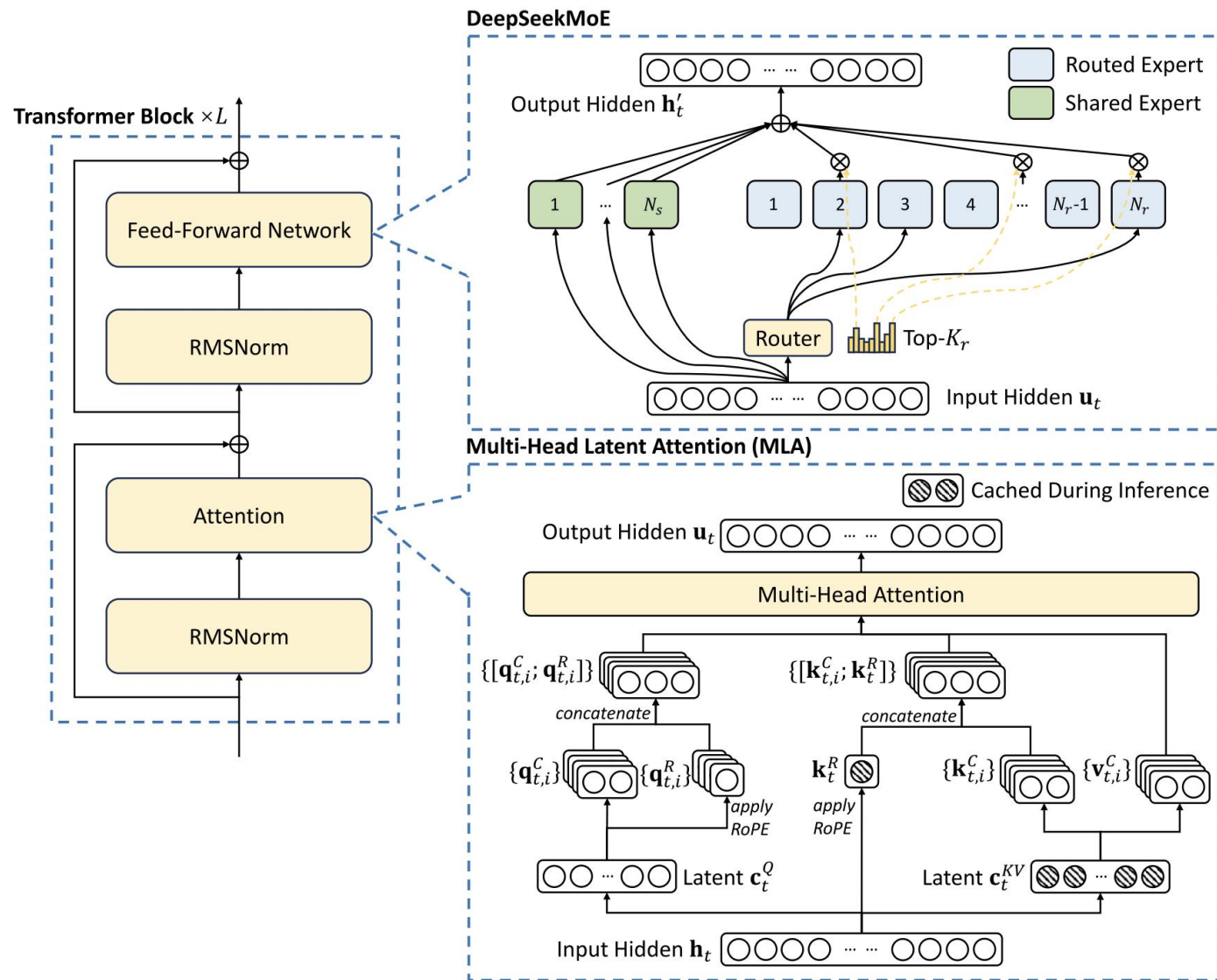
Go deeper - optimization resources

Convex optimization: Boyd & Vandenberghe, Convex Optimization

Stochastic optimization: Bottou, Curtis & Nocedal, Optimization Methods for Large-Scale Machine Learning, SIAM Review 2018

Muon in practice: Keller Jordan's Muon blog (2024); Moonshot AI, Muon is Scalable for LLM Training (2025)

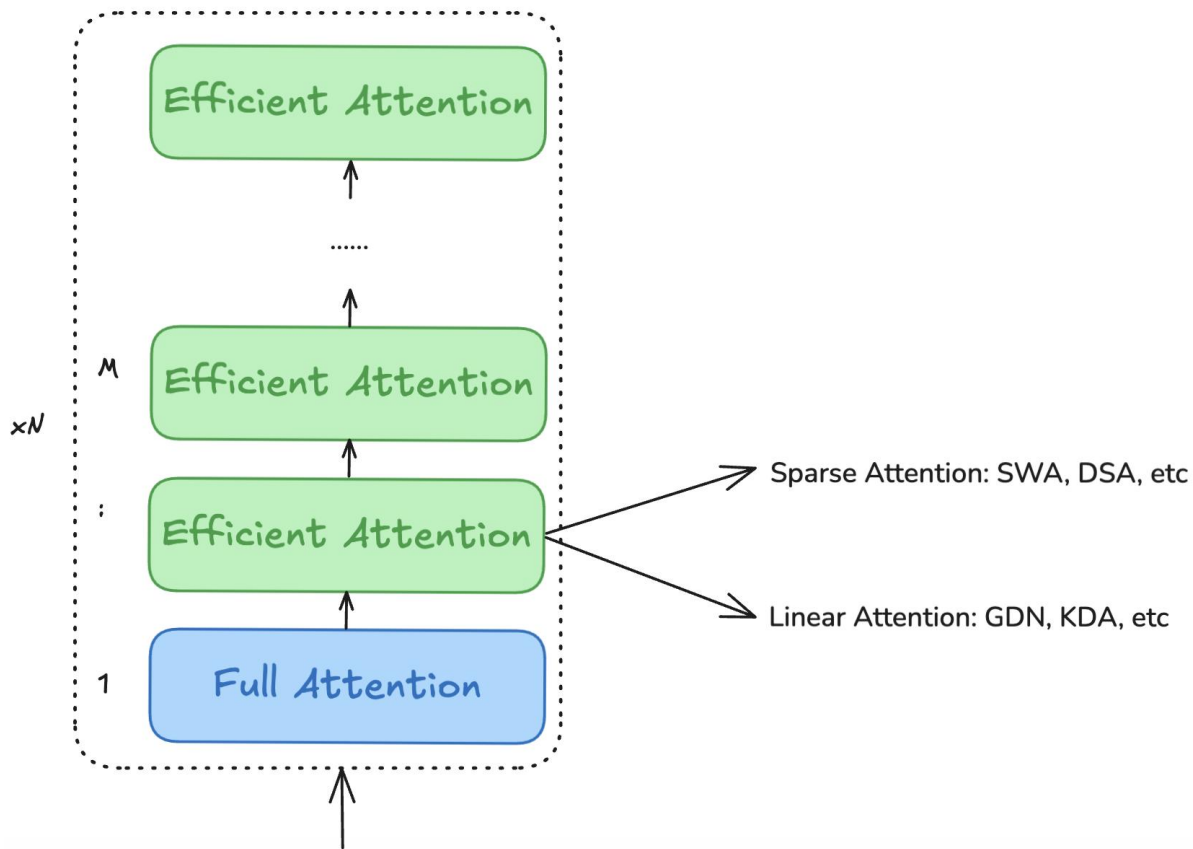
Architecture



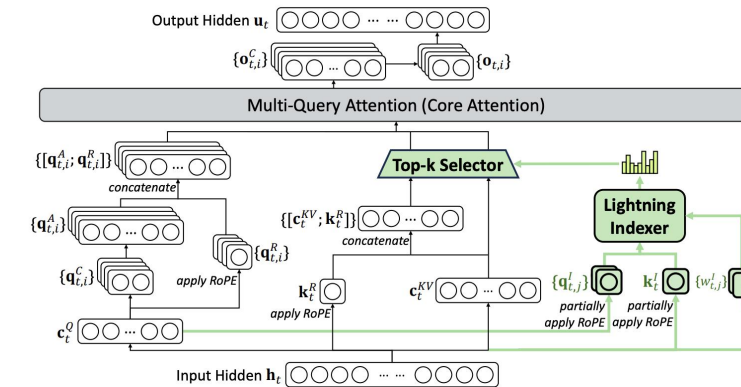
Efficient Training & Inference:

- dynamically activate a few experts for each token
- 671B parameters, of which 37B are activated for each token.

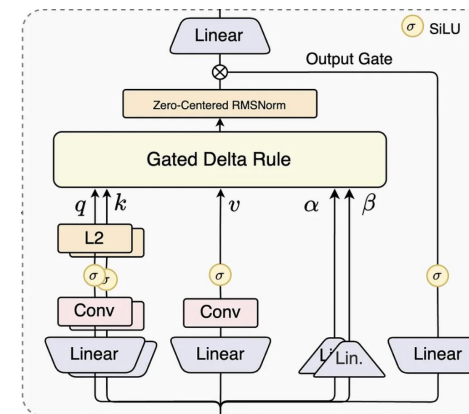
Architecture



DSA



GDN



Hybrid Architecture (国内):

- **Two hybrid strategies:** 1. Full Attention with Sparse Attention (SWA, DSA, etc). 2. Full Attention with Linear Attention (GDN, KDA, etc)
- Reduce the memory and computation overhead brought by long context

Efficient Training - Parallelism Strategies

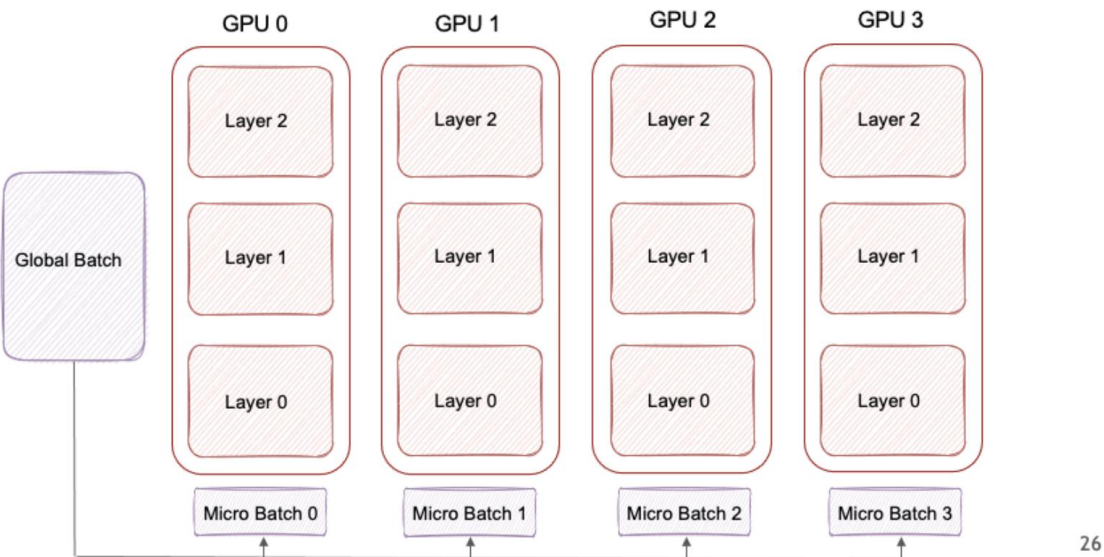
并行策略	切分对象	优点	缺点
数据并行 (DP / ZeRO / FSDP)	batch 切到各 GPU; ZeRO/FSDP 进一步切分参数、梯度、优化器状态	简单;吞吐近线性扩展;无需改模型代码	显存冗余(除非分片); all-reduce 通信量随模型增大
张量并行 (TP)	层内权重矩阵	可训单卡放不下的大层;降低单卡显存与延迟	每层都要通信,需 NVLink 级带宽;一般限节点内
流水线并行 (PP)	按层切成多个 stage	通信量小;可跨节点扩展深度	流水线气泡;需 micro-batch 调度(1F1B、interleaved);stage 负载均衡难
专家并行 (EP)	MoE 专家分布到各 GPU	参数规模扩展而 FLOPs/token 基本不变;天然适配 MoE	all-to-all 路由通信;专家负载不均
序列/上下文并行 (CP)	序列维度(Ring / Ulysses attention)	支持百万 token 上下文;切分激活显存	额外 attention 通信; KV 交换复杂

Frontier runs compose them: DP × TP × PP × EP (+ CP) - “4D/5D parallelism” tuned to the interconnect topology.

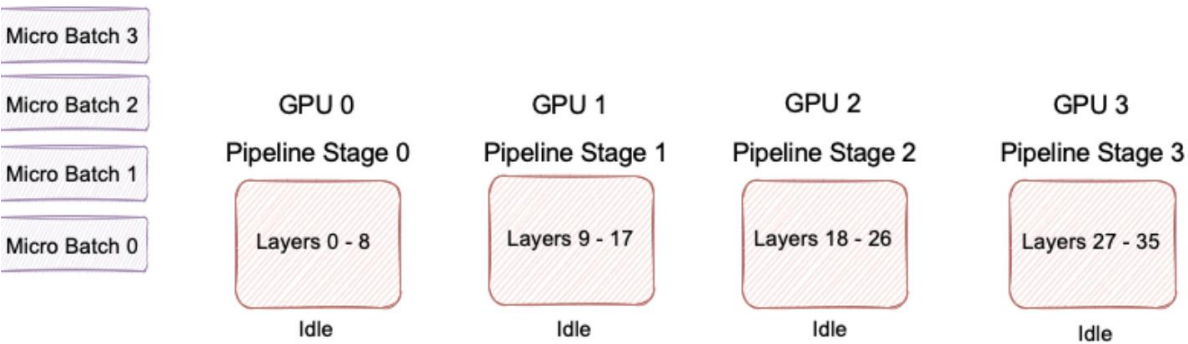
[1] Shoeybi et al., Megatron-LM, 2019. [2] Rajbhandari et al., ZeRO, SC 2020. [3] Zhao et al., PyTorch FSDP, VLDB 2023. [4] Lepikhin et al., GShard, ICLR 2021.

Efficient Training - Parallelism Strategies

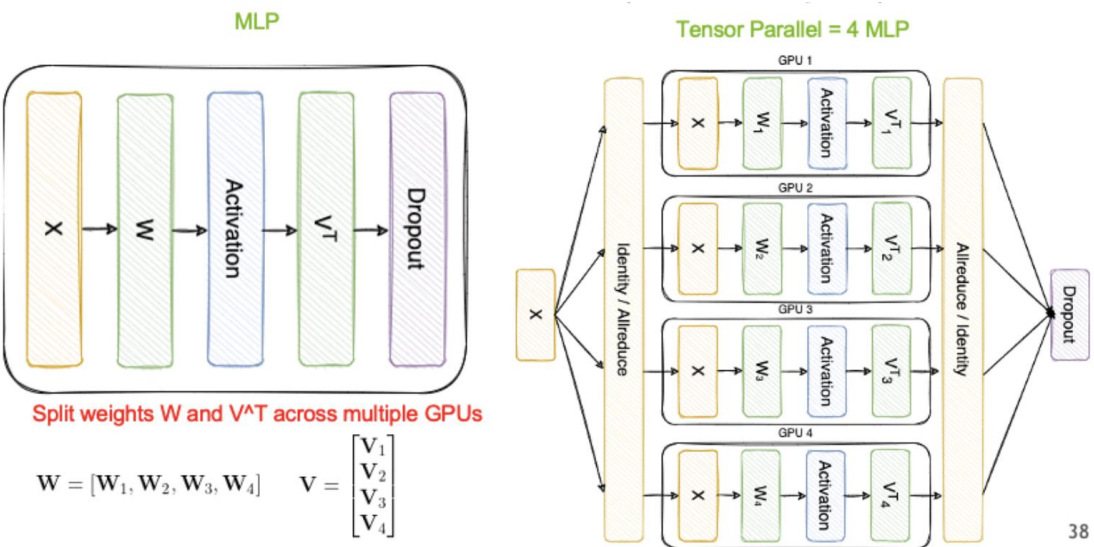
DP: Split across batch dimension



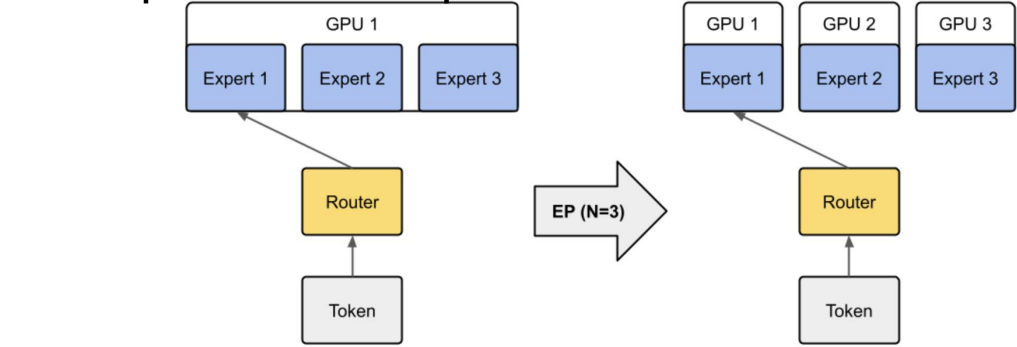
PP: Split across model layers



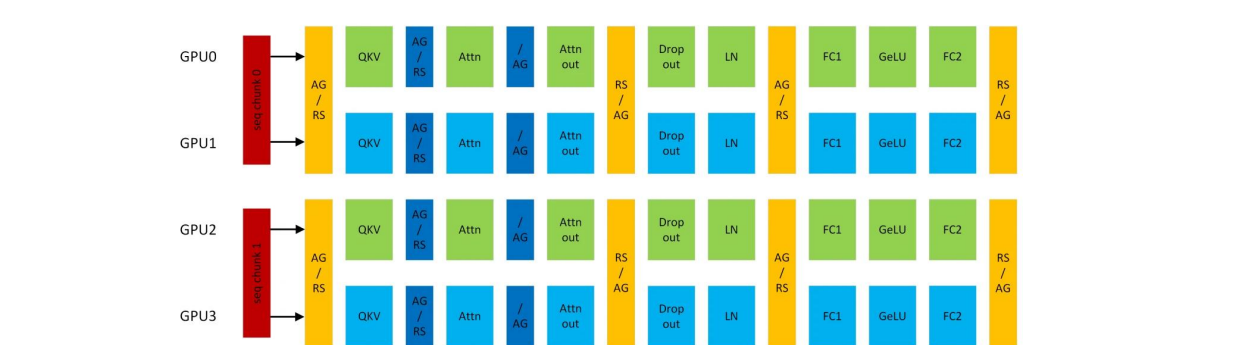
TP: Split every weight / activation matrix



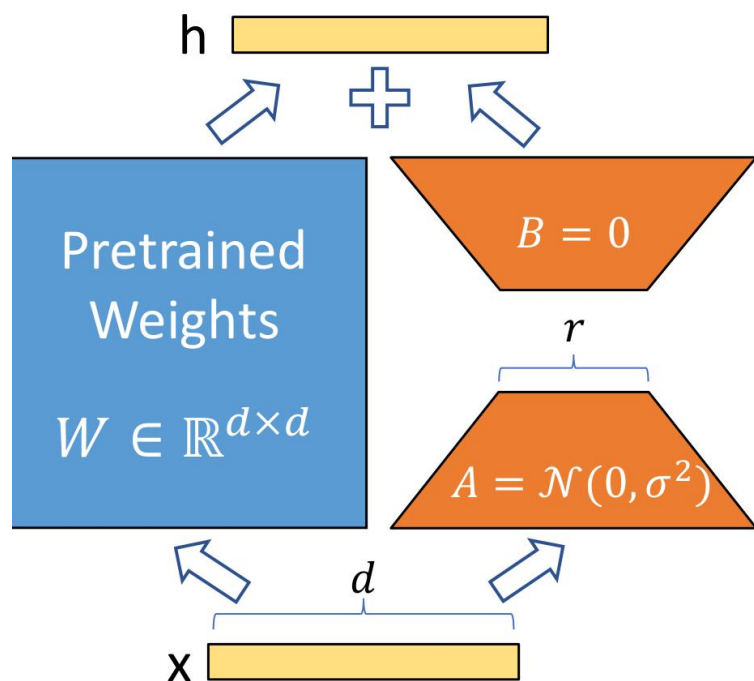
EP: Split across experts



CP: Split across sequence (Ring) or attention head (Ulysses)



Parameter-Efficient Fine-Tuning - LoRA & QLoRA



LoRA reparametrization: only A and B are trained [1]

LoRA: freeze pretrained W_0 ; learn a low-rank update $\Delta W = BA$ with rank $r \ll d$, so $h = W_0x + BAx$

Up to 10,000× fewer trainable parameters and 3× less GPU memory (GPT-3 175B)

Merge $W_0 + BA$ after training → **zero extra inference latency**

QLoRA: LoRA on a 4-bit quantized base - fine-tune a 65B model on one 48GB GPU

$$\begin{aligned} h &= xW' = x(W + AB) \\ &= xW + xAB. \end{aligned}$$

Adapt a large model by training <1% of its weights - near full fine-tuning quality at a fraction of the cost.

Pre-training Frameworks - FSDP, Megatron, DeepSpeed

框架	核心思路	优点	缺点
PyTorch FSDP2	原生全分片: 参数 / 梯度 / 优化器状态 (ZeRO-3 式, 基于DTensor)	PyTorch 原生;与 torch.compile、TP、激活重算可组合; 上手容易	无内置流水线并行; fused kernel 少; 超大规模需自行调优
Megatron-LM / Megatron Core	NVIDIA 全栈:TP / PP / DP / EP / CP + fused kernel + FP8 Transformer Engine	前沿规模 MFU 最高;100B-1T 模型久经考验 (Nemotron、GLM 等)	学习曲线陡, 代码屎山; 绑定 NVIDIA
DeepSpeed	ZeRO-1/2/3 + CPU/NVMe offload,MoE 与流水线引擎	功能全;靠 offload 用小集群训大模型;与 HF 集成顺滑	迭代放缓;配置繁杂;前沿规模性能落后 Megatron

Megatron for max MFU at scale; FSDP / TorchTitan for PyTorch-native flexibility; DeepSpeed for tight memory.

[1] Zhao et al., PyTorch FSDP, VLDB 2023. [2] Shoeybi et al., Megatron-LM, arXiv 2019. [3] Rasley et al., DeepSpeed, KDD 2020. [4] Liang et al., TorchTitan, ICLR 2025.

Post-training - SFT, RL, and On-Policy Distillation

Typical share of post-training effort at frontier labs



方法	目的	数据来源	目标分布	训练信号	On-policy?	遗忘 / 成本
SFT	注入新知识与格式，快速建立基础能力	外部数据集	固定的外部数据集，把模型拉向它	稠密：逐 token 交叉熵 (Forward KL)	否	高 / 低
Offline RL DPO 等	用固定偏好数据做对齐，简单便宜	固定的偏好数据集	偏好数据中的最优策略	成对偏好 / 隐式 reward	否	中 / 低
Online RL PPO / GRPO	在可验证任务上探索，”提升推理与 Agent 能力”	当前 policy rollout	当前模型附近的 reward 最优策略	稀疏：只有 outcome 级 reward (Reverse KL)	是	低 / 高
OPD	合并多个teachers的能力	学生采样轨迹	teacher 在学生自采样数据上的 logits	稠密：逐 token 对齐 teacher (Reverse KL)	是	低 / 中

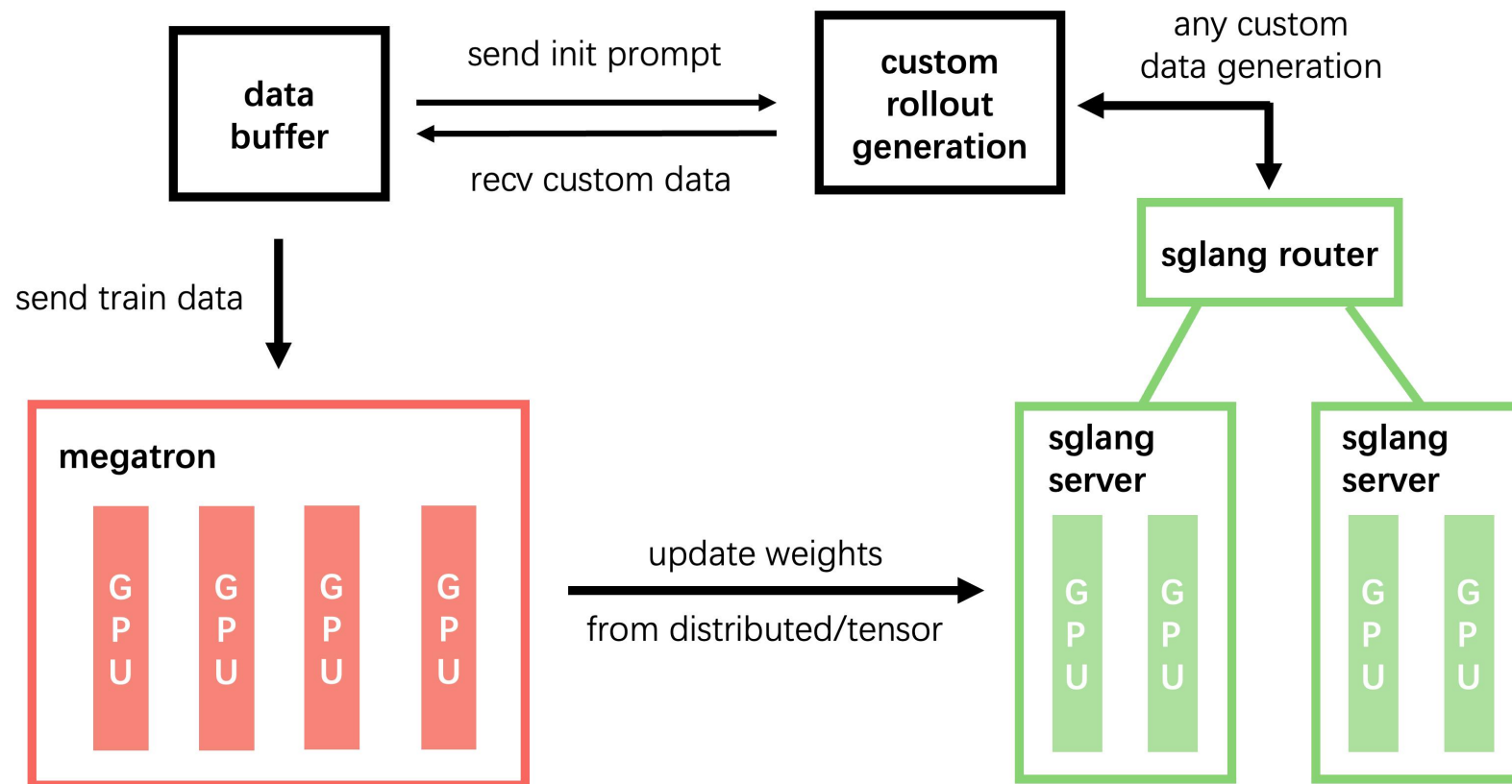
On-policy data is the key variable: training on the model's own samples keeps updates near the current policy and prevents catastrophic forgetting [1].

[1] nrehiew, A Distributional View of Post-Training: SFT, RL, OPD, 2025. https://nrehiew.github.io/blog/sft_rl_opd/

Scalable RL infra

System	Scheduler	Inference	Training	Characters
VeRL (HybridFlow)	Ray	vLLM/SGLang	FSDP/Megatron	最大的生态
OpenRLHF	Ray	vLLM	DeepSpeed	最早在RL中采用Ray架构和vLLM推理加速
ARes (RealHF)	Custom	SGLang	Megatron	截断式Rollout, 纯异步RL更新
ROLL	Ray	vLLM/SGLang	DeepSpeed/Megatron	专注于Agent多轮function calling能力
Slime	Ray	SGLang	Megatron	极简, 超Lightweight
Nemo RL	Ray	vLLM	Megatron	Nvidia原汁原味

Slime – Minimalist Engineering Aesthetics



slime = Megatron training + SGLang rollout + data buffer: sync or fully async agentic RL (used for GLM-4 and 5).

Training Infra vs. Inference Infra

负载维度	训练引擎	推理引擎
计算路径	Forward + Backward + Optimizer	Forward: Prefill + 逐 Token Decode
关键状态	参数、梯度、优化器状态、Activation	参数 + 按请求增长的 KV Cache
Batch / 形状	大批次、形状较稳定、全局同步	动态长度、持续批处理、请求进出频繁
时间模式	离线、同步、按 Iteration 推进	在线、异步、请求持续到达
多卡通信	梯度 All-Reduce / RS+AG; 流水线 P2P	TP / MoE 集合通信; 无梯度同步
典型瓶颈	算力、显存容量与互联带宽	Prefill 偏算力; Decode 偏显存带宽
核心指标	训练吞吐、MFU、Time-to-Train	TTFT、TPOT、吞吐、并发与 SLO

目标：训练以同步迭代更新参数；推理以动态调度持续服务请求

训练引擎：算力 + 显存 + 通信，围绕稳定大批次更新参数

推理引擎：Prefill 算力 + Decode 带宽 + 请求调度，围绕吞吐、延迟和并发之间权衡

负载结构决定优化的关键指标：训练看吞吐、MFU、训练时长；推理看 TTFT、TPOT、吞吐与并发

Agentic RL Infra

维度	Agentic RL infra
Rollout	多轮循环:LLM ↔ 工具 / 环境, 长时程交互，长尾严重
Reward	环境结果:测试通过、任务完成
执行方式	异步、解耦的 rollout 服务; 训练和rollout分离; 支持 partial rollout
瓶颈	长尾轨迹 + 海量并发工具调用拖停 GPU
环境	sandbox、浏览器、代码执行器:长生命周期、I/O 密集、需独立扩缩
权重同步	trainer → 推理引擎跨 GPU 池频繁同步

Agentic RL is a distributed-systems problem: decouple rollout from training and keep GPUs busy.

[1] OpenRLHF, Asynchronous RL and Agent RL (docs). [2] Shang et al., rStar2-Agent Technical Report, arXiv 2025.

Part 3 - Inference

Understanding Inference from the Beauty of Algorithm & Infra Co-design

Background - Self-Attention in Transformers

Obtain queries, keys and values with three learnable projection matrices.

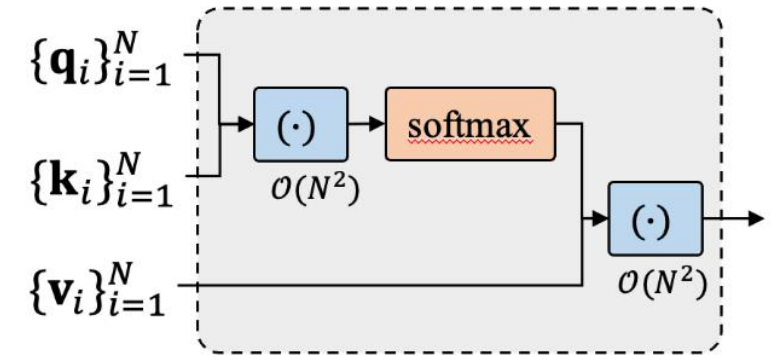
$$\{\mathbf{q}_t\}_{t=1}^N = \mathbf{X}\mathbf{W}_q, \{\mathbf{k}_t\}_{t=1}^N = \mathbf{X}\mathbf{W}_k, \{\mathbf{v}_t\}_{t=1}^N = \mathbf{X}\mathbf{W}_v.$$

Compute the attention as

$$\text{Attention}(\mathbf{q}_t, \{\mathbf{k}_i\}, \{\mathbf{v}_i\}) = \sum_i \frac{\exp(\mathbf{q}_t \cdot \mathbf{k}_i / \tau)}{\sum_j \exp(\mathbf{q}_t \cdot \mathbf{k}_j / \tau)} \mathbf{v}_i.$$

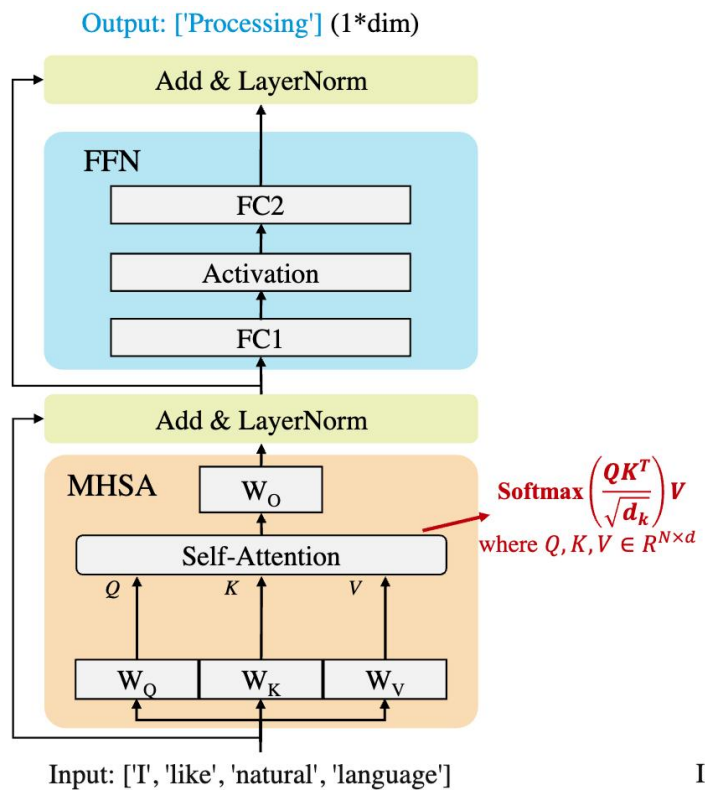
Limitation: The softmax attention is quadratic time complexity $\mathcal{O}(N^2)$ regarding the number of tokens N .

■ Multiply-accumulate



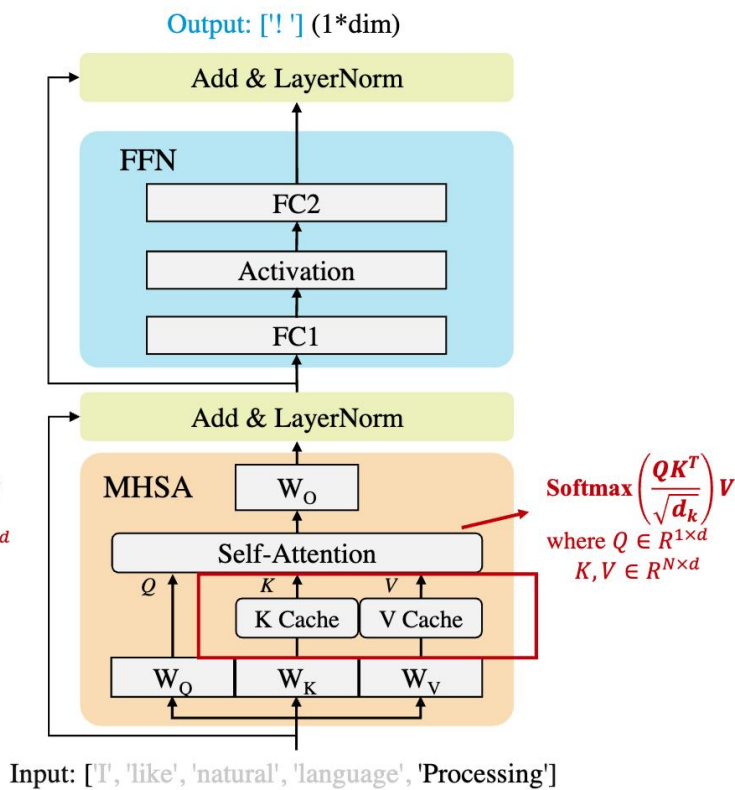
Inference Latency and Memory of LLMs

Computation-bound

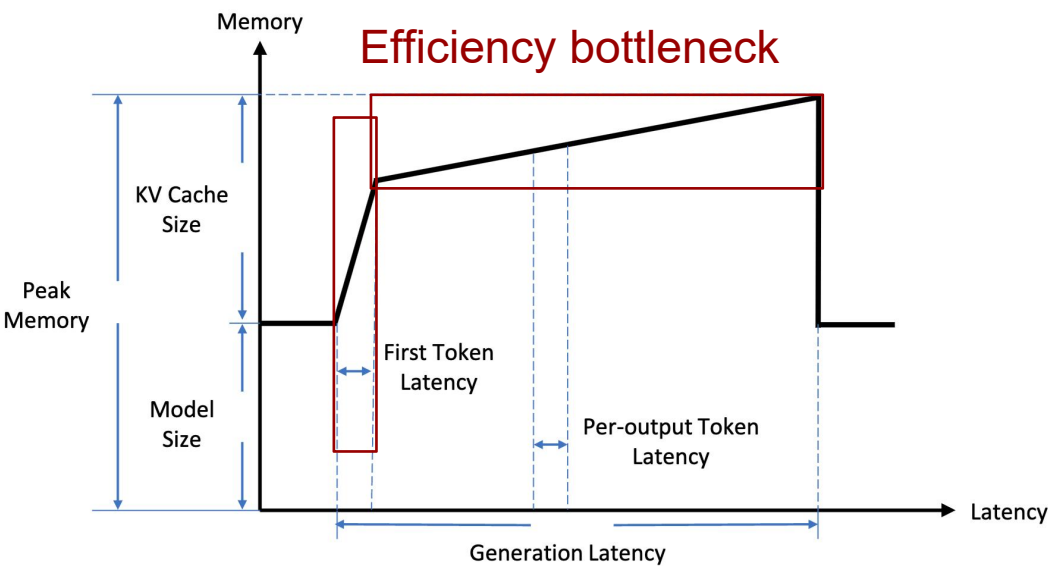


(a) The prefilling stage

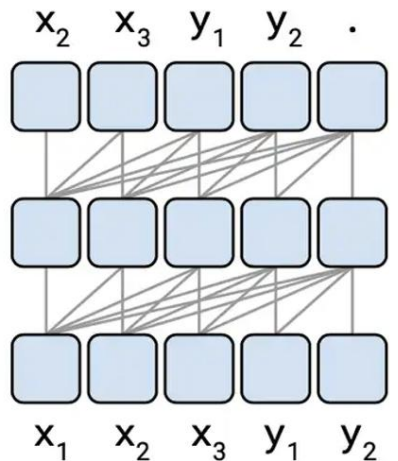
Memory-bound



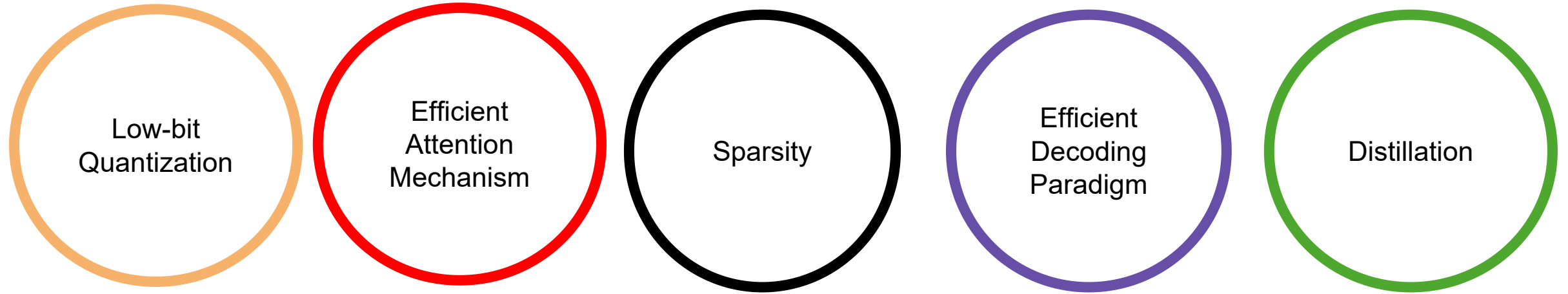
(b) The decoding stage



Language model



Efficient Inference - Overview

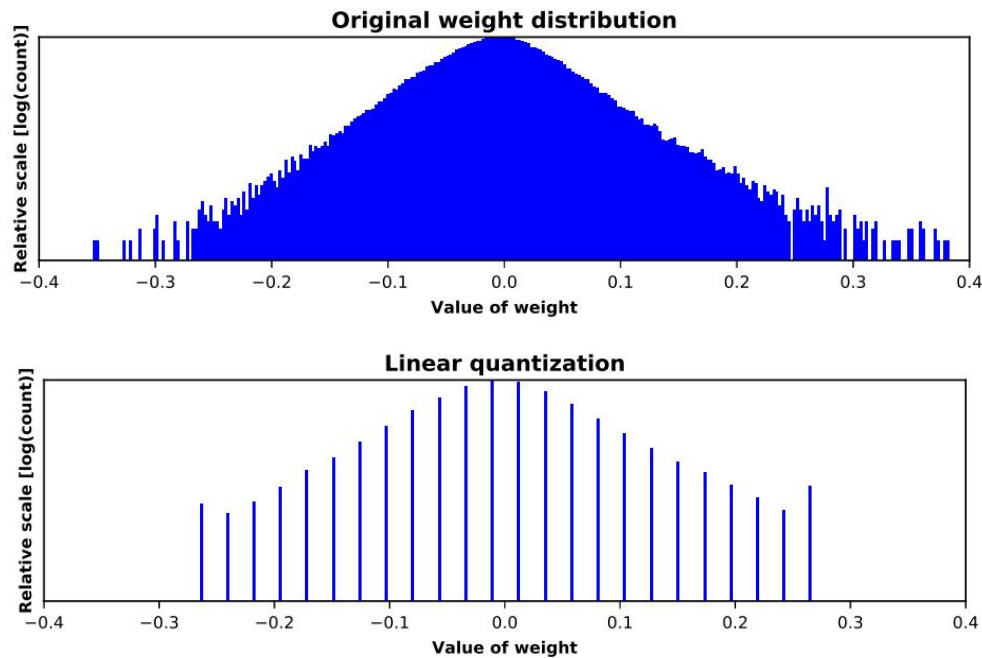


Principle: Algorithm and Infra Co-design

Inference: low-bit quantization

Low-bit Quantization Background

Neural network quantization maps high-precision weights and activations to low-bit representations—such as INT8/INT4 or FP8/FP4—reducing memory footprint and bandwidth while enabling matrix multiplications to run efficiently on low-precision integer or floating-point hardware.



Lower Power Consumption



Lower Storage



Lower Latency



Lower Communication
Bandwidth

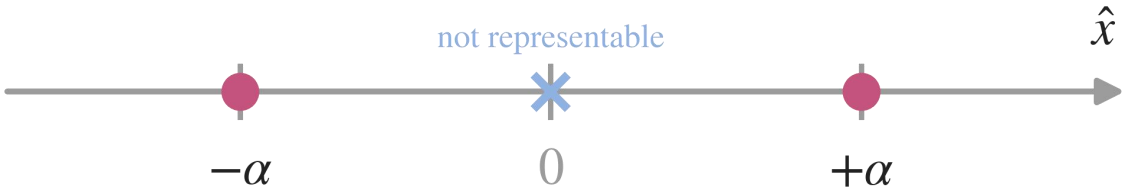
Quantization Formats: Binary/Ternary vs. Integer vs. Floating-Point

Binary (1-bit): a single scale $\alpha \times$ the sign

$$\hat{x} = \alpha \operatorname{sign}(x), \quad \operatorname{sign}(x) \in \{-1, +1\}$$

$$\alpha^* = \arg \min_{\alpha} \|x - \alpha \operatorname{sign}(x)\|^2 = \frac{1}{n} \|x\|_1$$

2 levels: $\{-\alpha, +\alpha\}$ — one sign bit

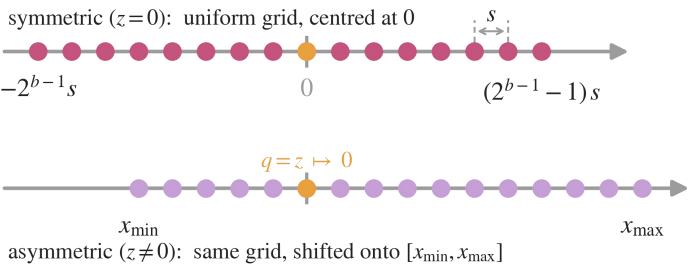


Fixed-point INT-b: uniform grid — zero-point-free (symmetric) vs. zero-point (asymmetric)

$$x_q = \operatorname{clip}(\lfloor x/s \rfloor, -2^{b-1}, 2^{b-1} - 1), \quad \hat{x} = s x_q, \quad s = \frac{\max |x|}{2^{b-1} - 1}$$

$$x_q = \operatorname{clip}(\lfloor x/s \rfloor + z, 0, 2^b - 1), \quad \hat{x} = s(x_q - z)$$

$$s = \frac{x_{\max} - x_{\min}}{2^b - 1}, \quad z = \lfloor -x_{\min}/s \rfloor$$



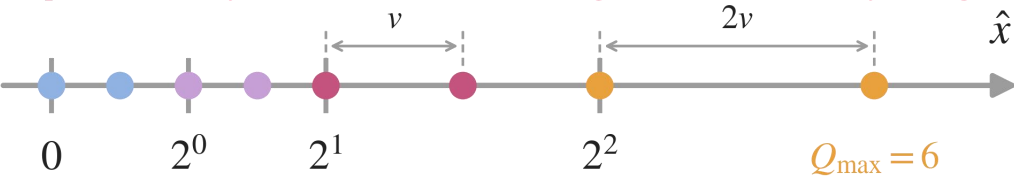
Floating-point FP4 (NVFP4, E2M1): logarithmic grid + two-level scaling

$$\hat{x} = (-1)^S 2^{E - \text{bias}} \left(1 + \frac{M}{2^m}\right), \quad \text{E2M1} : \{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$$

$$Q_{\max} = (2 - 2^{-m}) 2^{E_{\max} - \text{bias}} = 1.5 \times 4 = 6$$

$$\text{NVFP4} : \hat{x} = s_{\text{tensor}}^{\text{FP32}} s_{\text{block}}^{\text{E4M3}} Q_{\text{E2M1}}\left(\frac{x}{s_{\text{tensor}} s_{\text{block}}}\right), \text{ block} = 16$$

step doubles every binade — dense near 0 (negative side mirrored by the sign bit)



1 bit keeps only the sign; INT spreads its 2^b levels uniformly (the zero-point z shifts the grid); FP spreads them logarithmically — dense near zero, where values cluster.

[1] Rastegari et al., XNOR-Net, ECCV 2016. [2] Jacob et al., Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, CVPR 2018. [3] NVIDIA, Pretraining Large Language Models with NVFP4, 2025.

Example: Integer-Only Networks & Ternary Networks for Edge Devices

AQD — fully-quantized object detection (CVPR 2021 Oral)

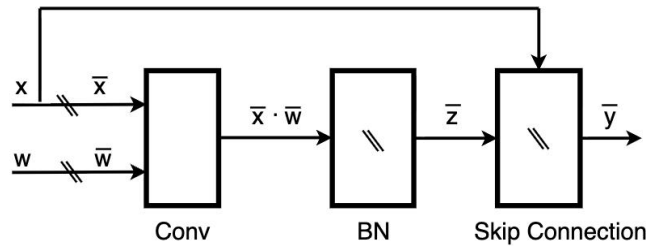
Pure integer network: conv, BN, and detection heads all run in fixed-point — no float ops left at inference

Huge Kernel

Result: 4-bit FCOS matches its full-precision counterpart on COCO

Table 1 – Energy and Area cost for different precision operations (on 45nm CMOS technology) [9, 14, 25].

Operation	Energy (pJ)	Area (μm^2)
16-bit Floating-point Add	0.4	1360
16-bit Floating-point Mult	1.1	1640
32-bit Floating-point Add	0.9	4184
32-bit Floating-point Mult	3.7	7700
8-bit Fixed-point Add	0.03	36
8-bit Fixed-point Mult	0.2	282
32-bit Fixed-point Add	0.1	137
32-bit Fixed-point Mult	3.1	3495



FATNN — fast & accurate ternary networks (ICCV 2021)

Ternary $\{-1, 0, +1\}$: inner product re-designed into two parallel bitwise ops — 2× lower complexity than 2-bit fixed-point

Same energy budget, more accuracy: closes the gap to full precision on ImageNet

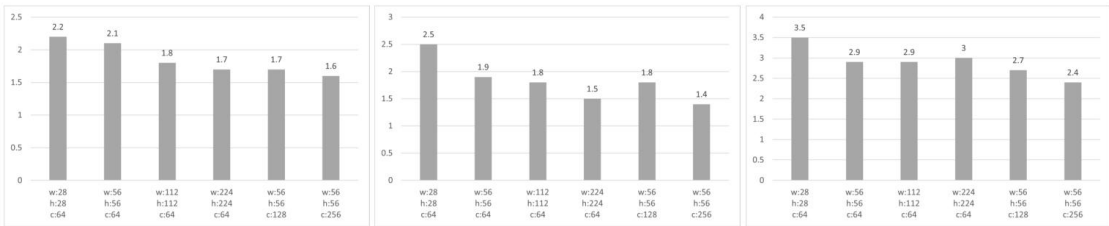
Half a year C++ and OpenCL implementation: for binary/ternary/2-bit fixed-point quant

x_c	y_c	z_c
-1(2'b00)	-1(2'b00)	1(2'b11)
-1(2'b00)	0(2'b01)	0(2'b10)
-1(2'b00)	0(2'b10)	0(2'b01)
-1(2'b00)	1(2'b11)	-1(2'b00)
0(2'b01)	-1(2'b00)	0(2'b10)
0(2'b01)	0(2'b01)	0(2'b01)
0(2'b01)	0(2'b10)	0(2'b01)
0(2'b01)	1(2'b11)	0(2'b01)
0(2'b10)	-1(2'b00)	0(2'b01)
0(2'b10)	0(2'b01)	0(2'b10)
0(2'b10)	0(2'b10)	0(2'b10)
0(2'b10)	1(2'b11)	0(2'b10)
1(2'b11)	-1(2'b00)	-1(2'b00)
1(2'b11)	0(2'b01)	0(2'b01)
1(2'b11)	0(2'b10)	0(2'b10)
1(2'b11)	1(2'b11)	1(2'b11)

$$switch = ((y_c >> 1) \& auxi) \mid ((y_c << 1) \& \sim auxi), \tag{5}$$

$$mask = switch \wedge y_c, \tag{6}$$

$$TM(\cdot) = (mask \& auxi) \mid ((\sim mask) \& xnor).$$



(a) Speedup on Qualcomm 821 GPU

(b) Speedup on Qualcomm 835 GPU

(c) Speedup on Nvidia 2080Ti GPU

Layer-wise speedup: ternary (FATNN) vs. 2-bit, on Qualcomm 821 / 835 and RTX 2080 Ti

Integer-only (AQD) and ternary (FATNN) arithmetic make low-bit networks edge-deployable without accuracy loss.

[1] Chen, Liu, Zhuang, Tan, Shen. AQD: Towards Accurate Fully-Quantized Object Detection. CVPR 2021 (Oral).

[2] Chen, Zhuang, Shen. FATNN: Fast and Accurate Ternary Neural Networks. ICCV 2021.

NVFP4: 4-Bit Floating Point Across Training and Inference

Two-level scaling — a fine ruler per 16 values, a global trim per tensor



NVFP4



MXFP4



E2M1 = 15 values: {0, ±0.5, ±1, ±1.5, ±2, ±3, ±4, ±6} — ~4.5 bits/value incl. scales

W4A4 GEMM on Blackwell: weights and activations both 4-bit → up to 4× throughput, ¼ memory traffic vs. BF16

Adopted by design: DeepSeek-V4 trains its 1.6T MoE expert weights in FP4 (QAT); GPT-OSS experts use MXFP4

Production stack: TensorRT-LLM + TransformerEngine fused quantize-GEMM kernels;

3×

peak inference throughput vs. FP8 on Blackwell

1.9×

faster pretraining than FP8 (Llama-3.1 405B run)

<1%

accuracy drop: DeepSeek-R1 PTQ to NVFP4 vs. FP8 (MMLU-Pro 85→84)

+36%

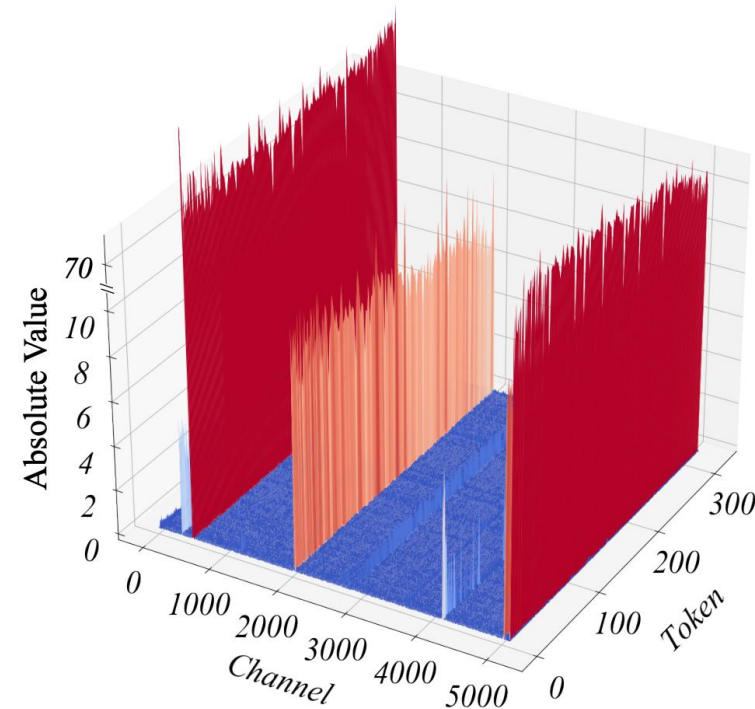
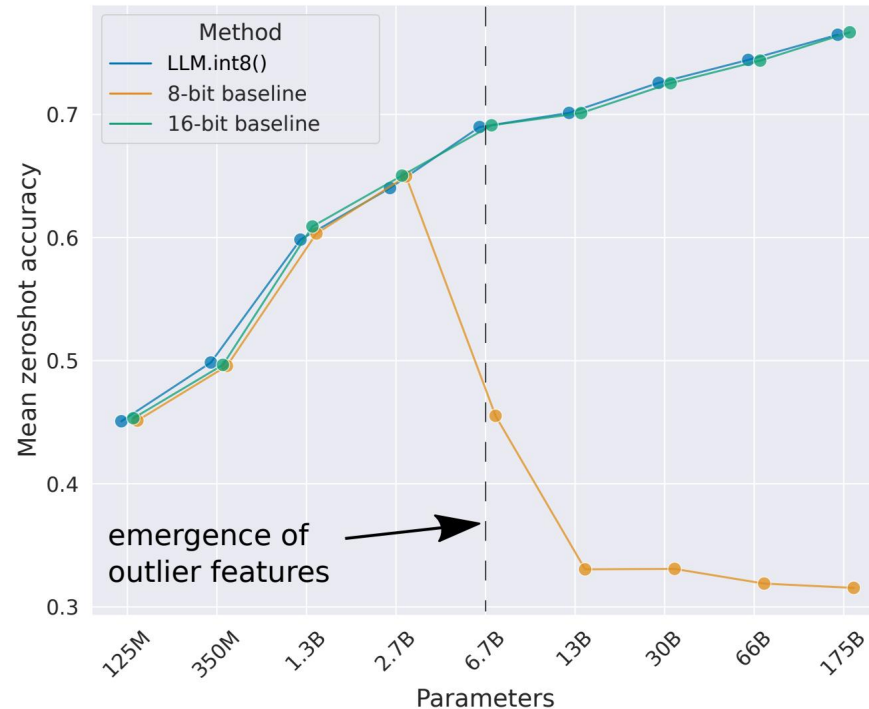
training tokens MXFP4 needs to match NVFP4's loss

Two-level scaling makes 4-bit nearly lossless: FP8-level accuracy at up to 3× the throughput.

[1] NVIDIA, Introducing NVFP4 for Efficient and Accurate Low-Precision Inference, 2025. [2] Huang et al., Pushing Intelligence to 4-bit, NVIDIA Research (EAI) blog, June 2026.

Low-bit Quantization for LLMs

Outlier issues in Post-training Quantization

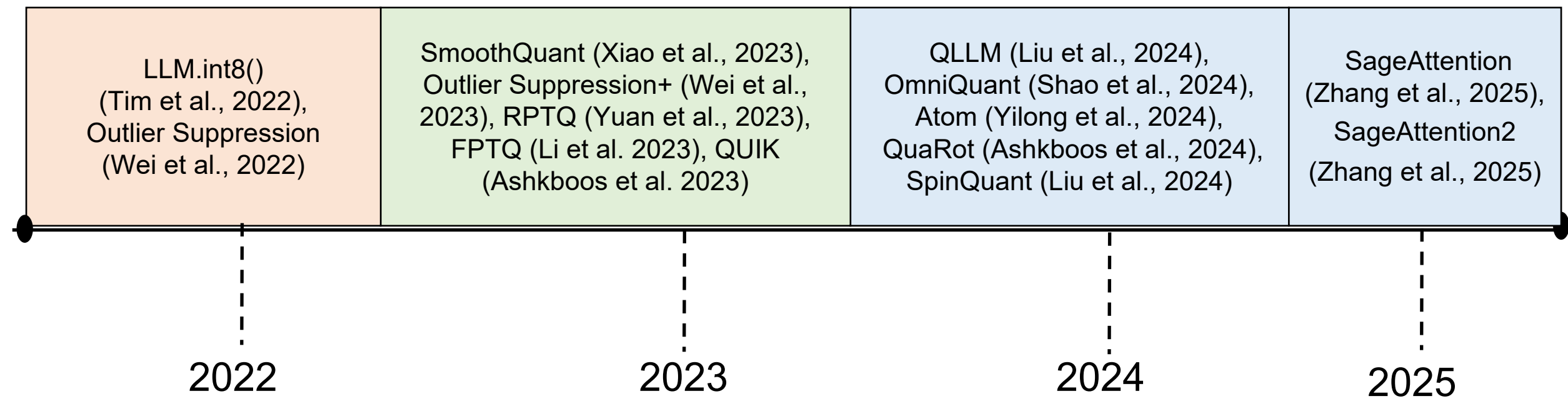


- ❑ Activation channel outliers emerge when scaling up beyond 6.7B parameters.
- ❑ The activation outliers in specific channels make existing quantization methods less effective.

Tim et al., LLM.int8(): 8-bit matrix multiplication for transformers at scale, NeurIPS 2022.

Xie et al., SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models, ICML 2023.

Dealing with Activation Outliers in LLMs



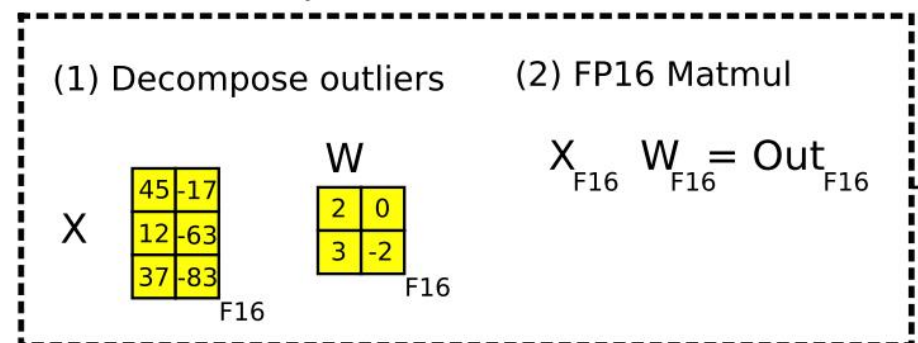
Tim et al., LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale, NeurIPS 2022.
Wei et al., Outlier suppression: Pushing the limit of low-bit transformer language models, NeurIPS 2022.
Xiao et al., SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models, ICML 2023.
Wei et al., Outlier suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling, EMNLP 2023.
Yuan et al., Rptq: Reorder-based post-training quantization for large language models, arXiv 2023.
Li et al., Fptq: Fine-grained post-training quantization for large language models, arXiv 2023.
Ashkboos et al., Towards end-to-end 4-bit inference on generative large language models, arXiv 2023.
Liu et al., QLLM: Accurate and Efficient Low-Bitwidth Quantization for Large Language Models, ICLR 2024.
Shao et al., Omniquant: Omnidirectionally calibrated quantization for large language models, ICLR 2024.
Zhao et al., Atom: Low-bit quantization for efficient and accurate llm serving, MLSys 2024.
Ashkboos et al., Quarot: Outlier-free 4-bit inference in rotated llms, arXiv 2024.
Liu et al., SpinQuant-LLM quantization with learned rotations, arXiv 2024.
Zhang et al., SageAttention: Accurate 8-Bit Attention for Plug-and-Play Inference Acceleration, ICLR 2025.
Zhang et al., SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization, ICML 2025.

Low-bit Model Quantization for LLMs

LLM.int8() [Dettmers et al., 2022]:

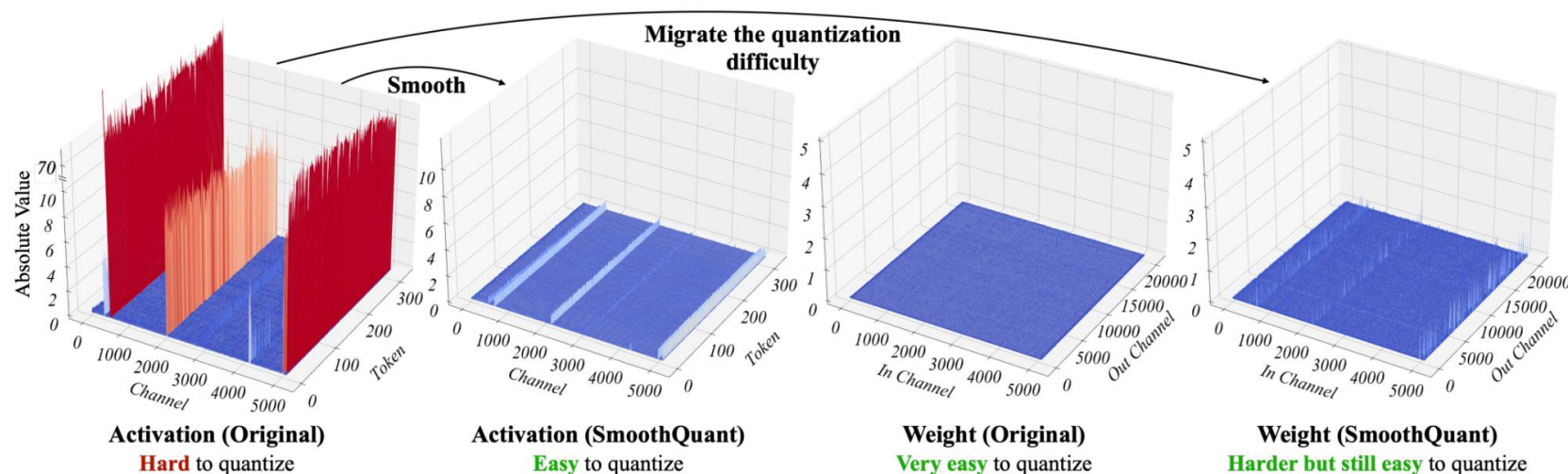
Outliers: FP16
Others: INT8

16-bit Decomposition



SmoothQuant [Xiao et al., 2023]:

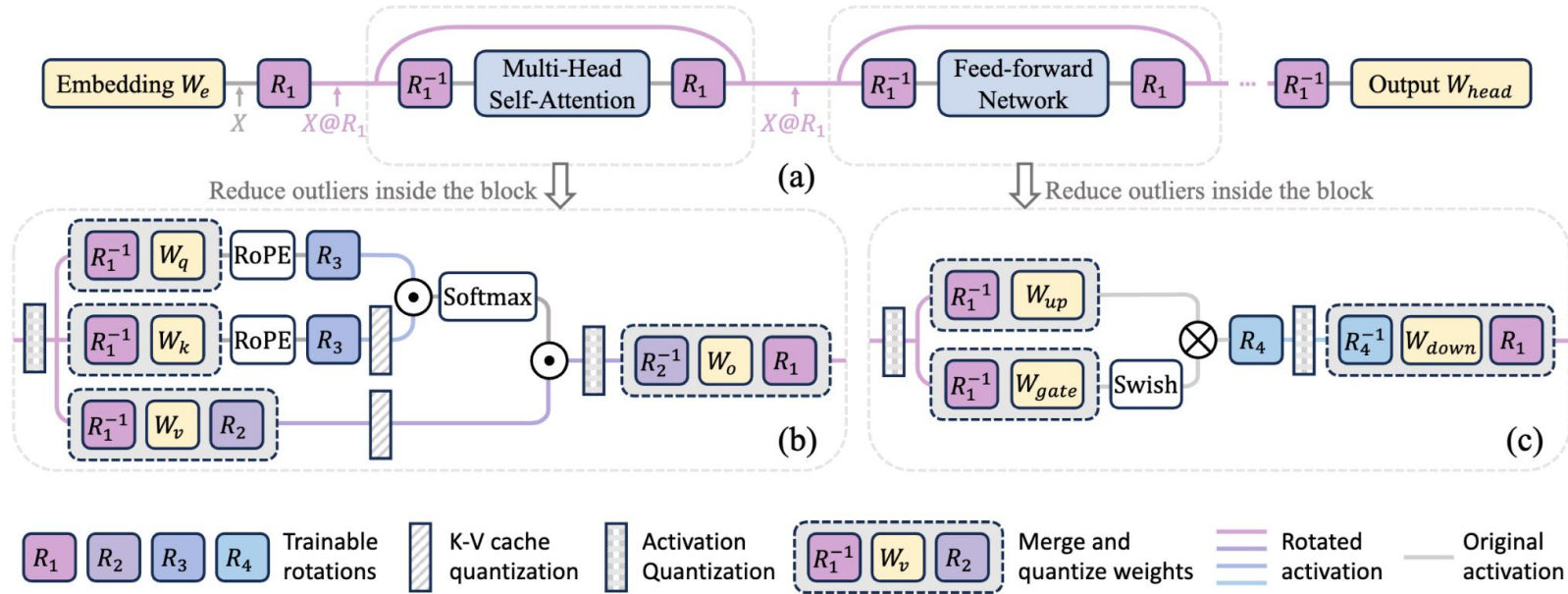
$$Y = (X \text{diag}(s)^{-1}) \cdot (\text{diag}(s)W) = \hat{X}\hat{W}$$



Xiao et al., SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models, ICML 2023.

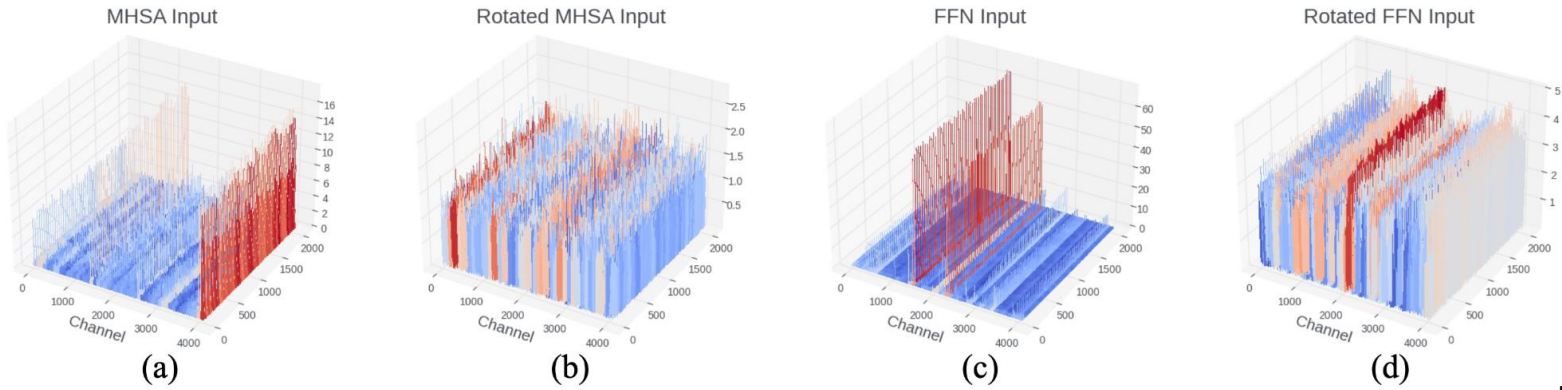
Tim et al., LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale, NeurIPS 2022.

Rotary Quantization – QuaRot & SpinQuant



$$X = (X R) R^{-1}$$

Rotation Invariant



- ❑ QuaRot: random orthogonal R
- ❑ SpinQuant: learned orthogonal R

For example, R can be Walsh-Hadamard transform

Efficient Attention

Attention for Efficient Inference: Three Families

Challenges: 1: Heavy KV Cache for long context
2: $O(n^2)$ computation complexity

Softmax Attention

exact · $O(N^2)$ compute, IO-optimized

FlashAttention v1-v4: tiling + recomputation — exact attention, no $N \times N$ matrix in HBM

Quality gold standard; KV cache still grows with context

Default in every serving stack

Linear Attention

recurrent · $O(N)$ time, $O(1)$ state

Kernelized / RNN form: replace softmax with a decaying state — constant memory per token

Models: Mamba series, RWKV series, Gated DeltaNet (Qwen3.5 series), Kimi Linear, MiniMax lightning attention

Cheap long context, but weaker exact recall
→ used in hybrids

Hybrid layers in frontier LLMs

Sparse Attention

select tokens · $\approx O(N \cdot k)$ compute

Attend only where it matters: trainable block/token selection

Models: DeepSeek DSA (V4), MoBA (Kimi); KV-selection at decode: Quest, H2O

Near-full quality at a fraction of prefill FLOPs and KV reads

Long-context prefill workhorse

FlashAttention: IO-Aware Exact Attention

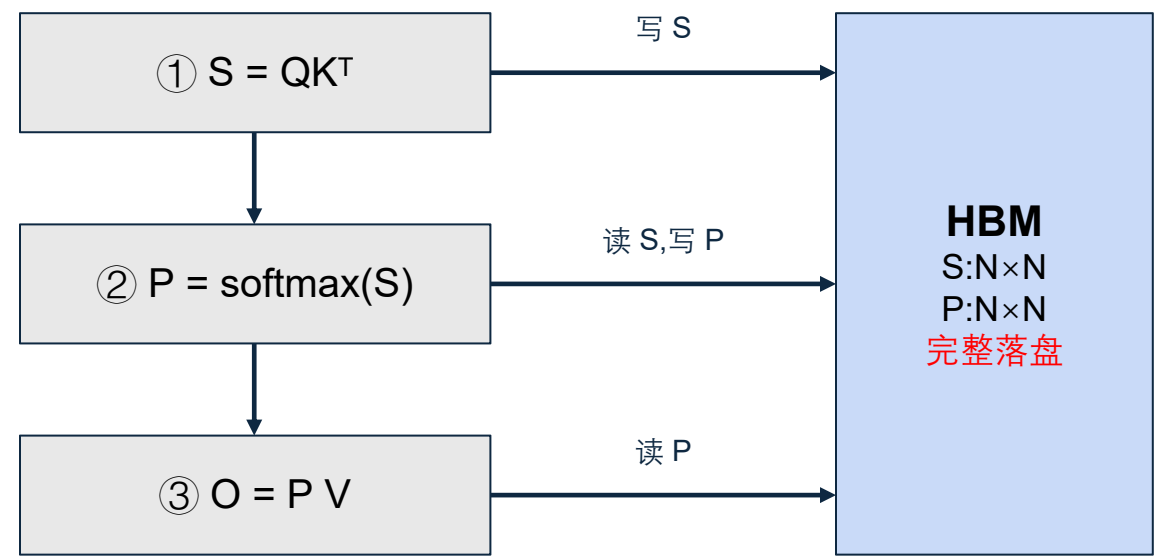
SRAM(片上) $\approx 19 \text{ TB/s} \cdot \sim 20 \text{ MB}$

带宽差 $\sim 10\times$

HBM $\approx 1.5\text{--}2 \text{ TB/s} \cdot 40 \text{ GB}$

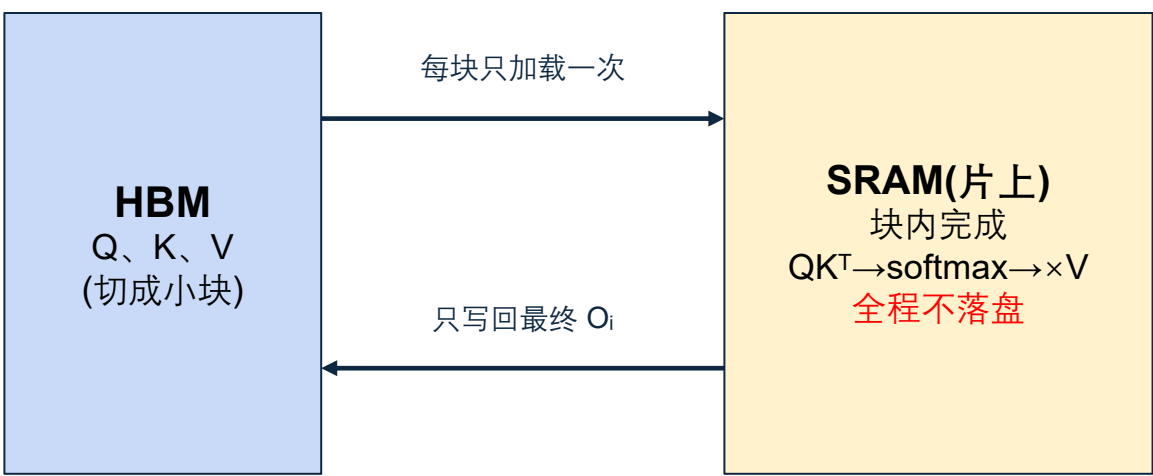
反复读写大矩阵 $\rightarrow$ attention 是 **memory-bound**

标准 Attention: 中间矩阵反复读写 HBM



S、P 的读写量 $O(N^2)$, 远大于 Q/K/V 的 $O(Nd)$ ($N \gg d$)
 $\rightarrow$ 时间花在 HBM 搬运数据, 而不是计算

FlashAttention: 分块(Tiling)在 SRAM 内算完

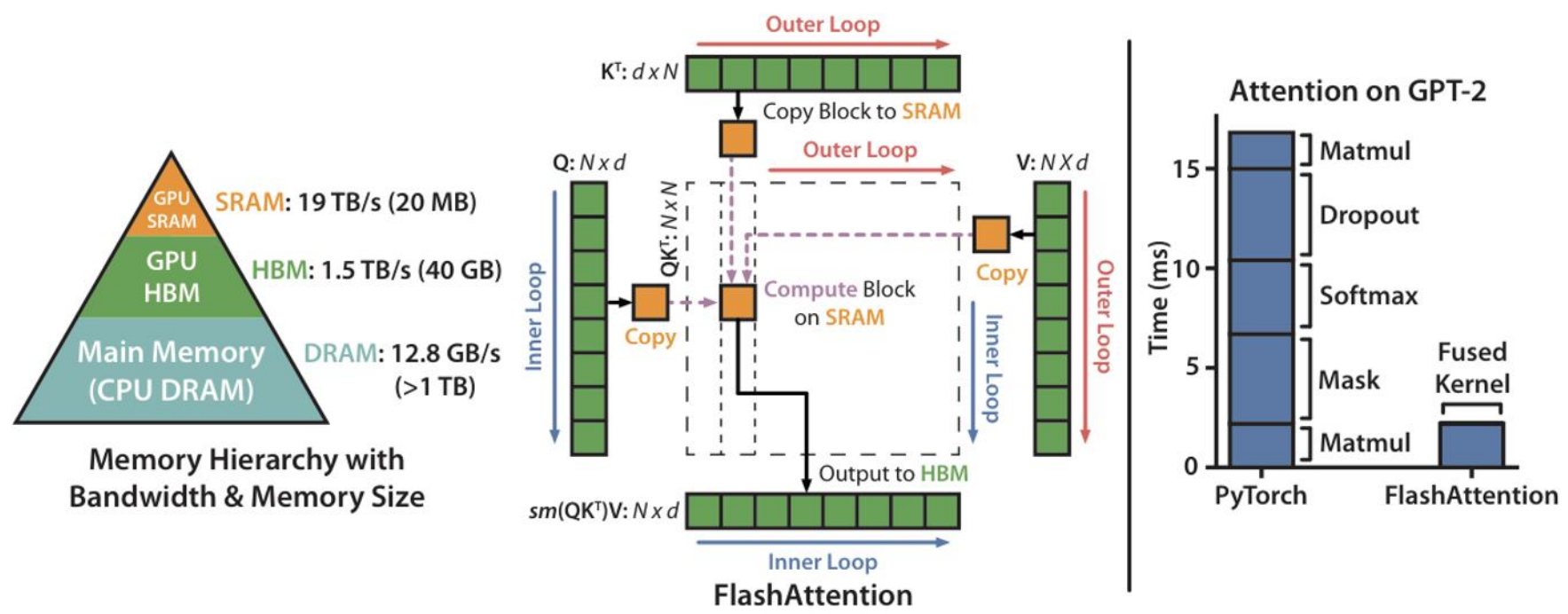


$N \times N$ 的 S、P 从未完整写入 HBM, 只有块大小的中间结果短暂存于 SRAM

极大减少 HBM 访存!

[1] Dao et al., FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, NeurIPS 2022.

FlashAttention: IO-Aware Exact Attention



	Full (standard) attention	FlashAttention
Compute (FLOPs)	$\Theta(N^2d)$	$\Theta(N^2d)$ — same, + recompute in backward
HBM accesses (I/O)	$\Theta(N \cdot d + N^2)$	$\Theta(N^2d^2 / M) \rightarrow$ up to 9 $\times$ fewer for GPT-2 (M = SRAM size)
Extra memory	$O(N^2)$ — materializes $N \times N$ scores	$O(N)$ — never forms the $N \times N$ matrix 只额外保存每行的 softmax 统计量(最大值和归一化因子)
Measured speed	1 $\times$ (PyTorch baseline)	7.6 $\times$ attention op (GPT-2); 3 $\times$ GPT-2 training; +15% over BERT MLPerf record

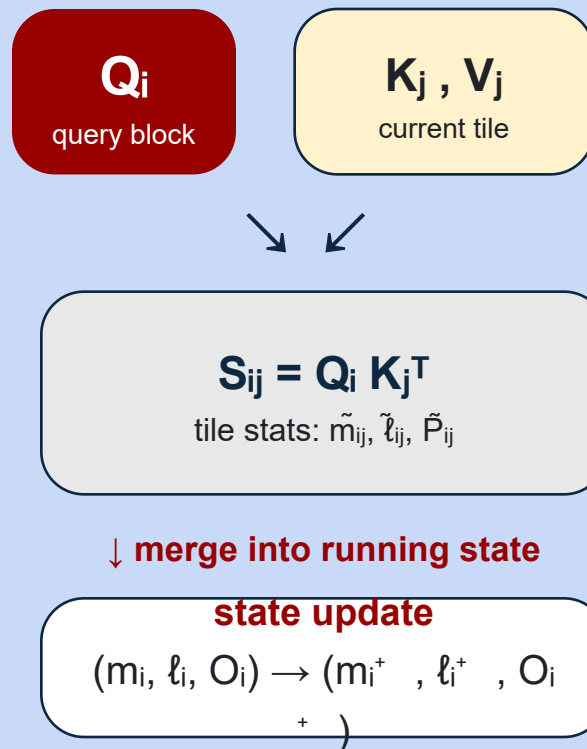
tiling + recomputation give exact attention with significantly less HBM traffic.

[1] Dao et al., FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, NeurIPS 2022. [2] Dao, FlashAttention-2, ICLR 2024. [3] Shah et al., FlashAttention-3, NeurIPS 2024.

FlashAttention: IO-Aware Exact Attention

1. Tile, merge, discard

one score tile lives in SRAM at a time



2. Online softmax via log-sum-exp

merge one K / V tile into each query row

1 Local tile statistics

$$\tilde{m}_{ij} = \text{rowmax}(S_{ij})$$

$$\tilde{P}_{ij} = \exp(S_{ij} - \tilde{m}_{ij}), \quad \tilde{\ell}_{ij} = \text{rowsum}(\tilde{P}_{ij})$$

2 Merge maximum + normalizer

$$m_i^+ = \max(m_i, \tilde{m}_{ij})$$

$$\ell_i^+ = e^{m_i - m_i^+} \ell_i + e^{\tilde{m}_{ij} - m_i^+} \tilde{\ell}_{ij}$$

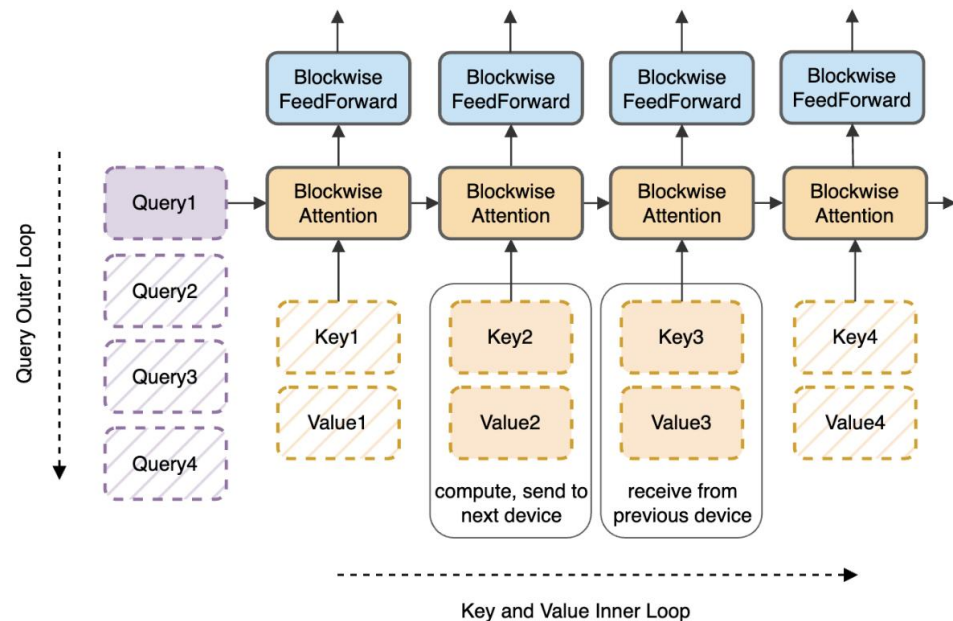
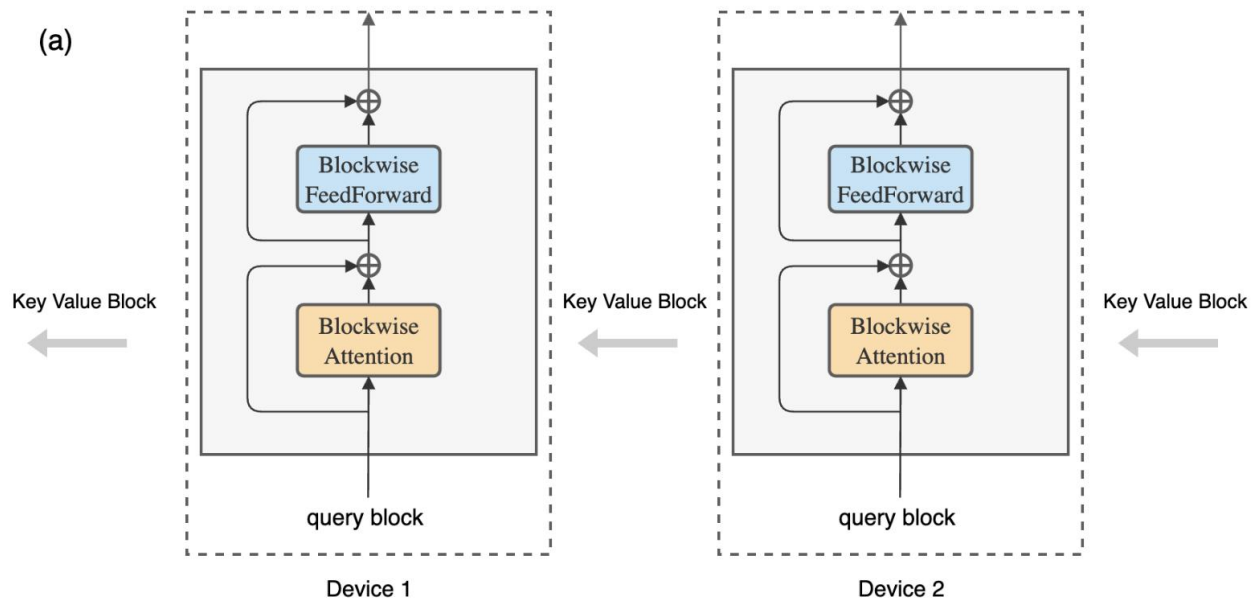
3 Rescale + update output

$$O_i^+ = \frac{e^{m_i - m_i^+} \ell_i O_i + e^{\tilde{m}_{ij} - m_i^+} \tilde{P}_{ij} V_j}{\ell_i^+}$$

Final tile: LSE_i = m_i + log ℓ_i

Exact result • O(N) auxiliary memory • no N×N score matrix in HBM

Ring Attention



- Each host holds one query block
- The communication is overlapped with the computation.

Background -- Linear Attention

Background: Linear Complexity Attention

Approximate the $\exp(\langle \cdot, \cdot \rangle)$ using Gaussian RBF kernel

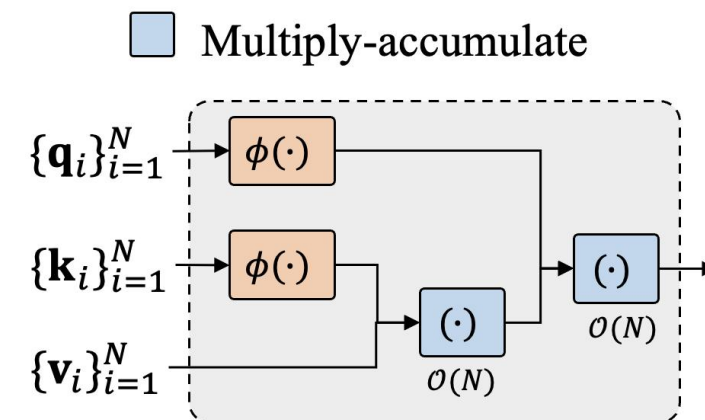
$$\begin{aligned}\exp(\mathbf{x} \cdot \mathbf{y} / \sigma^2) &= \exp(\|\mathbf{x}\|^2 / 2\sigma^2 + \|\mathbf{y}\|^2 / 2\sigma^2) \exp(-\|\mathbf{x} - \mathbf{y}\|^2 / 2\sigma^2) \\ &\approx \exp(\|\mathbf{x}\|^2 / 2\sigma^2 + \|\mathbf{y}\|^2 / 2\sigma^2) \phi(\mathbf{x})^\top \phi(\mathbf{y})\end{aligned}$$

Approximate the attention as

$$\begin{aligned}\text{Attention}(\mathbf{q}_t, \{\mathbf{k}_i\}, \{\mathbf{v}_i\}) &\approx \sum_i \frac{\phi(\mathbf{q}_t)^\top \phi(\mathbf{k}_i) \mathbf{v}_i^\top}{\sum_j \phi(\mathbf{q}_t)^\top \phi(\mathbf{k}_j)} \\ &= \frac{\phi(\mathbf{q}_t)^\top \sum_i \phi(\mathbf{k}_i) \otimes \mathbf{v}_i}{\phi(\mathbf{q}_t)^\top \sum_j \phi(\mathbf{k}_j)}\end{aligned}$$

The complexity is reduced into linear.

Limitation: the intensive floating-point multiplications still consume a large amount of energy.



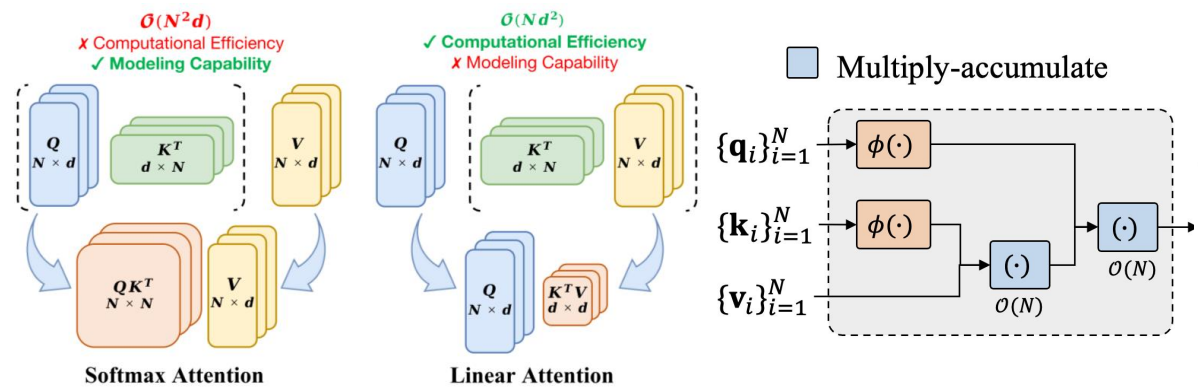
Linear Attention

Recall: Softmax Attention

$$O = \text{Softmax}(\frac{QK^T}{\sqrt{d}})V$$

Linear Compute Complexity

- Reorder matrix multiplication to reduce attention complexity to $O(N)$.
- Replace explicit pairwise attention with a compressed recurrent state.



Causal Mask + Linear Attention -> RNNs

- Compress the entire history into a constant-size recurrent state.

$$S_t = S_{t-1} + k_t v_t^T$$
$$O_t = q_t S_t$$

Linear Attention

$$O = \phi(Q)(\phi(K)^T V)$$

Variants of Linear Attention

Gated Linear Attention (GLA)

Add gate decay for forgetting

Delta Net

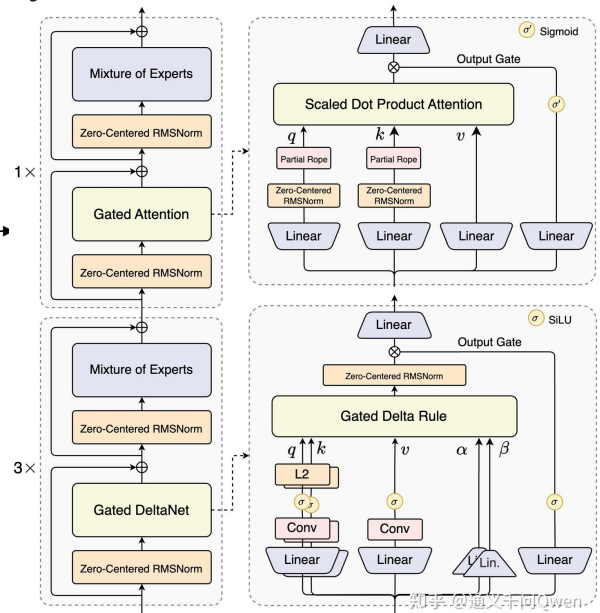
Learn what to write

Gated Delta Net (GDN)

Learn forgetting & writing

KDA, EDA, ...

Hybrid Architecture



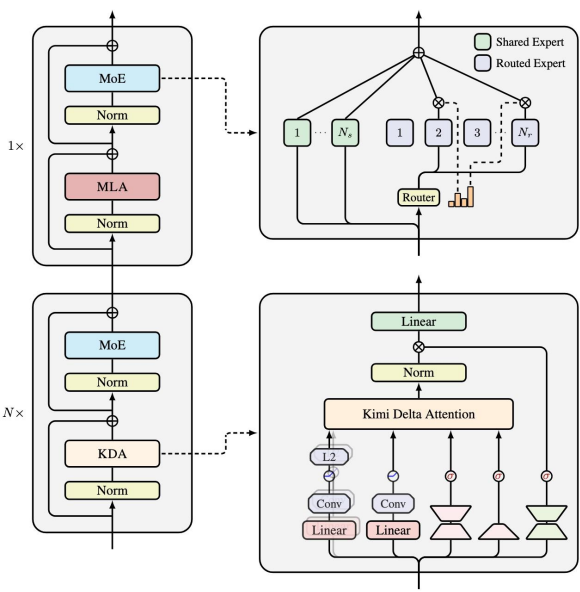
Qwen 3 Next

Linear attention prefix caching 命中率极低，但可能适用于agentic 场景

$$S_t = \alpha_t S_{t-1} + k_t v_t^T, \alpha_t \in [0, 1]$$

$$S_t = S_{t-1}(I - \beta_t k_t k_t^T) + \beta_t k_t v_t^T, \beta_t \in [0, 1]$$

$$S_t = \alpha_t S_{t-1}(I - \beta_t k_t k_t^T) + \beta_t k_t v_t^T$$
$$\alpha_t, \beta_t \in [0, 1]$$



Kimi Linear

Linear Attention: FlashQLA

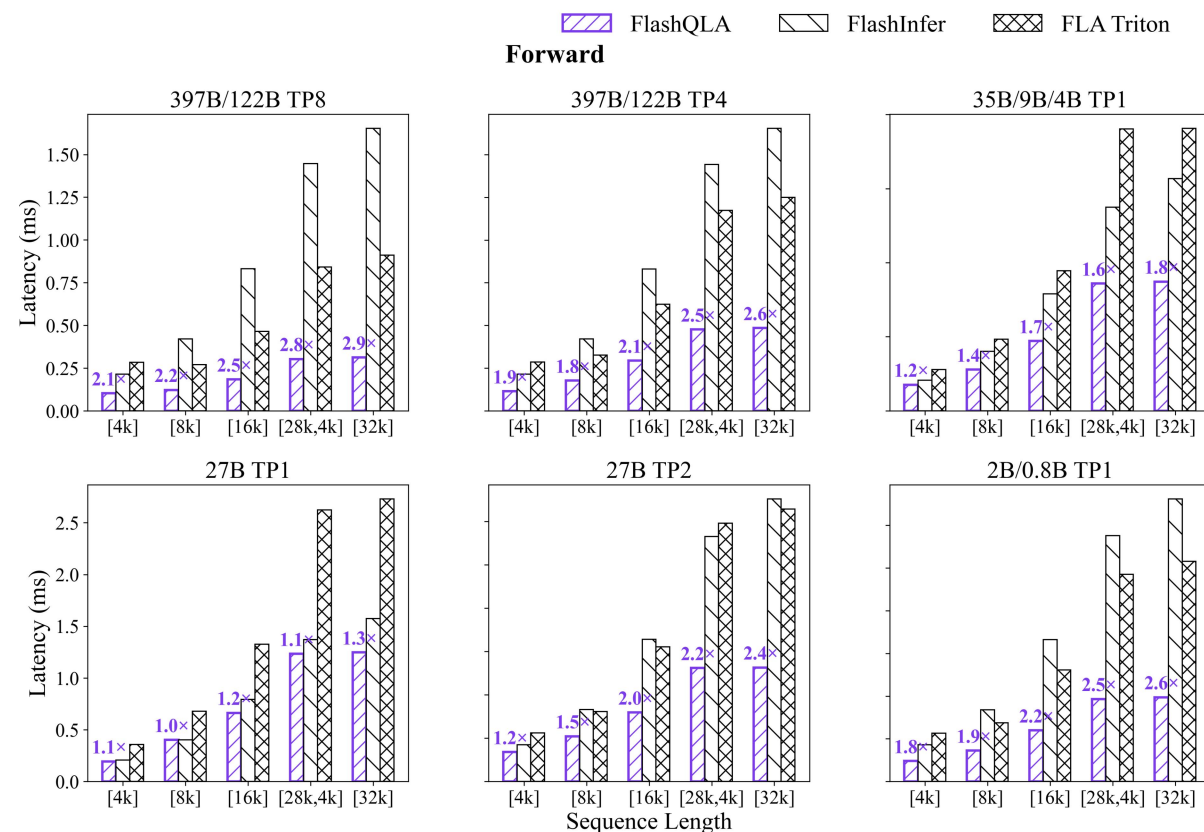


FlashQLA

CP- and Bwd-Friendly Fused Linear Attention Kernels for GDN

- **Gate-aware Auto Context Parallelism**
 - Scalable to TP & long sequences
 - Automatic workload-aware CP Partitioning
 - Gate-induced error decay
- **Hardware-friendly GDN Reformulation**
 - Factor gate decay out
 - Lower Tensor Core / CUDA Core / SFU cost
- **Warp-specialized Fused TileLang Kernels**
 - Manual warpgroup specialization
 - Overlap data transfer, Tensor Core & CUDA Core

<https://github.com/QwenLM/FlashQLA>
<https://qwen.ai/blog?id=flashqla>

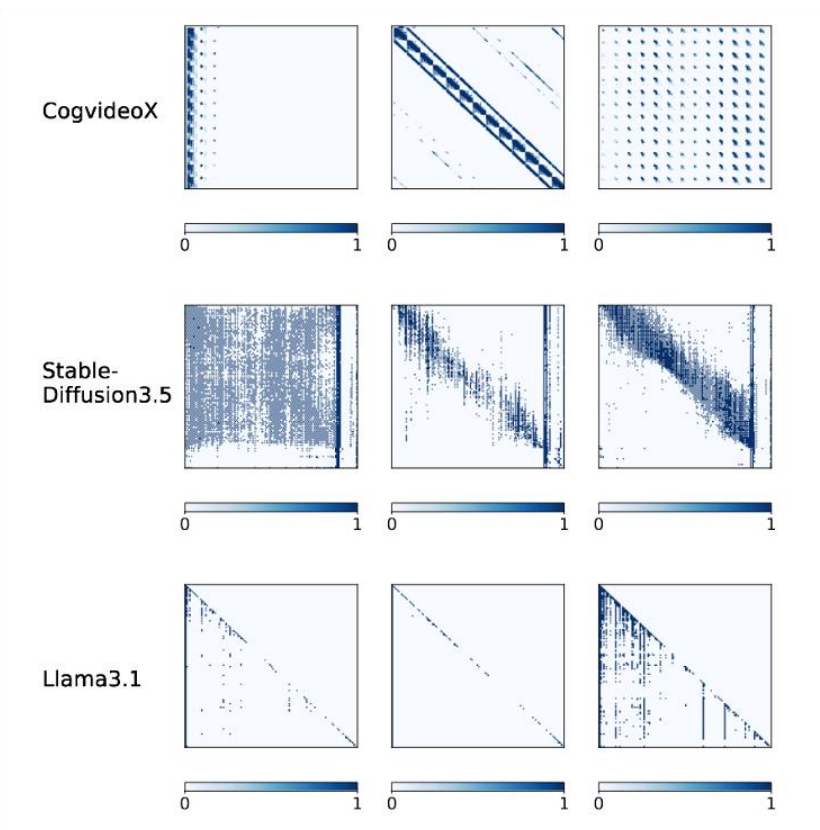


- **Forward: 2–3× Speedup**
- **Backward: 2× Speedup**

Sparse Attention

Attention Maps Are Inherently Sparse

Only a small fraction of tokens receive most of the attention.



Static Sparse Attention

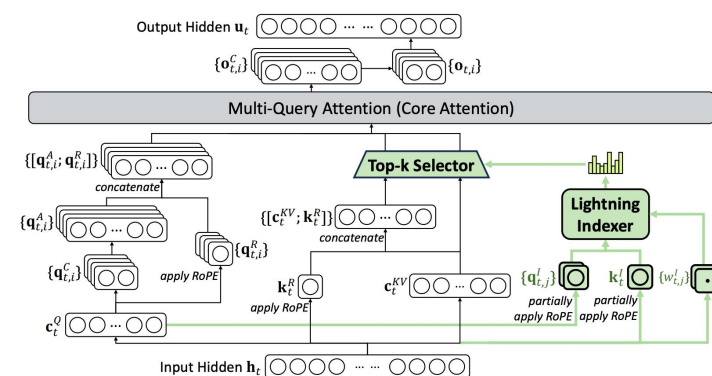
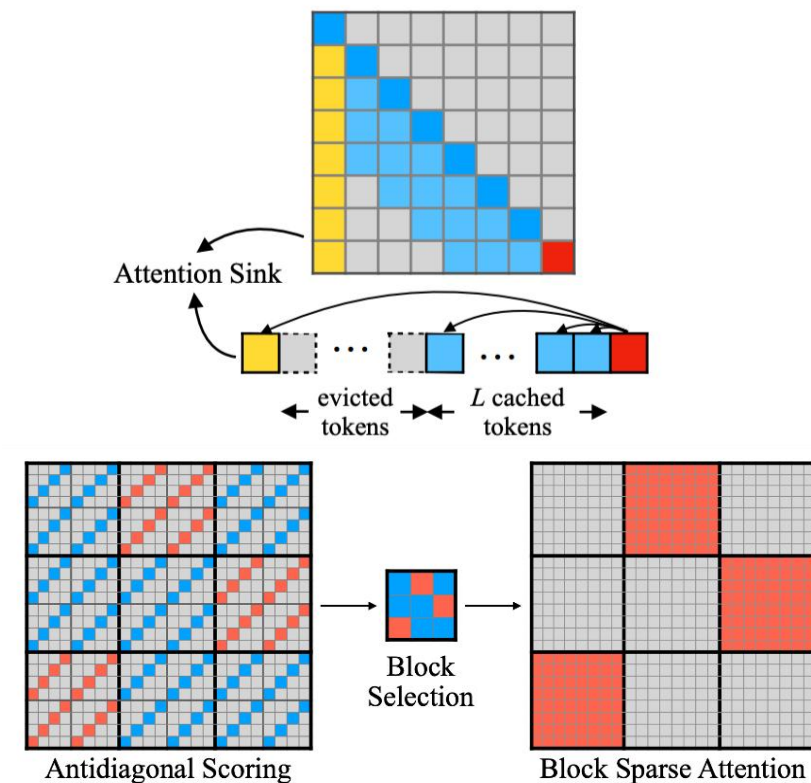
- Fixed patterns
- StreamingLLM, MInference, ...

Dynamic Block Sparse Attention

- Adaptive block selection
- SparseAttn, XAttention, ...

Dynamic Token Sparse Attention

- Fine-grained token selection
- DeepSeek Sparse Attention, ...



A 1M-Token GLM-5.2 Agent Holds ≈ 90.5 GiB of Cache

GLM-5.2 is a 744B-class MoE (~ 40 B active), built for long-horizon coding and research agents with a 1,048,576-token context window.

Its MLA + DSA stack compresses retained attention state, but long histories and concurrent agents still multiply memory.

GLM-5.2 @ 1M · vLLM cache layout

90.46 KiB/token $\times$ 1,048,576 tokens

≈ 90.5 GiB — for ONE agent

4 independent histories $\rightarrow \approx 362$ GiB, before weights and runtime buffers

Why long-horizon agents stress both phases

Prefill



↓ one parallel forward pass



All N prompt tokens attend to each other in a single pass.

Compute-bound — high GPU utilization.

Decode



↓ attend over all 7, append 1 entry



Every step re-reads the whole cache, appends one new K/V pair.

Memory-bandwidth-bound — cache grows every step.

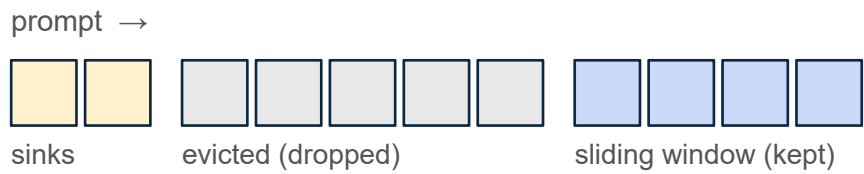
Even compressed, GLM-5.2 holds ≈ 90.5 GiB at 1M tokens; memory scales with context $\times$ active agents.

[1] vLLM GLM-5.2 layout: $78 \times (512 \text{ MLA latent} + 64 \text{ RoPE}) \text{ BF16} + 21 \text{ DSA index caches } (128 \text{ FP8} + \text{scale}) = 90.46 \text{ KiB/token}$. At 2^{20} tokens: 90.46 GiB; excludes weights/runtime.

KV Cache Compression: A Taxonomy

Fixed-Rule (Static)

- A predetermined keep/evict rule — no runtime attention observation.
- StreamingLLM: keep the first “sink” tokens + a sliding window of recent tokens; evict everything between.



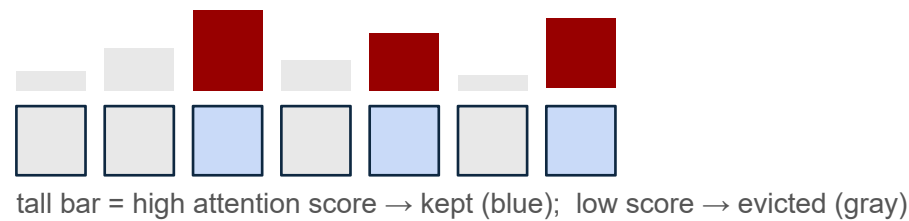
e.g. StreamingLLM (Xiao et al., ICLR 2024)

Linear attention: KV cache states keep the constant size

	Initial state	Update rule	Output rule	Cost
Linear attention	$s_0 = \text{vector}()$	$s_t = \sigma(\theta_{ss}s_{t-1} + \theta_{sx}x_t)$	$z_t = \theta_{zs}s_t + \theta_{zx}x_t$	$O(1)$
Self-attention	$s_0 = \text{list}()$	$s_t = s_{t-1}.\text{append}(k_t, v_t)$	$z_t = V_t \text{softmax}(K_t^T q_t)$	$O(t)$

Attention-Score-Based (Dynamic)

- Rank cached tokens by (historical) attention score; evict/unread the lowest scorers.
- H2O keeps a running cumulative score per step; ZipCache dynamically ranks tokens with normalized cumulative attention scores and probe tokens.
- ~20% of tokens receive ~80% of total attention weight — attention is highly non-uniform.



e.g. H2O (NeurIPS 2023), ZipCache (NeurIPS 2024)

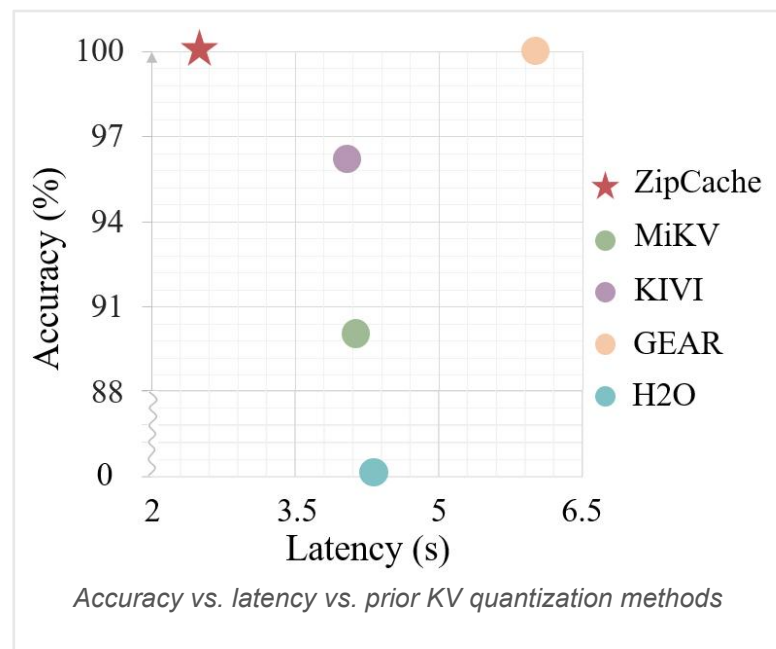
Both hit the same infra wall: FlashAttention hides scores, PagedAttention fragments memory.

Our KV Cache Compression Methods

ZipCache

NeurIPS 2024

- Score cached tokens by normalized attention weight to estimate saliency. Compatible with FlashAttention.
- Salient tokens quantized at 4-bit, the rest at 2-bit — mixed precision
- $4.98\times$ compression on Mistral-7B/GSM8K, only 0.38% accuracy drop

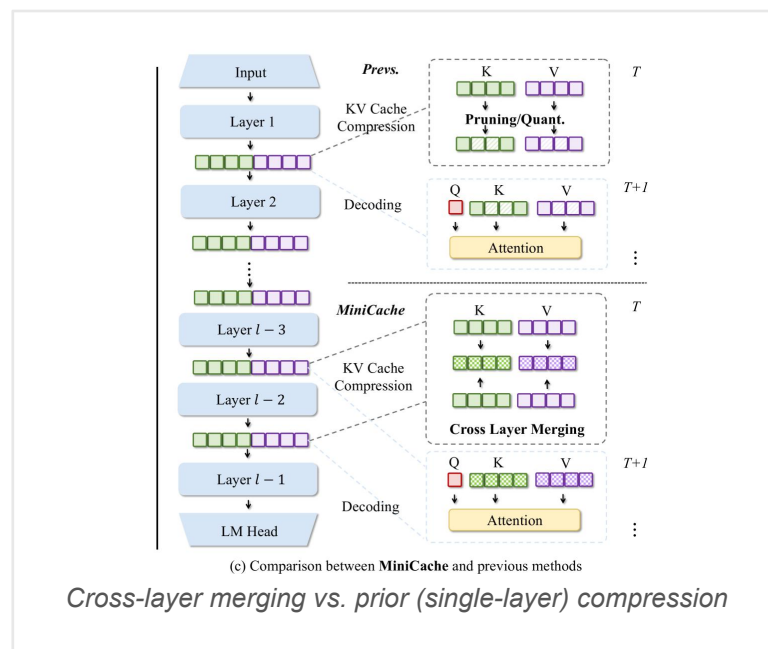


MiniCache

NeurIPS 2024

- Adjacent middle-to-deep layers hold near-identical KV states
- Merge across depth: interpolate direction, preserve magnitude; keep outlier tokens unmerged
- Up to 41% memory reduction, $\sim 5\times$ throughput vs. FP16

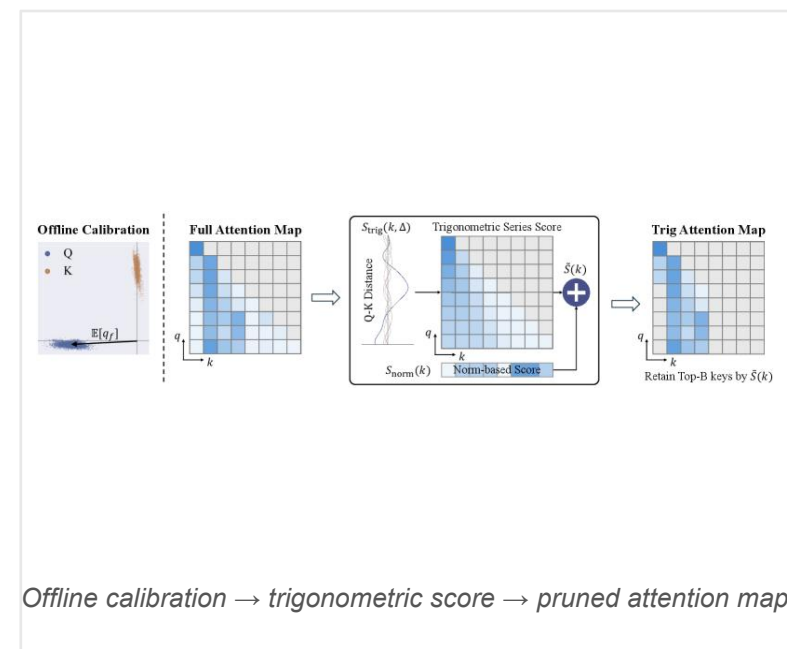
MiniCache是当前主流模型标配



TriAttention

ICML 2026 · w/ MIT, NVIDIA

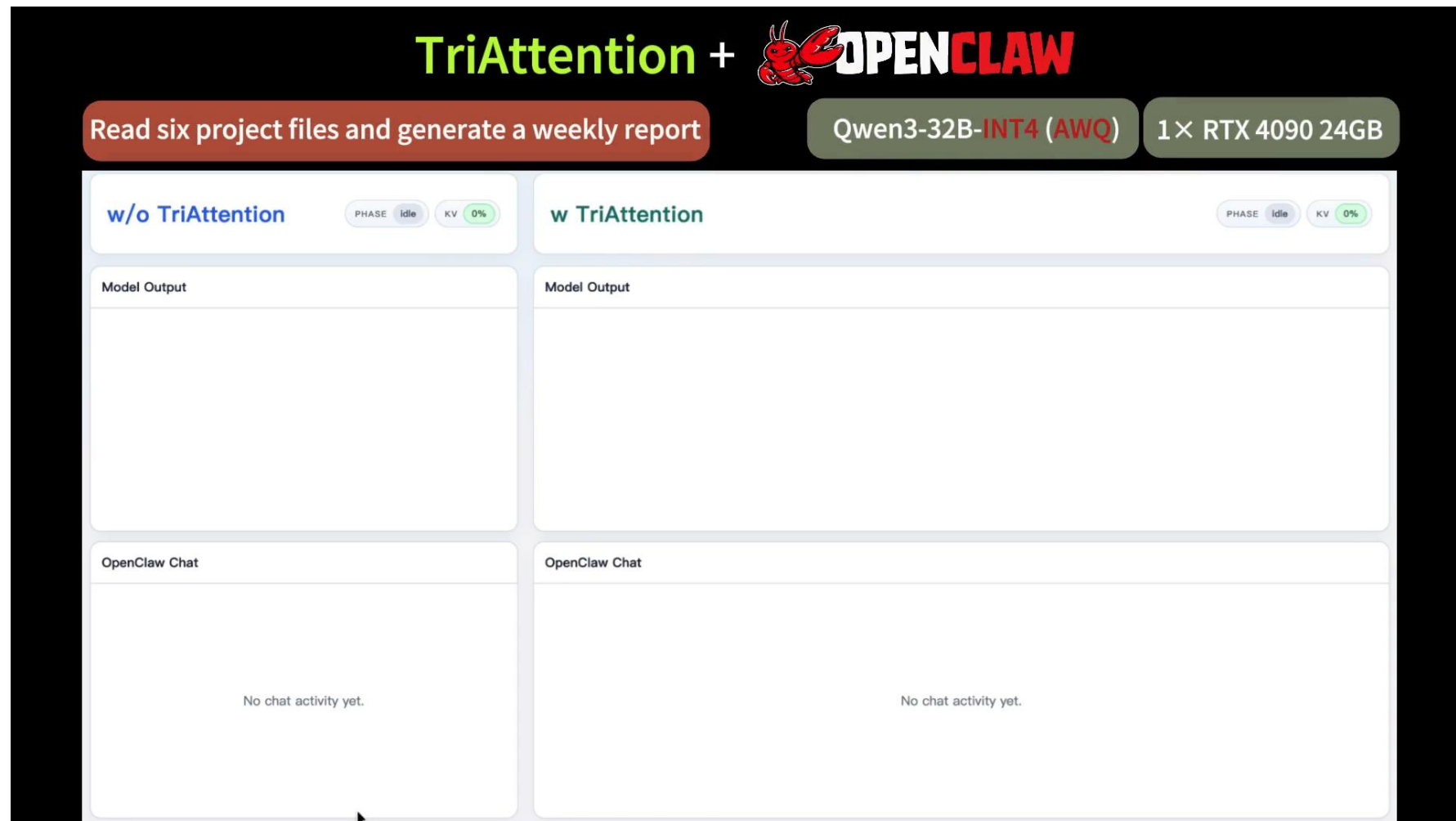
- Q/K vectors concentrate in pre-RoPE space $\rightarrow$ attention predicted from geometry, no observed scores needed
- Compatible with FlashAttention; compacts freed blocks under PagedAttention
- $2.5\times$ throughput, $10.7\times$ KV memory reduction at matched accuracy



[1] He et al., ZipCache: Accurate and Efficient KV Cache Quantization with Salient Token Identification, NeurIPS 2024.

[2] Liu et al., MiniCache: KV Cache Compression in Depth Dimension for LLMs, NeurIPS 2024. [3] Mao et al., TriAttention: Efficient Long Reasoning with Trigonometric KV Compression, arXiv 2026.

Demo: TriAttention Running Inside OpenClaw



Task: read six project files and write a weekly report — Qwen3-32B-INT4 (AWQ) on a single RTX 4090 (24GB).

Without TriAttention: KV cache exhausted mid-task → OOM. With TriAttention: completes within the memory budget.

Hierarchical KV Cache Systems

▲ hot · fast · small capacity

GPU HBM

capacity: 10s of GB · bandwidth: ~TB/s

hottest — active generation

CPU DRAM

capacity: 100s of GB–TB · PCIe ~64 GB/s (~2% of HBM)

warm — recently evicted

Local SSD / NVMe

capacity: 10s of TB · bandwidth: a few GB/s

cold — overflow beyond DRAM

Remote / Distributed Storage

capacity: ~unlimited, scales out · network-limited RDMA

shared across instances / sessions

▼ cold · slow · huge capacity

Why tiered caching?

- GPU HBM is fast but small — long-context KV cache quickly outgrows it
- Systems (e.g. NVIDIA Dynamo) offload cold KV blocks HBM → DRAM → SSD → remote as context grows
- Latency vs. capacity trade-off: moving 50GB of KV costs ~15ms from HBM but ~800ms from CPU DRAM

In agent scenarios

Hot memory

the active session's recent turns / tool outputs — kept resident for instant reuse.

Cold memory

older turns and long-term facts — pushed to SSD / remote, fetched back only when relevant.

Like OS memory hierarchies: serving systems keep 'hot' KV close, push 'cold' KV to cheaper tiers.

Case Study: Hy-Memory (Tencent Hunyuan)

Long-term agent memory system for the OpenClaw agent framework

The problem

Agents degrade over long use — the “three-week trajectory”: week 1 enthusiasm → week 2 frustration as context is forgotten → week 3 downgraded to simple queries.

Architecture

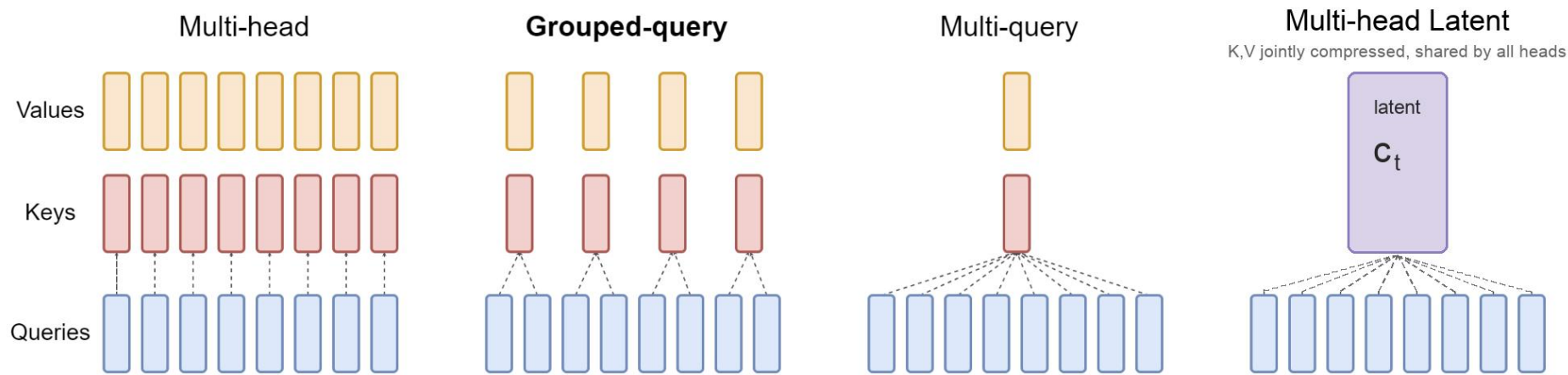
- System1 (real-time) handles L1–L4; System2 (async, seconds–minutes) distills L5–L6 without blocking dialogue

- Achieves an 85.2 score on LongMemEval benchmark
- Uses 35% fewer tokens when handling long- context inputs
- Delivers 3- 4× higher memory density compared with peer methods
- Enables 8- times- faster write operations than Graphiti



Same idea as the systems hierarchy: keep hot context fast in the foreground, distill cold memory in the background.

KV Cache 与 Attention 架构协同设计: 从 MHA、MQA、GQA 到 MLA



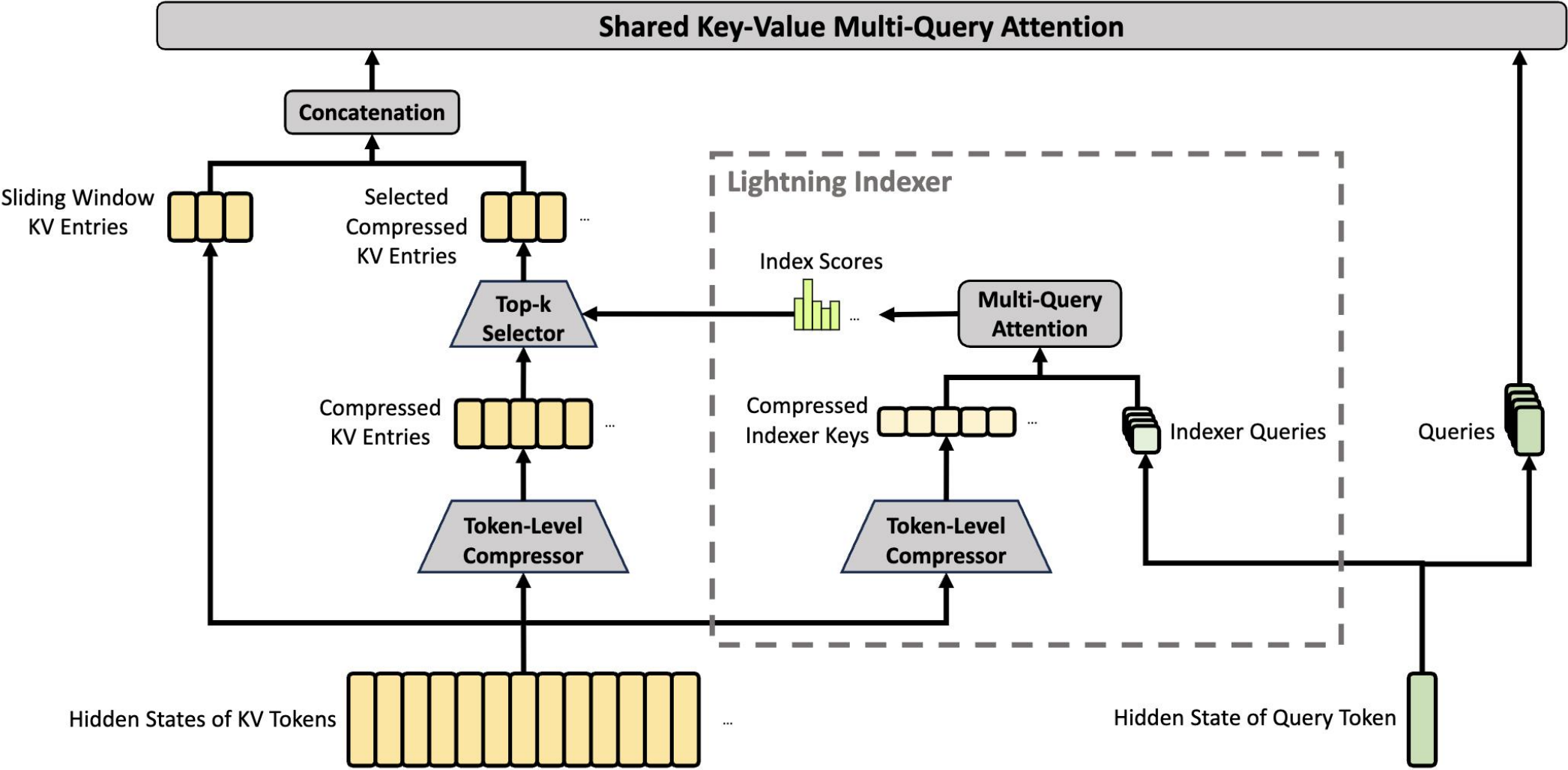
KV Cache 随 context 长度动态增长,是长文推理的显存与带宽瓶颈。MQA / GQA 靠让多个 head 共享 K, V 来压缩缓存;MLA 则把 K, V 低秩联合压缩为所有 head 共享的潜向量 c_t ,推理时只缓存 c_t (恒等变换为 MQA)。

	MHA	MQA	GQA	MLA
K / V 设计	每个 head 独立一组 K, V	全部 h 个 head 共享 1 组 K, V	每 h/g 个 head 一组,共 g 组 K, V($1 < g < h$)	K, V 低秩联合压缩为潜向量 c_t ,再升维恢复各 head
每 token·每层缓存	$2 \cdot h \cdot d_k$ (最大)	$2 \cdot d_k$,为 MHA 的 $1/h$	$2 \cdot g \cdot d_k$,为 MHA 的 g/h	$d_c + d_r = 512 + 64$,与 h 无关
效果 ↔ 显存	效果最好,缓存最大	压缩到极限,效果有损	两者折中;70B 常取 $g=8$ 匹配单机 8 卡	缓存 $\approx$ MQA,效果 $\approx$ MHA;RoPE 需解耦 d_r 维单独加
代表模型	Transformer, LLaMA2-7B	PaLM, StarCoder, Gemini	LLaMA2/3-70B, Yi, ChatGLM2/3	DeepSeek-V3

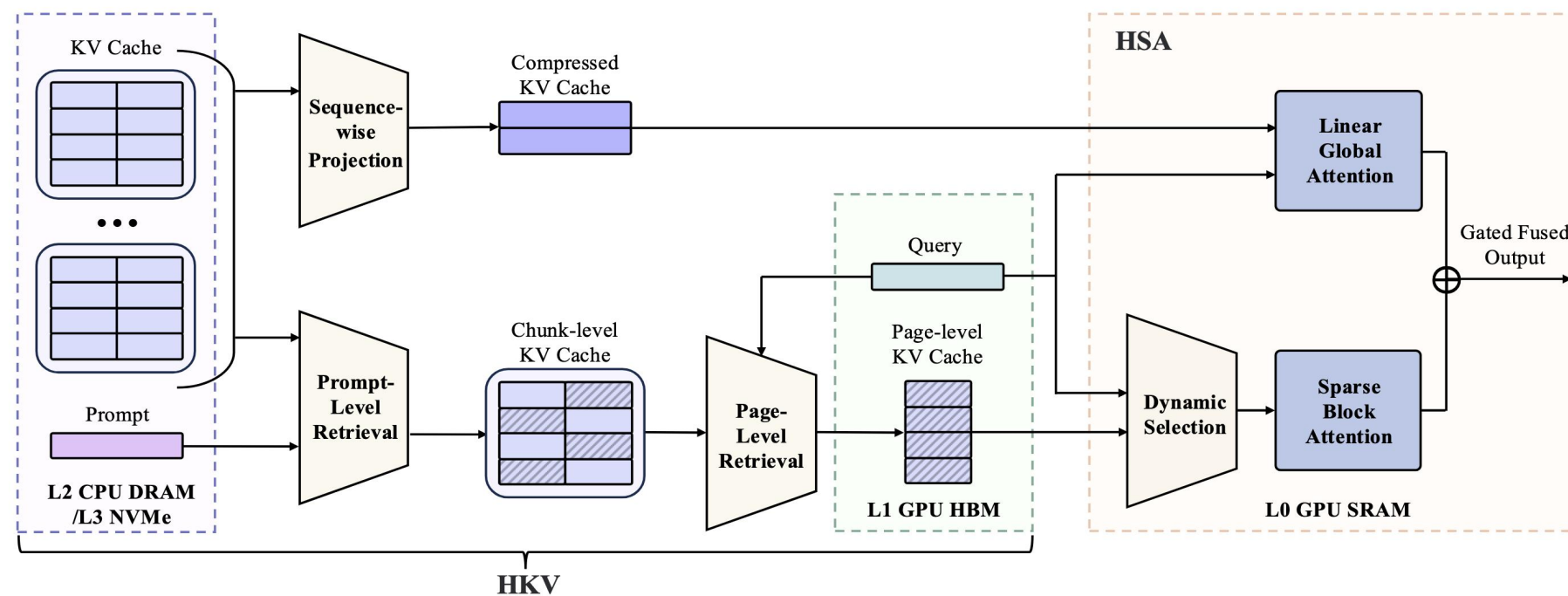
KV Cache 越小 → 同等显存下更长 context、更大 batch、更低推理成本; MLA 以「低秩压缩 + 推理时恒等变换 + 解耦 RoPE」拿到 MQA 级缓存、MHA 级效果。

[1] Ainslie et al., GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, EMNLP 2023. [2] 苏剑林, 缓存与效果的极限拉扯:从MHA、MQA、GQA到MLA, spaces.ac.cn/archives/10091, 2024. [3] DeepSeek-AI, DeepSeek-V2, 2024.

DeepSeek-V4 CSA



WorldAttention – Efficient Attention for World Models

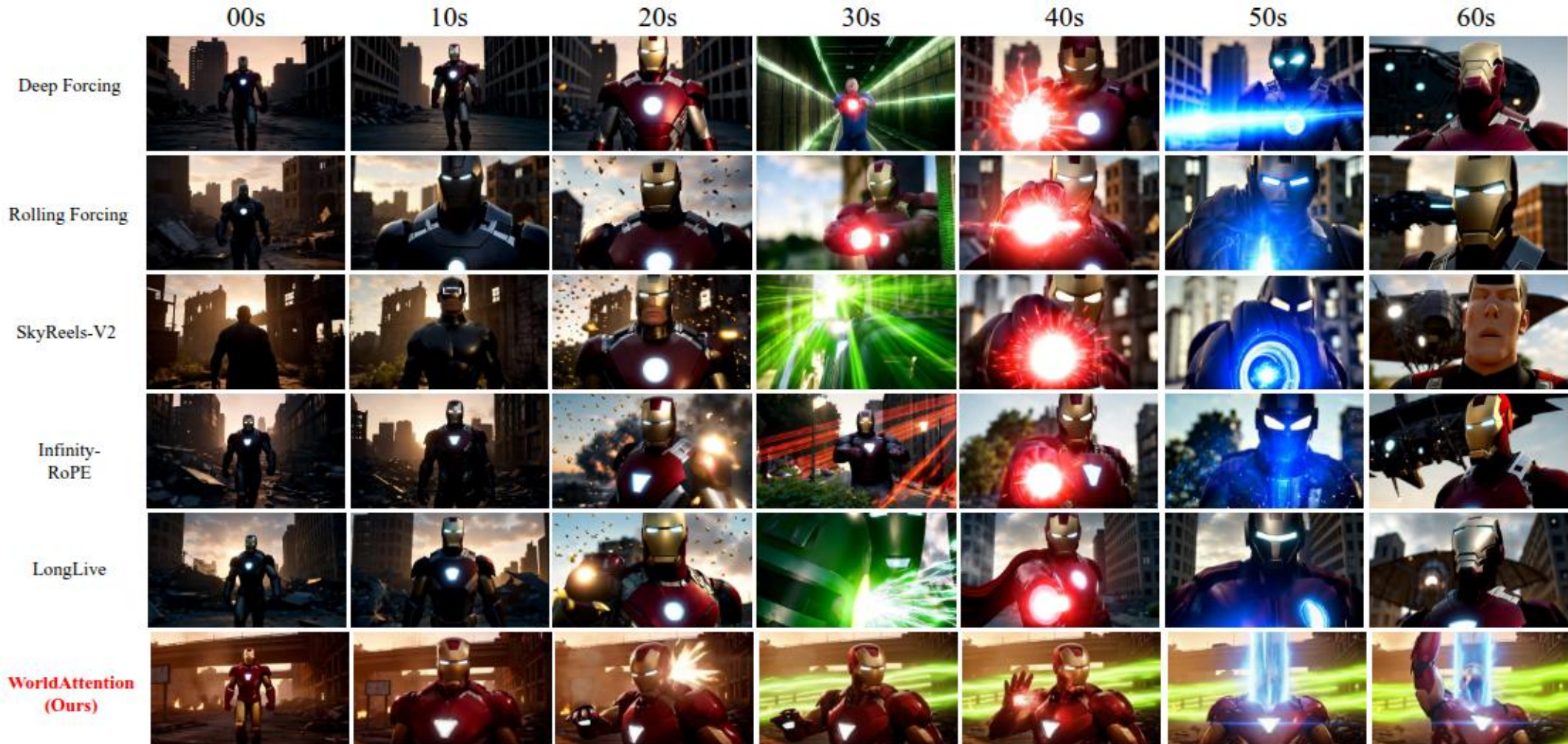


Current Challenges:

- **Interaction versus memory efficiency:** sliding-window KV cache forgets earlier context, while full-history KV cache rapidly exceeds GPU memory capacity.
- **Efficient attention computation:** retrieved KV cache contain many redundant tokens and fragmented memory access patterns, causing large decoding overhead and low hardware efficiency.

Overall architecture of WorldAttention: We propose a system-oriented co-design using Hierarchical KV Cache (HKV) for coarse-to-fine *paged* memory retrieval, and Hybrid Sparse Attention (HSA) for efficient dual-branch attention computation.

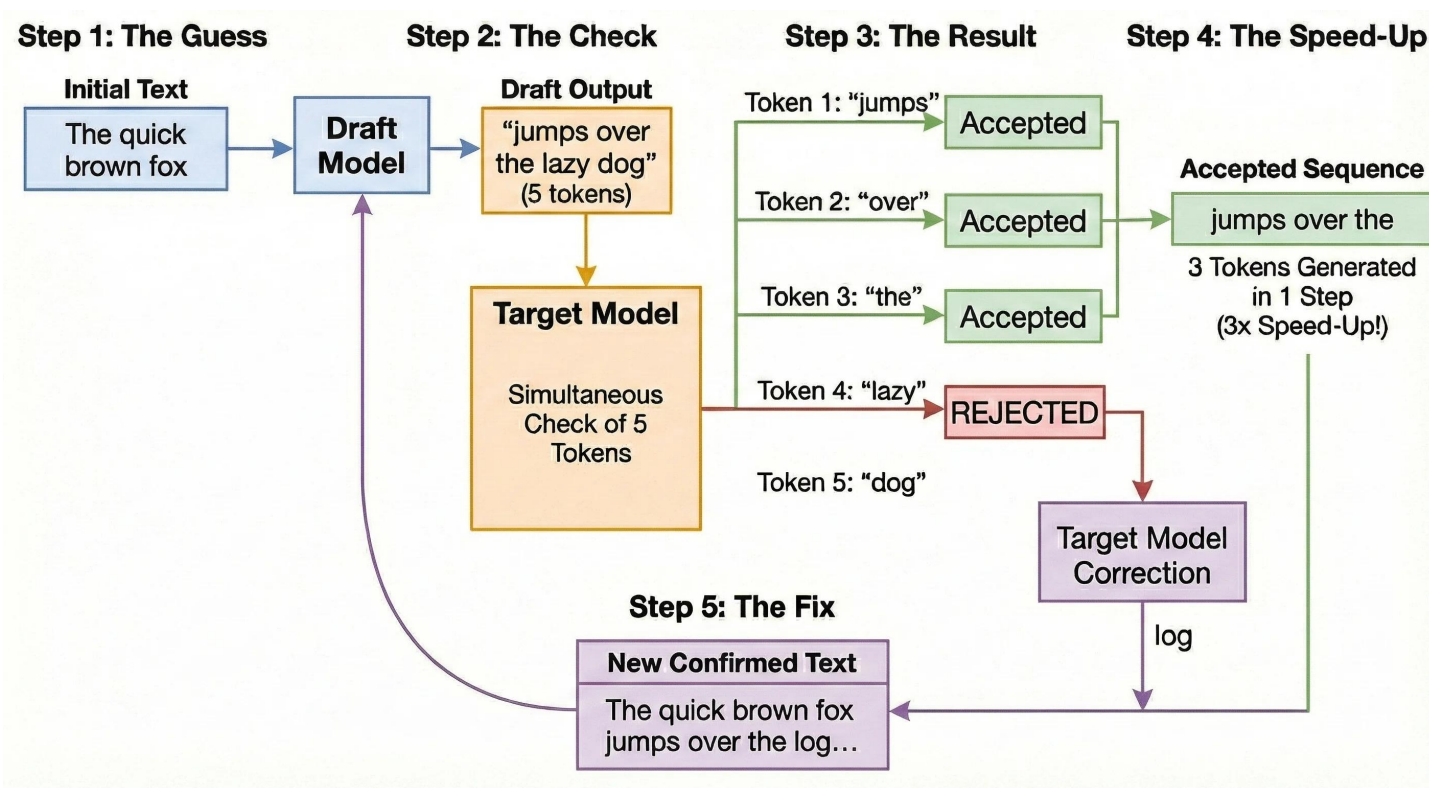
WorldAttention – Efficient Attention for World Models



Compared with existing approaches of interactive video generation, **WorldAttention** maintains stronger subject consistency, smoother motion transitions, and more stable scene structure across the entire sequence

Inference: Efficient Decoding

Speculative Decoding



Input: prefix x , draft model q , target model p , lookahead K

while not EOS:

1. Draft K tokens sequentially

$y_1 \dots y_K \sim q(\cdot \mid x)$

2. Target scores all drafted positions in parallel

compute $p(\cdot \mid x, y_{<1}), \dots, p(\cdot \mid x, y_{<K})$

3. Accept / reject

for $i = 1 \dots K$:

$\alpha = \min(1, p(y_i) / q(y_i))$

if $\text{Uniform}(0,1) \leq \alpha$:

append y_i to x

else:

sample $z \sim \text{normalize}(\max(p - q, 0))$

append z to x

break

4. If all K tokens accepted, sample one extra target token

if all accepted:

sample $z \sim p(\cdot \mid x, y_1 \dots y_K)$

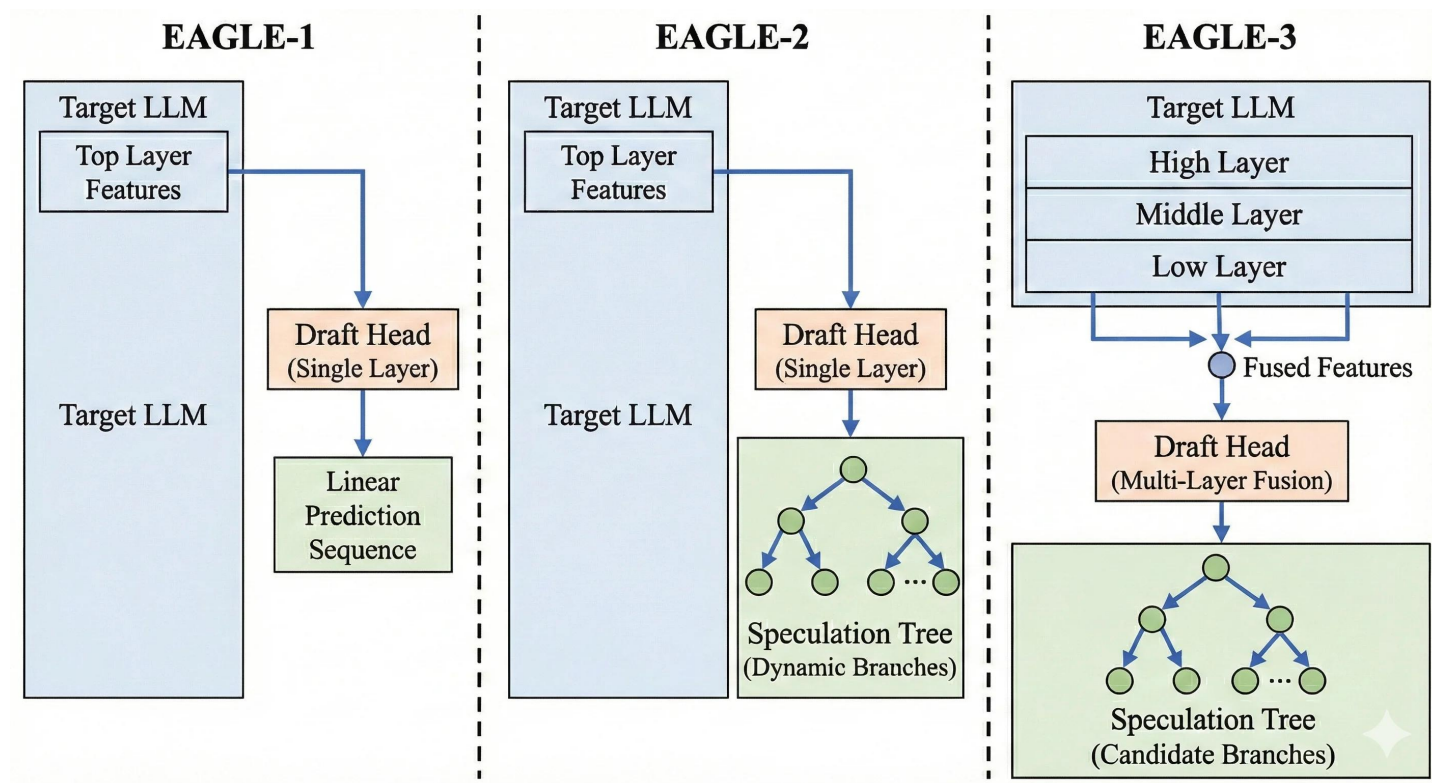
append z to x

Motivation: a target model can score all positions of the drafted sequence in parallel (prefilling), although the draft itself may be sequential (decoding). Correct acceptance and recovery preserve the target model's output distribution.

Limitation:

- Autoregressive decoding emits one token per step; decode is often memory-bandwidth bound.
- The target model requires a separate LLM.

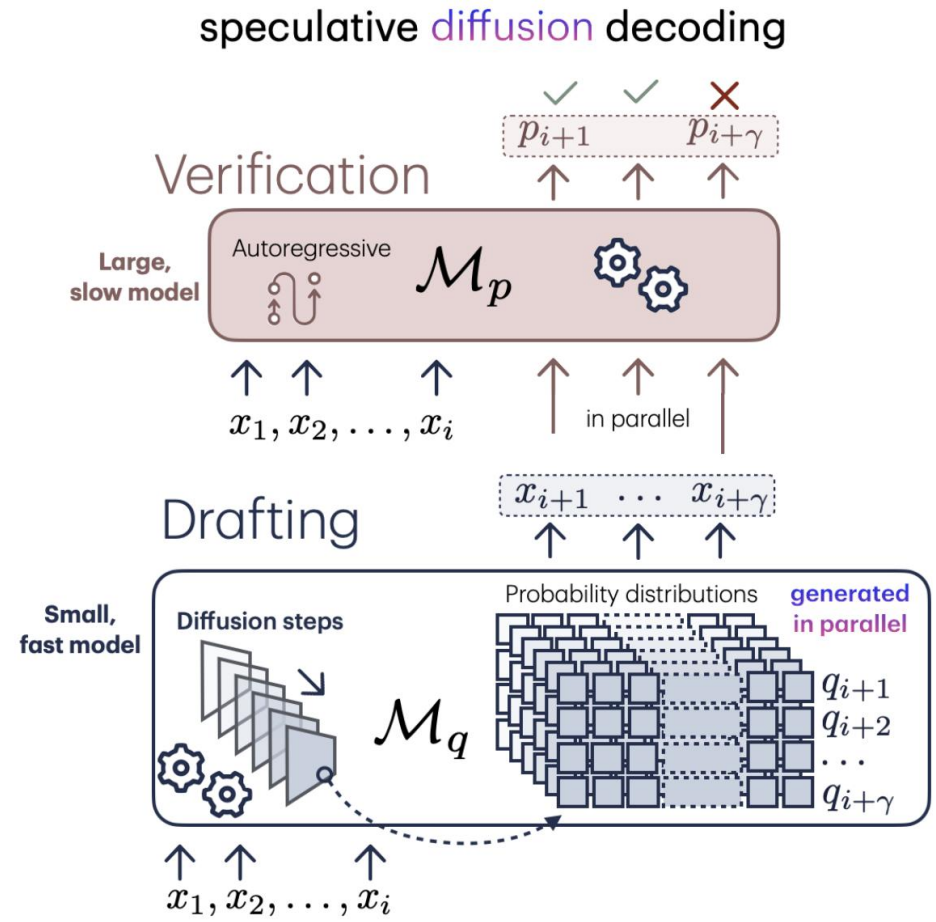
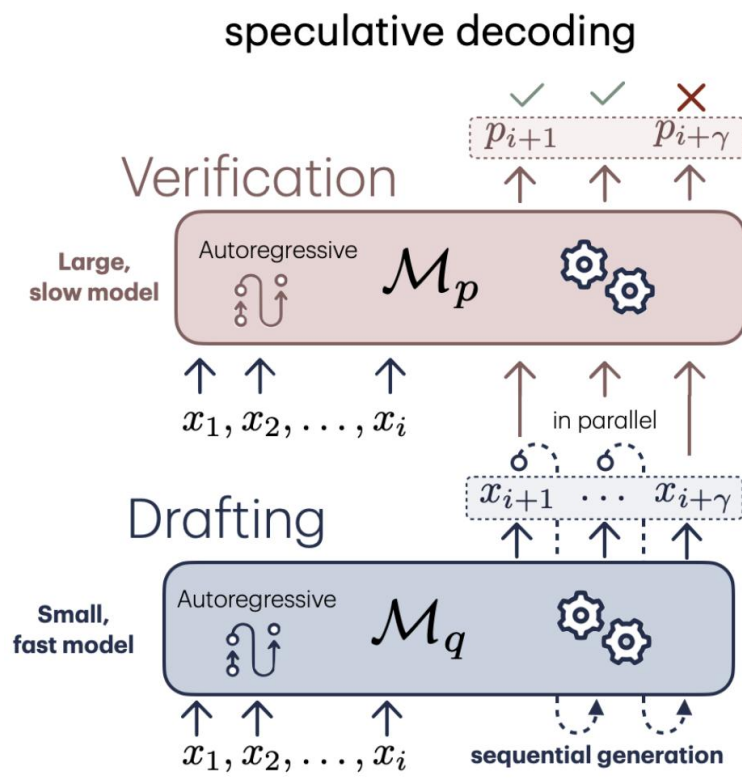
Multiple-Token Prediction (MTP)



Multiple heads → tree candidates → target verification

- **Predict several candidate tokens in parallel, then verify them with the target model.**
- Medusa/EAGLE family adds parallel decoding heads; top-k outputs form tree-structured candidate continuations.
- EAGLE-3 fuses multi-layer target features and predicts draft tokens directly.
- **The target model verifies many candidate positions in one batched forward pass.**
- More candidates can increase accepted tokens—but also add verification work (computation-bound).

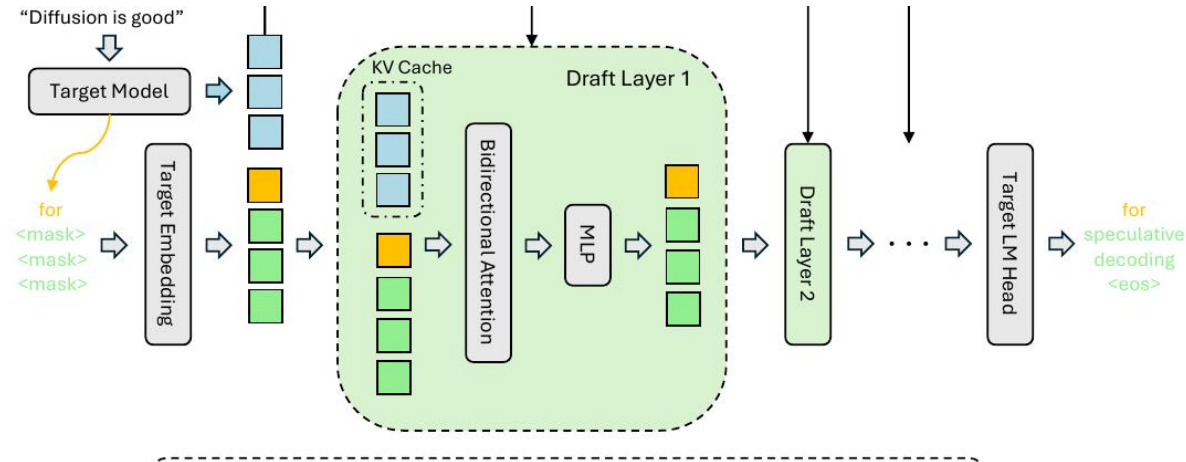
Diffusion-LLM based MTP



Diffusion-LLM based MTP (Drafter)

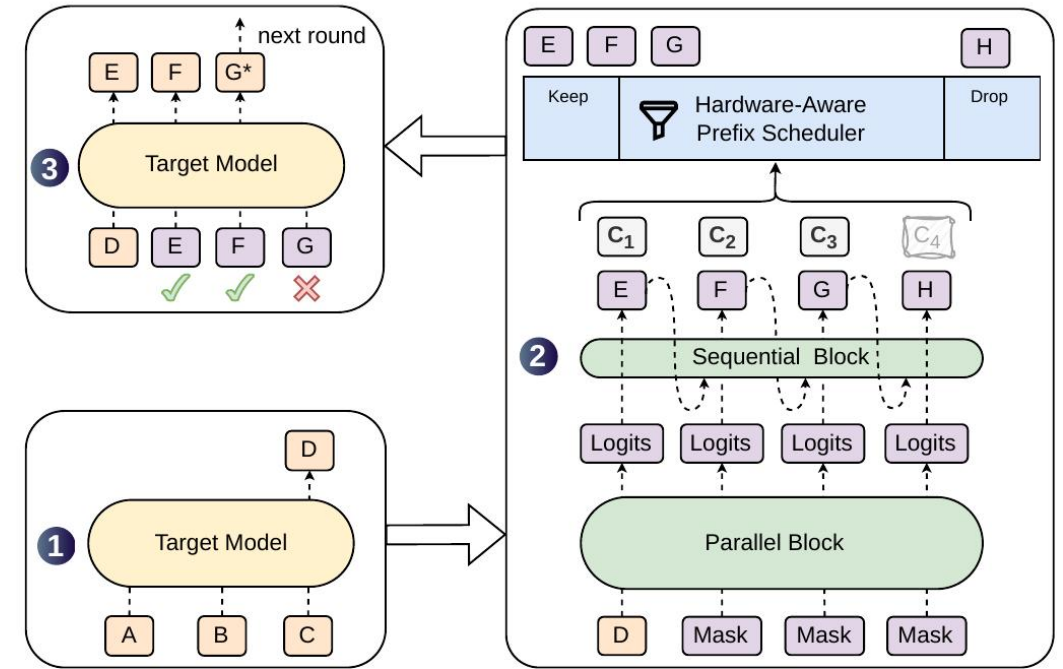
DFlash — block-diffusion parallel drafting

Target features → masked block → one-pass proposal → verify
(Source: DFlash)



DSpark — dependency-aware drafting + scheduled verify

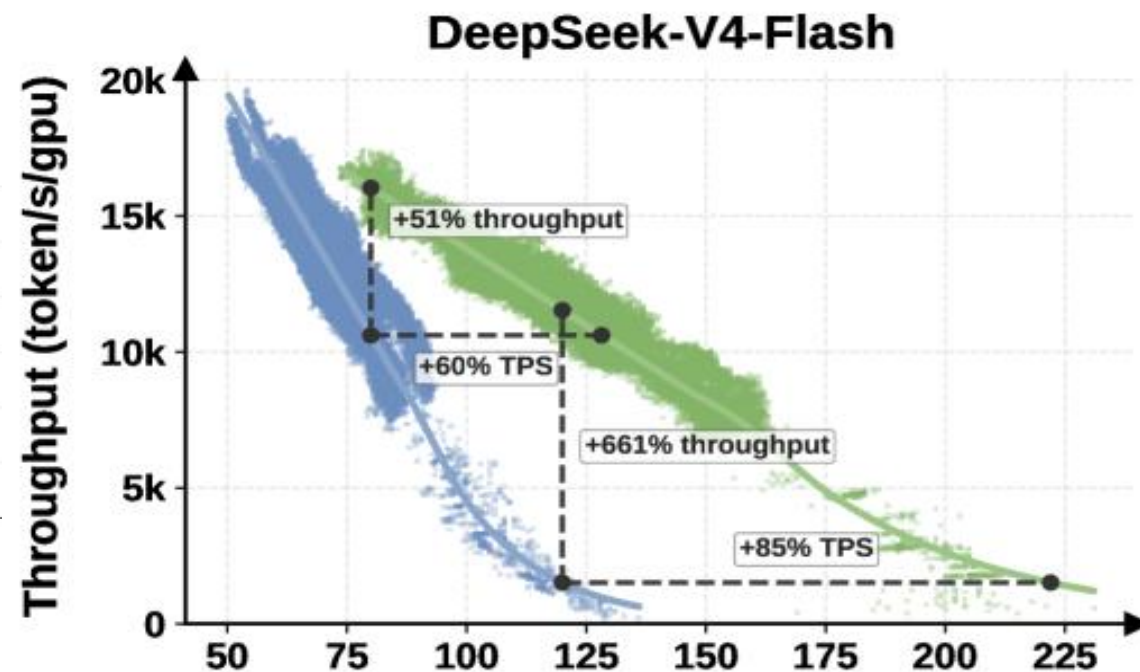
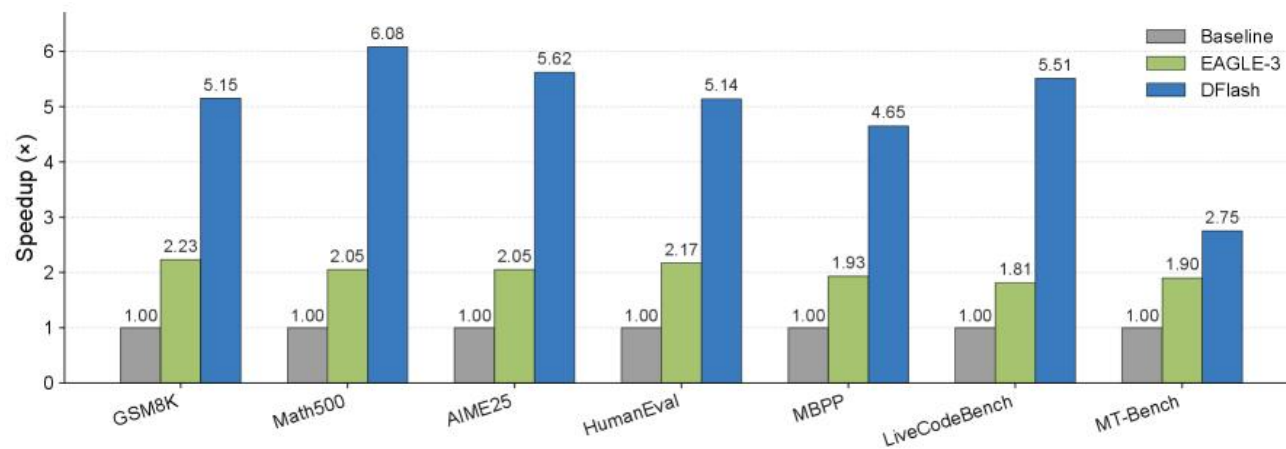
Parallel backbone → sequential block → confidence prefix → verify
(Source: DSpark)



DFlash minimizes draft latency; DSpark reduces wasted verification.

Trade-off: proposal quality and accepted length versus drafting latency, verification overhead, and serving complexity.

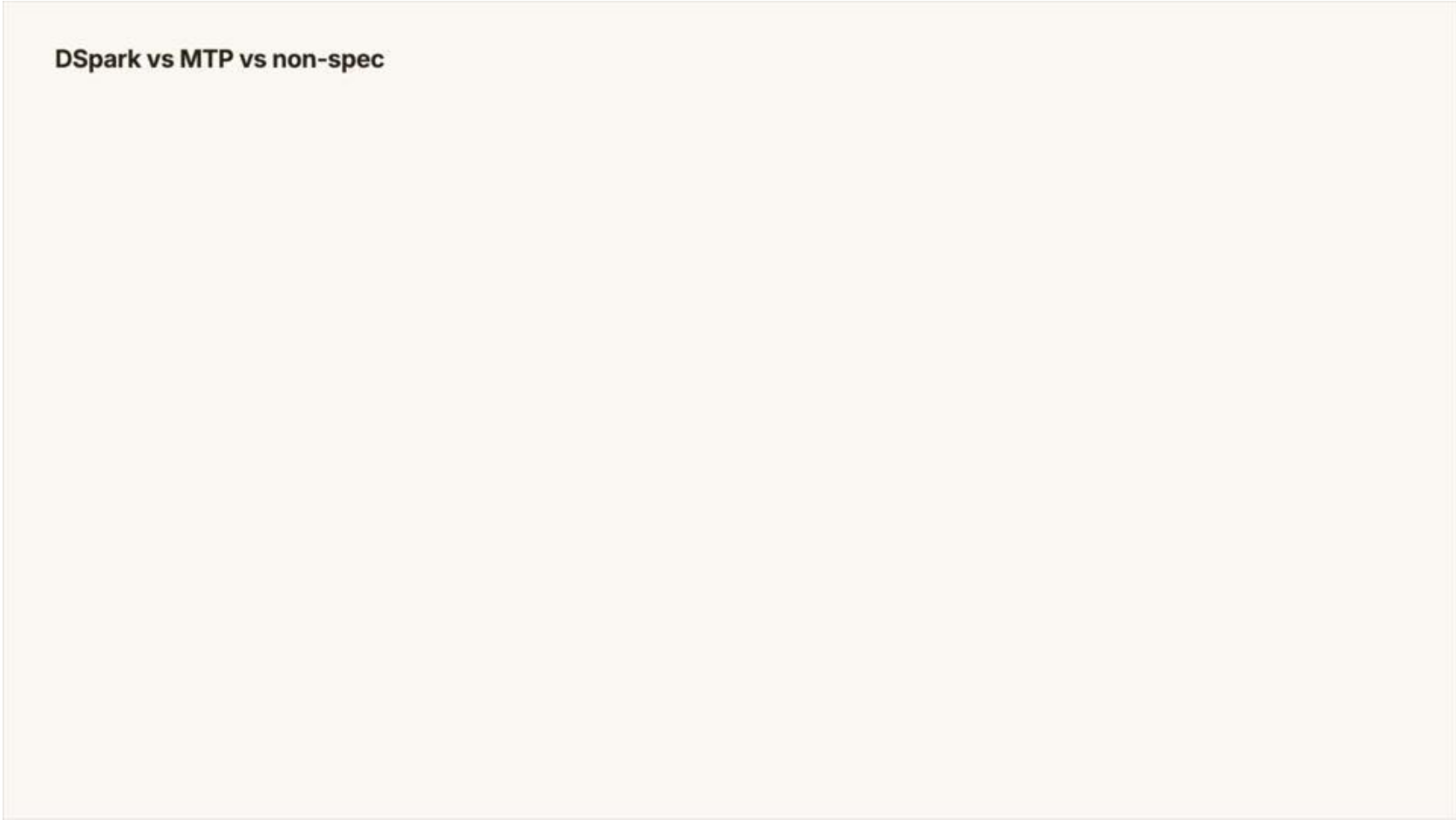
Measured Impact: Faster Drafting, More Efficient Verification



DFlash: 2.75–6.08× across tasks; DSpark: +60–85% per-user speed at matched throughput.

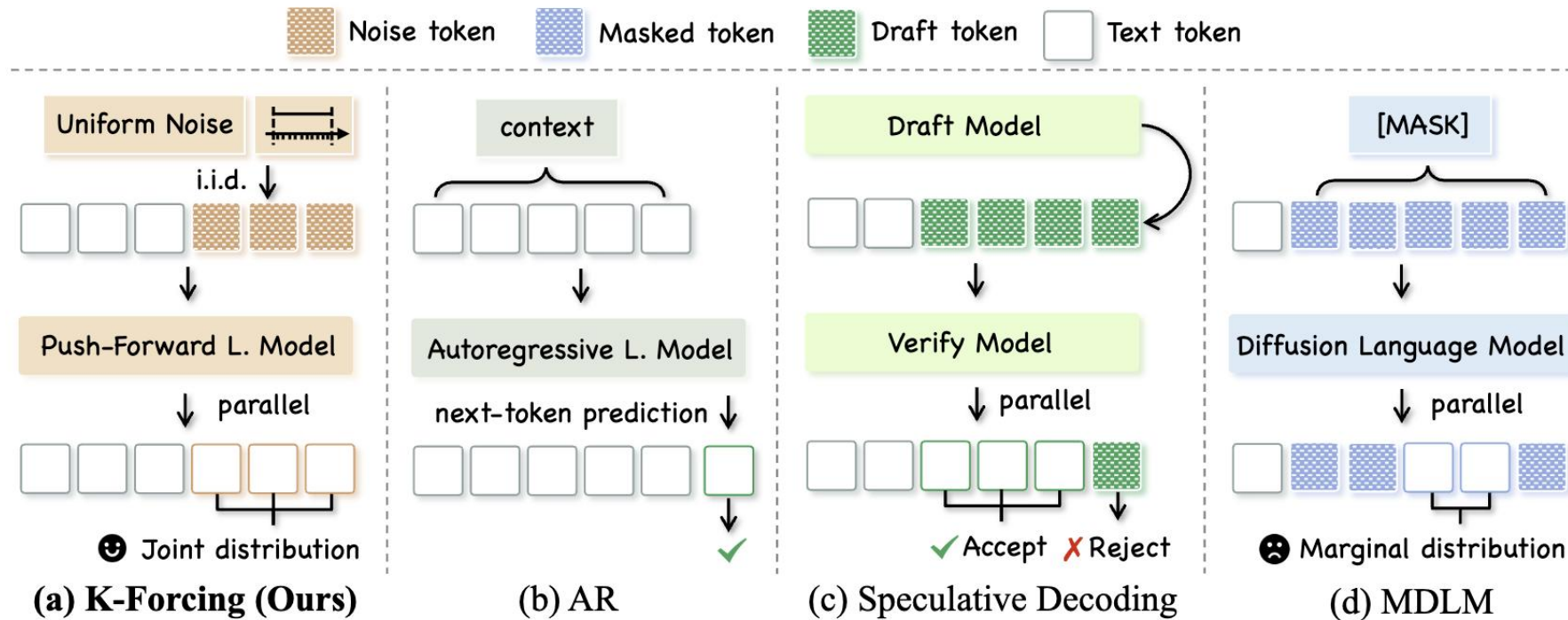
Diffusion-LLM based MTP

DSpark vs MTP vs non-spec



DSpark addresses a key bottleneck under load: instead of verifying every draft token, it verifies only where the draft model is confident, so the gains hold even as batch size scales.

K-Forcing – Next-K-Token Decoding for Language Models



Observation: (b) **AR** leads to memory-bound decoding. (c) **Speculative decoding** yields a variable number of accepted tokens that breaks regular batching. (d) **MDLM** predicts masked positions in parallel from per-position marginals, rather than their joint distribution.

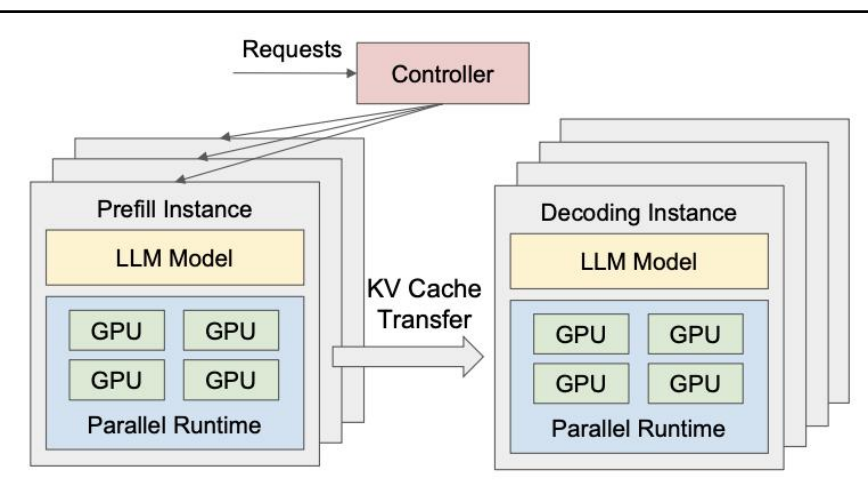
Solution: (a) **K-Forcing** uses a push-forward language model to map i.i.d. uniform noise tokens to a fixed-length block of future tokens, modeling their joint distribution.

Infrastructure and Inference Frameworks

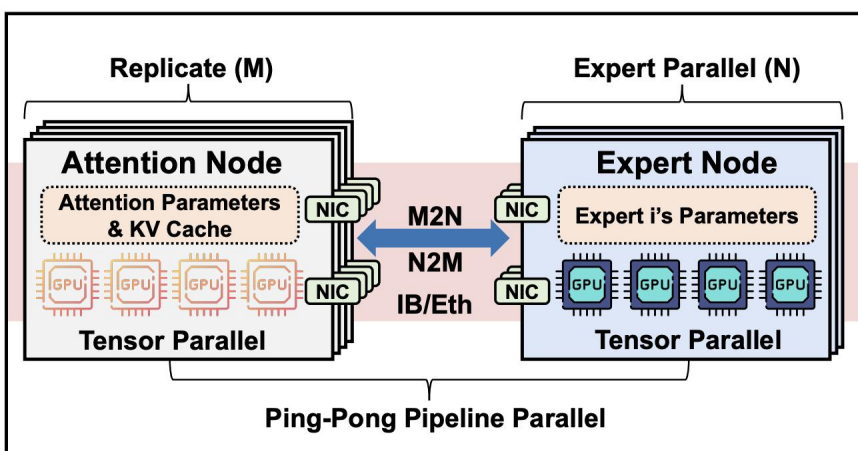
Framework	Primary Focus	Key Techniques	Best Fit
SGLang	Production serving	RadixAttention; prefix cache; structured outputs; spec decode	Low-latency / high-throughput APIs and clusters
vLLM	General-purpose serving	PagedAttention; continuous batching; prefix cache; speculative decoding	Broad model and hardware ecosystem
FlashInfer	GPU kernel library	Attention, paged KV, GEMM, quantized and MoE kernels	Custom serving stacks and kernel backends
KTransformers	CPU–GPU hybrid MoE	CPU expert kernels; offload/scheduling; quantization	Large sparse models with limited GPU memory
TensorRT-LLM	NVIDIA inference runtime	In-flight batching; paged KV; quantization; multi-GPU; spec decode	NVIDIA-optimized production deployment

Choose by model structure, hardware, concurrency, latency target, and workload—not by a universal “fastest” label.

Infrastructure and Inference Frameworks



PD Disaggregation



AF Disaggregation

Motivation:

1. The property of different inference phases (Prefill vs Decode) and different model modules (Attention vs FFN) is greatly different.

Prefill: Compute-bound

Decode: Memory-bound

Attention: Massive memory access

FFN: Pure computation

1. The property of different GPUs is greatly different.

H20: Bandwidth-rich but compute-poor

Memory-bound

H100: Better compute but poorer bandwidth

Compute-bound

Method:

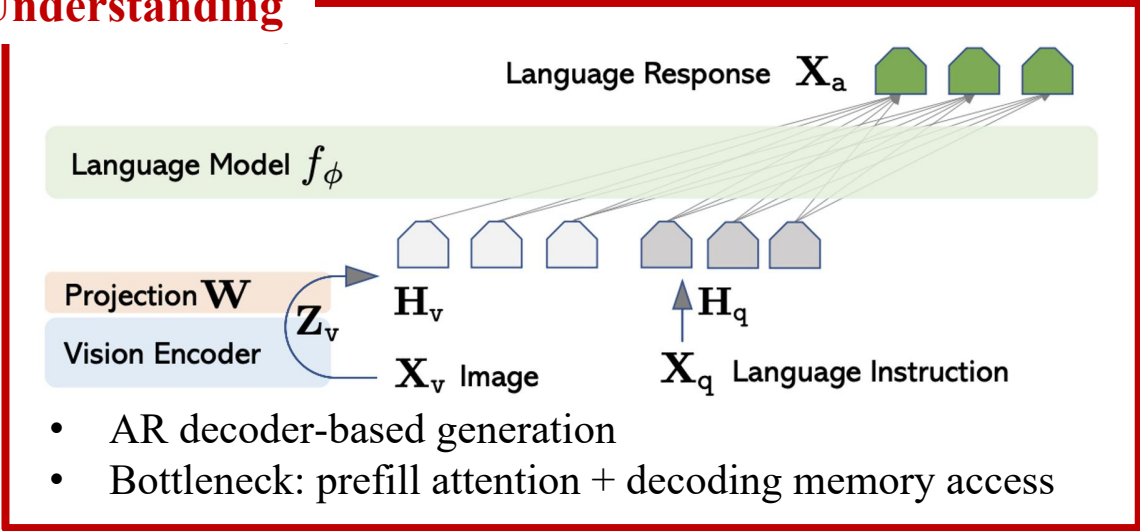
In a word, separate different inference phases / model modules in different GPU instance

Prefill vs Decode → PD Disaggregation

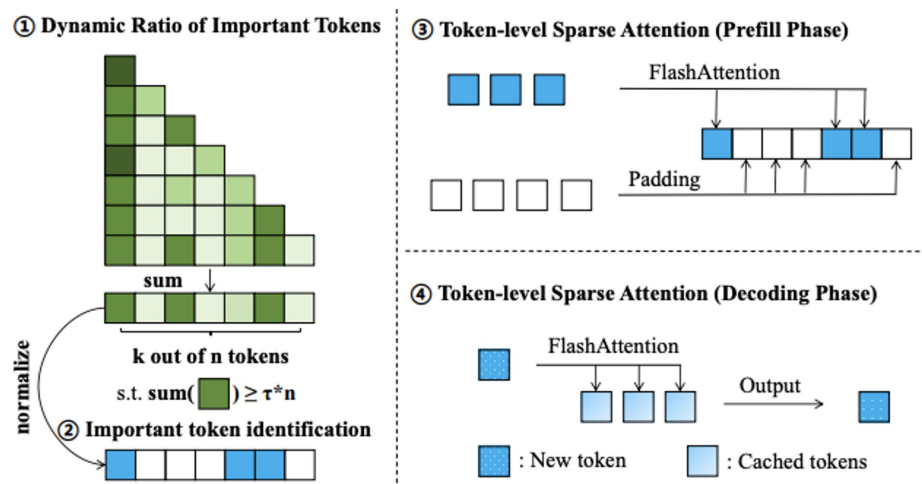
Attention vs FFN → AF Disaggregation

Inference Acceleration in Multimodal Understanding and Generation

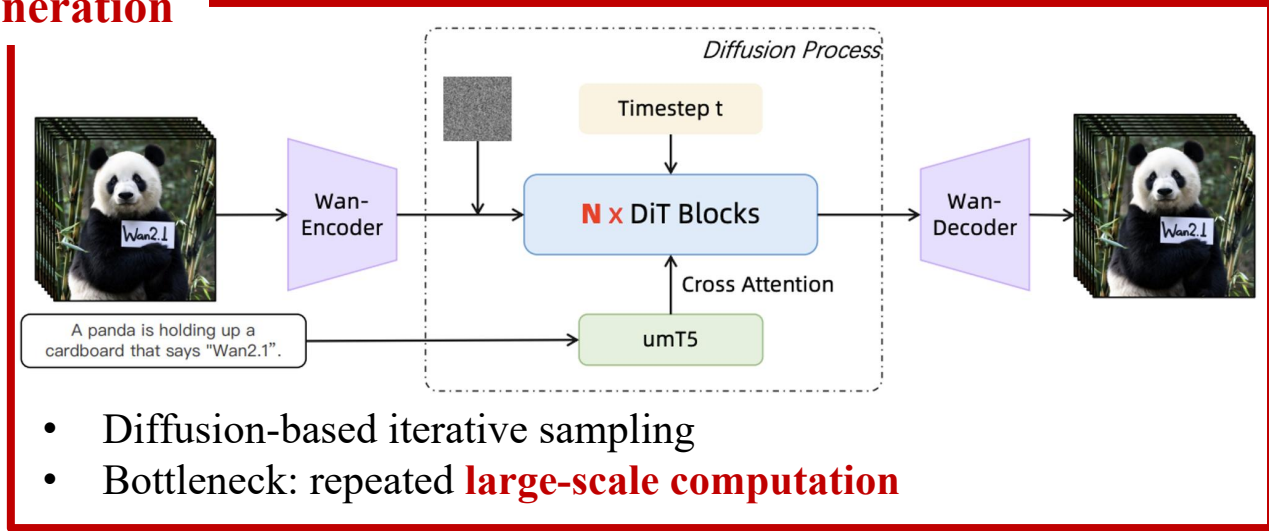
Understanding



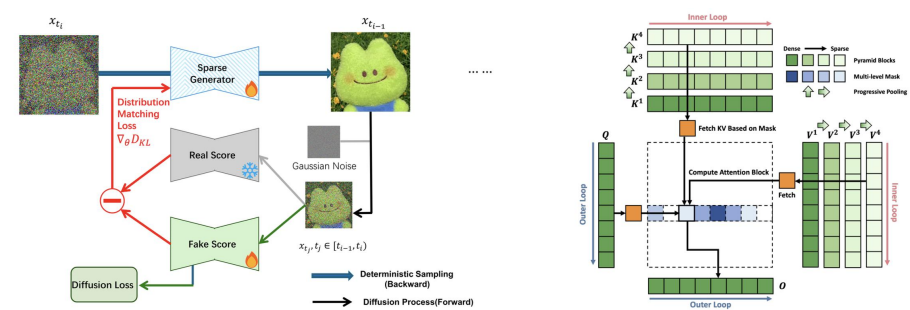
- **Sparse Attention: ZipVL** ICCV 2025
 - Efficient Approximation of KV Importance
 - Prefilling: Skip computations for unimportant ones
 - Decoding: Unimportant KV tokens will not be loaded



Generation



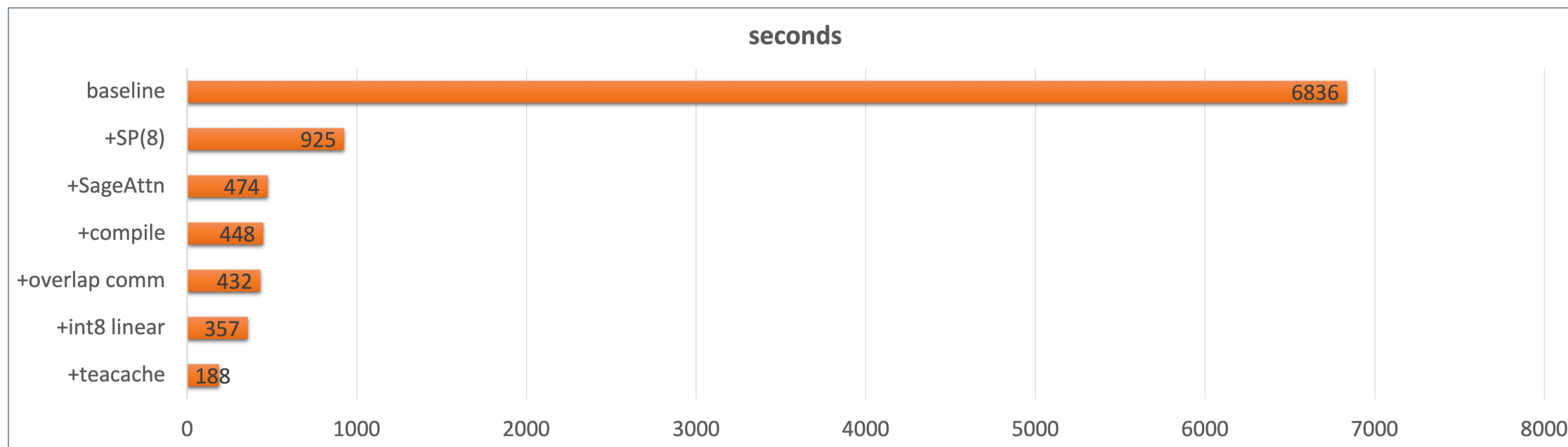
- **Reduce Denoising Steps**
Examples: VideoLCM, T2V-Turbo, DOLLAR
- **Reuse Computation Across Denoising Steps**
Examples: TeaCache, FasterCache
- **Sparse/Linear Attention**
Examples: SVG, BLADE ICLR 2026, **PSA** CVPR 2026



VideoGen Infra – DAX

DAX (Diffusion Accelerated eXecution): High performance inference engine for diffusion models (5s base)

Wall time for generating one video with Wan2.1 T2V 14B, 720p, 5s, 50steps, on Nvidia H20, with DAX



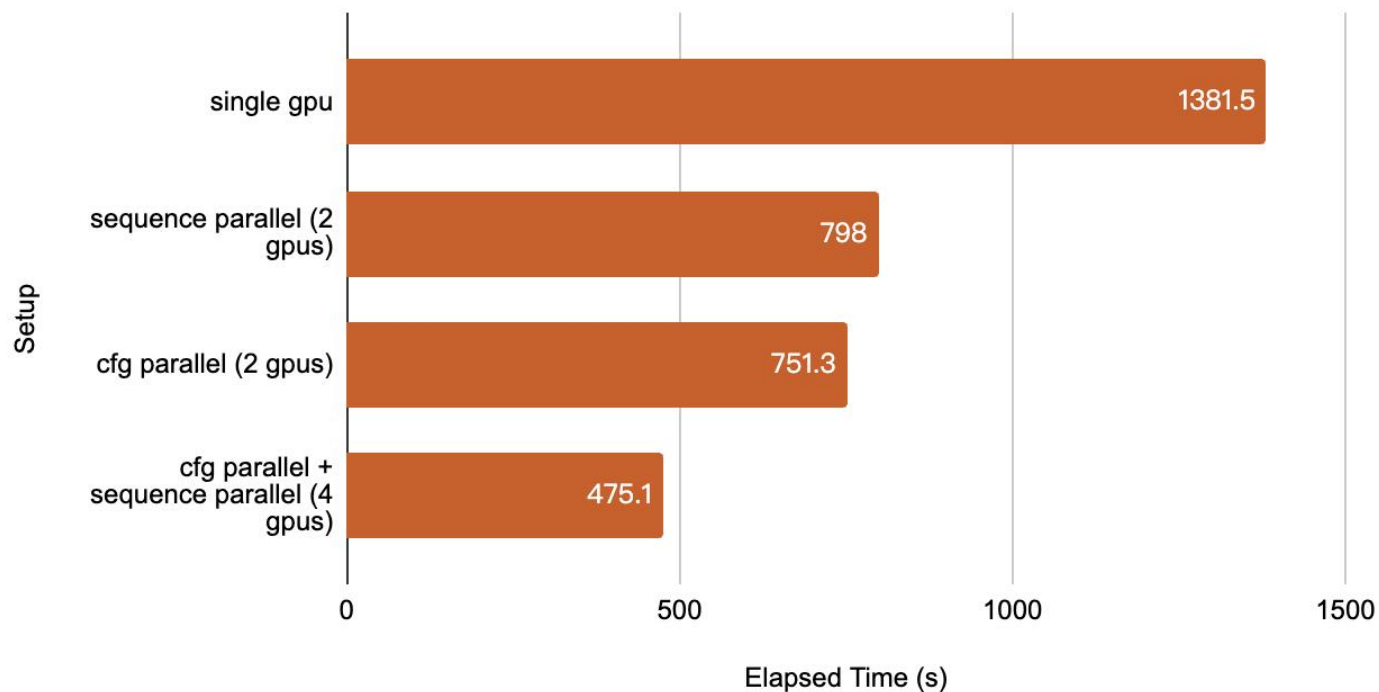
- **Quantization:** FP8/INT8 GEMM and SageAttention2 for attention.
- **Sequence parallelism** with carefully tuned communication overlap.
- **Teacache**, accelerating DiT model inference by skipping trivial denoising steps.
- **Torch.compile:** fusing quantization and communication ops.

<https://github.com/RiseAI-Sys/DAX>

VideoGen Infra – SGLang Diffusion

SGLang-Diffusion: High-performance inference and serving framework for image and video generation.

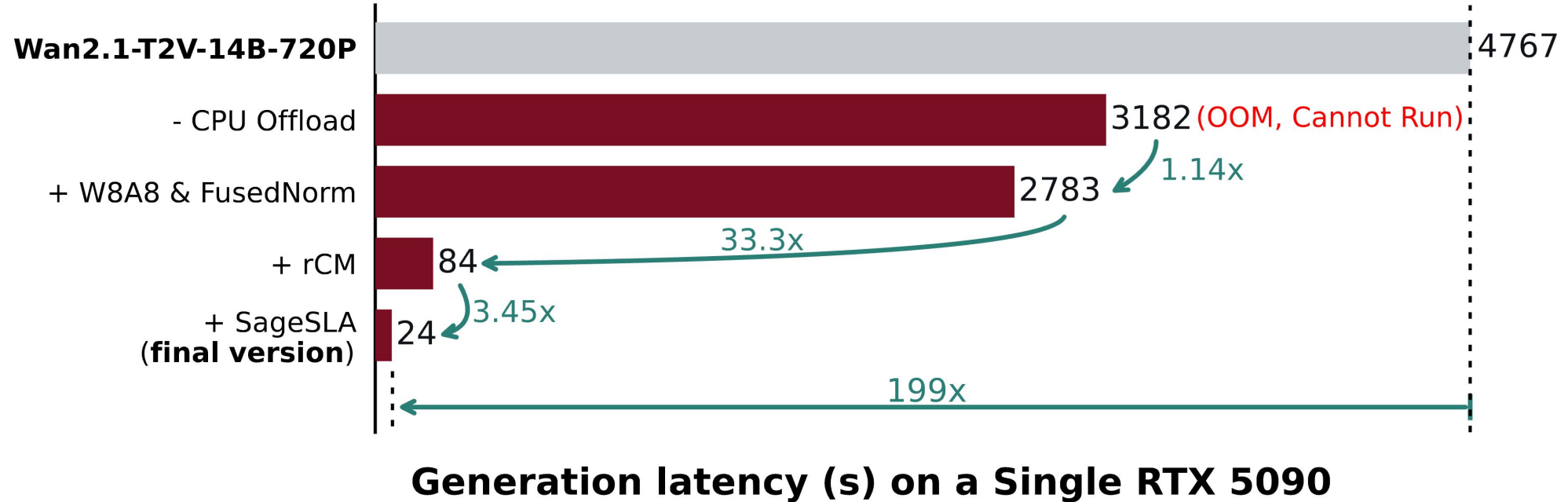
SGLang Diffusion Performance Under Different Parallelism Setups



- **Attention backends:** FlashAttention, SageAttention and multiple sparse-attention implementations.
- **Quantization & caching:** FP8/NVFP4/INT4 quantization, with Cache-DiT and TeaCache.
- **Parallel serving:** Ulysses/Ring sequence parallelism, TP/FSDP/CFG parallelism.

VideoGen Infra – Turbo Diffusion

TurboDiffusion: Model-system co-designed acceleration framework for video diffusion models (100–200× speedup).



- **W8A8 quantization:** quantizing both weights and activations to accelerate linear layers and reduce memory.
- **rCM step distillation:** reducing diffusion sampling to only 1–4 denoising steps.
- **SageSLA:** combining low-bit SageAttention with trainable Sparse-Linear Attention.
- **Kernel optimization:** fused normalization and quantized GEMM kernels.

Block Diffusion (Autoregressive Diffusion, Semi-AR)

Block Diffusion

Autoregression:

✓ High quality

✓ Arbitrary-length

✓ KV caching

✗ Not Parallelizable

Generation steps

There are three categories of the average
There are three categories of the average rate
There are three categories of the average rate of...

Diffusion:

✗ Lower quality

✗ Fixed-length

✗ No KV caching

✓ Parallelizable

the reusability will continue to the
Repeal the reusability cuts and the law will continue to reduce the
Repeal the reusability cuts and prove the law will continue to reduce the deficit.

Block Diffusion (Ours):

✓ High quality

✓ Arbitrary-length

✓ KV caching

✓ Parallelizable

On September 17, we be
On September 17, 2016, we will be giving the release of
On September 17, 2016, we will be giving the beta-release of the to our server testing ...

Figure from BD-3 [1]

Block Diffusion for Video Generation

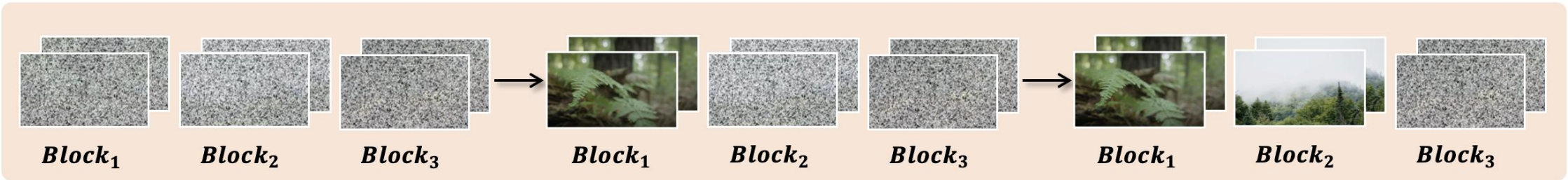
(1) Autoregressive ✓ Arbitrary-length ✓ KV caching ✗ Not Parallelizable



(2) Diffusion ✗ Fixed-length ✗ No KV caching ✓ Parallelizable



(3) Block Diffusion ✓ Arbitrary-length ✓ KV caching ✓ Parallelizable



Noisy video token

Clean video token

Video block

Inferix: An Inference Engine for World Simulation

- **Advanced KV Cache Management**

Intelligent memory management for persistent world simulation

- **Distributed World Synthesis**

Large-scale immersive environment generation across devices

- **Interactive Generation**

Real-time preview with Gradio UI; pause / resume / stop and on-the-fly prompt changes — runs on 16 GB consumer GPUs

- **Continuous Prompt Support**

Dynamic narrative control: different prompts for different video segments

- **Video Streaming**

Stream generated content live via RTMP and WebRTC

- **Quantized Inference**

8-bit (INT8 / FP8) quantization, per-tensor or per-token-per-channel, with DAX support

- **Built-in Profiling**

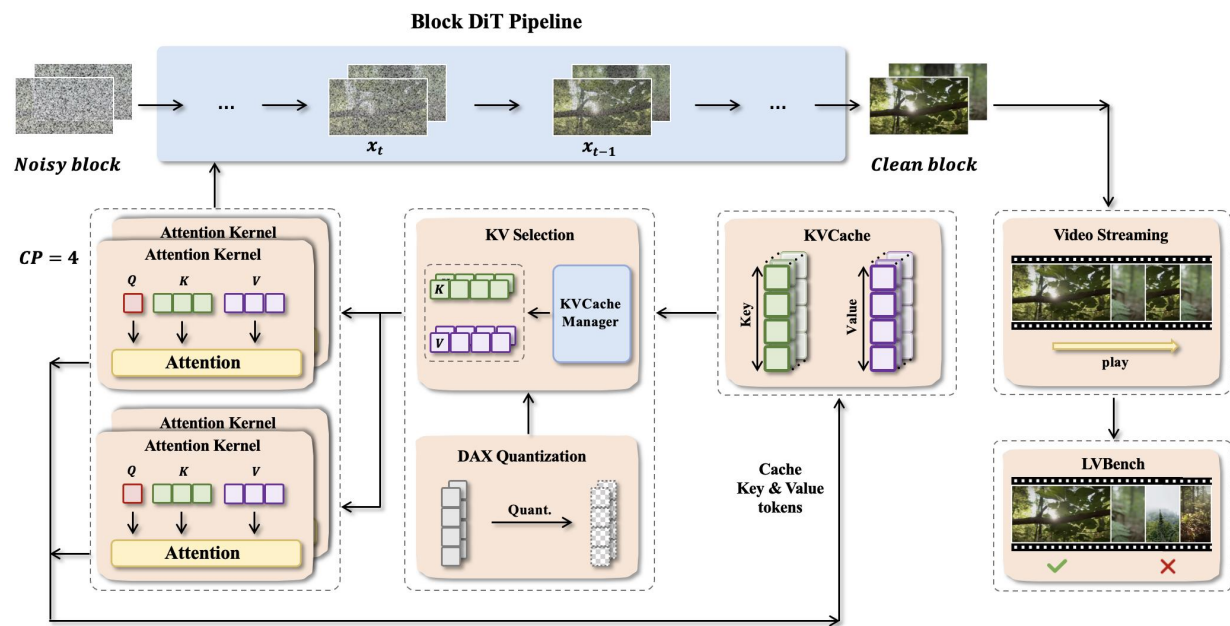
Performance monitoring with enhanced diffusion-model profiling

- **Seamless Model Integration**

Simple API for deploying new world models

- **Next-Gen Architecture**

Purpose-built for immersive world synthesis at scale



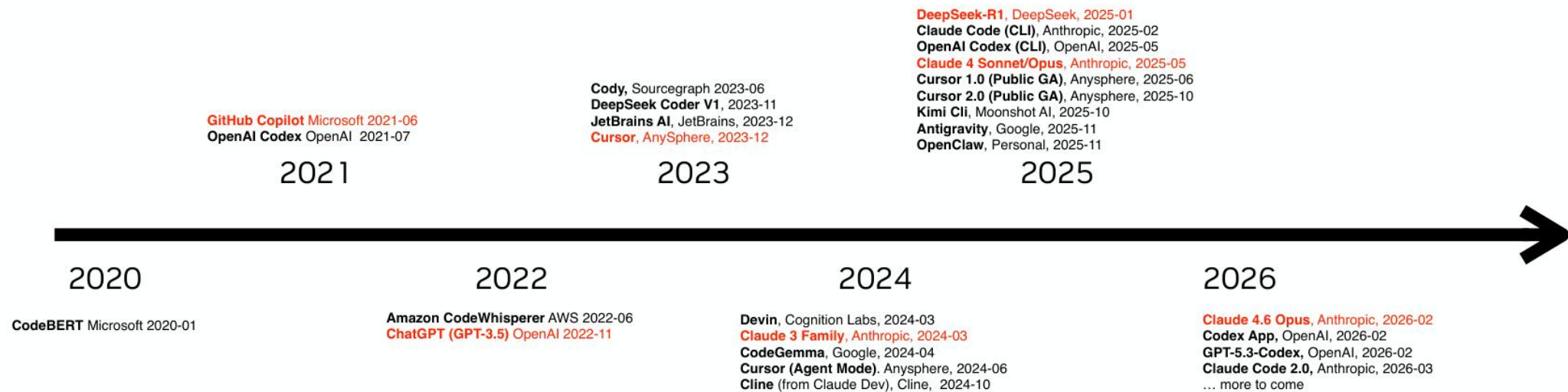
Takeaway: Inferix serves block-diffusion world models interactively, in real time — even on consumer GPUs.

Part 4 – Agent Loops

The Progress of AI Coding: Towards Software AGI

2020–22 Autocomplete (ghost text) → **2023** RAG context → **2024** Agentic shift (tools that act) → **2025** Reasoning, multi-step agents → **2026** Autonomous operations

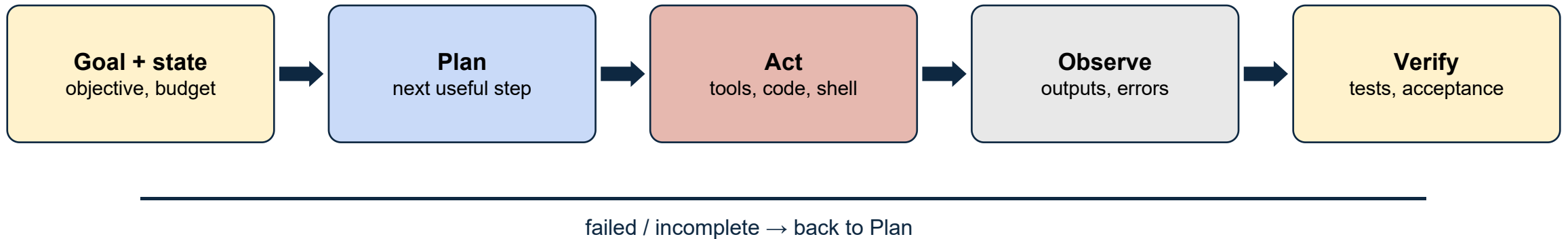
- From ticket to merged PR: parallel agents assign, delegate, and verify tasks as a team
- In 2026, we are at **the dawn of Software AGI**



AI coding compounds like the lily pond — if your workflow is still in the 2022–24 era, upgrade it now.

What is an Agent Loop?

- **Definition:** an agent loop is a program that repeatedly lets a model plan, act with tools, observe the results, and verify progress — until the goal is met or a stop condition fires.



- **Durable state** (files, memory, experiment records, checkpoints) lets iterations compound beyond one context window.

The human role shifts to defining goals, permissions, budgets, and stop conditions.

Agent Loops for Research: Karpathy's AutoResearch

program.md — the whole harness is one natural-language prompt

```
# program.md

LOOP FOREVER:

1. Look at the git state: the current branch/commit we're on
2. Tune `train.py` with an experimental idea by directly hacking the code.
3. git commit
4. Run the experiment: `uv run train.py > run.log 2>&1` (redirect everything — do NOT use tee or let output flood your context)
5. Read out the results: `grep "^val_bpb:\||^peak_vram_mb:" run.log`
6. If the grep output is empty, the run crashed. Run `tail -n 50 run.log` to read the Python stack trace and attempt a fix. If you can't get things to work after more than a few attempts, give up.
7. Record the results in the tsv (NOTE: do not commit the results.tsv file, leave it untracked by git)
8. If val_bpb improved (lower), you "advance" the branch, keeping the git commit
9. If val_bpb is equal or worse, you git reset back to where you started

**Timeout**: Each experiment should take ~5 minutes total (+ a few seconds for startup and eval overhead). If a run exceeds 10 minutes, kill it and treat it as a failure (discard and revert).

**Crashes**: If a run crashes (OOM, or a bug, or etc.), use your judgment: If it's something dumb and easy to fix (e.g. a typo, a missing import), fix it and re-run. If the idea itself is fundamentally broken, just skip it, log "crash" as the status in the tsv, and move on.

**NEVER STOP**: Once the experiment loop has begun (after the initial setup), do NOT pause to ask the human if you should continue. Do NOT ask "should I keep going?" or "is this a good stopping point?". The human might be asleep, or gone from a computer and expects you to continue working indefinitely until you are manually stopped. You are autonomous. If you run out of ideas, think harder — read papers referenced in the code, re-read the in-scope files for new angles, try combining previous near-misses, try more radical architectural changes. The loop runs until the human interrupts you, period.
```

- An LLM as an overnight research intern: hack train.py with an experimental idea → run → read val_bpb
- **Verifiable metric as the reward**: keep the commit if the loss improves, otherwise git reset — every idea judged by the same number
- **"NEVER STOP"**: no pausing to ask the human; out of ideas → read the papers, combine near-misses, try more radical changes
- Runs 7×24 on a GPU node until manually interrupted — **vibe research**, beyond vibe coding

A one-page natural-language loop turns an agent into a 7×24 self-driven experimenter.

Learning without Gradients/Heuristic Learning/Continuous Learning

- **Heuristic Learning (HL)/Continuous Learning:** a coding agent improves a programmatic policy from feedback — no gradient updates, the "weights" are code.

Deep RL vs. Heuristic Learning

Axis	深度强化学习	启发式学习（HL）
Policy	神经网络参数	代码：规则、状态机、控制器、MPC、宏动作
State	通常为显式观测	显式变量、检测器、缓存——可读的表示
Action	一次神经网络前向传播	执行代码逻辑
Feedback	主要是固定奖励	测试、环境反馈、日志、回放、人工输入
Update	基于梯度更新神经网络参数	编码智能体直接修改代码
Memory	同策略：极少；异策略：经验回放缓冲区	试验、摘要、失败原因、回放、版本差异

Core shift: learning becomes maintaining a software system with explicit update paths.

[1] Jiayi Weng, Learning Beyond Gradients, blog, May 2026.

Continual Learning Becomes Software Maintenance

- **A healthy Heuristic System balances three duties:**

- **Absorb feedback** — failures, logs, rewards, human corrections
- **Preserve capabilities** — regression tests, fixed-seed replays, golden traces
- **Compress history** — refactor local patches into simpler code

- **Advantages over Deep RL:**

Explainability — readable program logic

Overfitting control — multi-seed evaluation

Sample efficiency — large gains from one code update

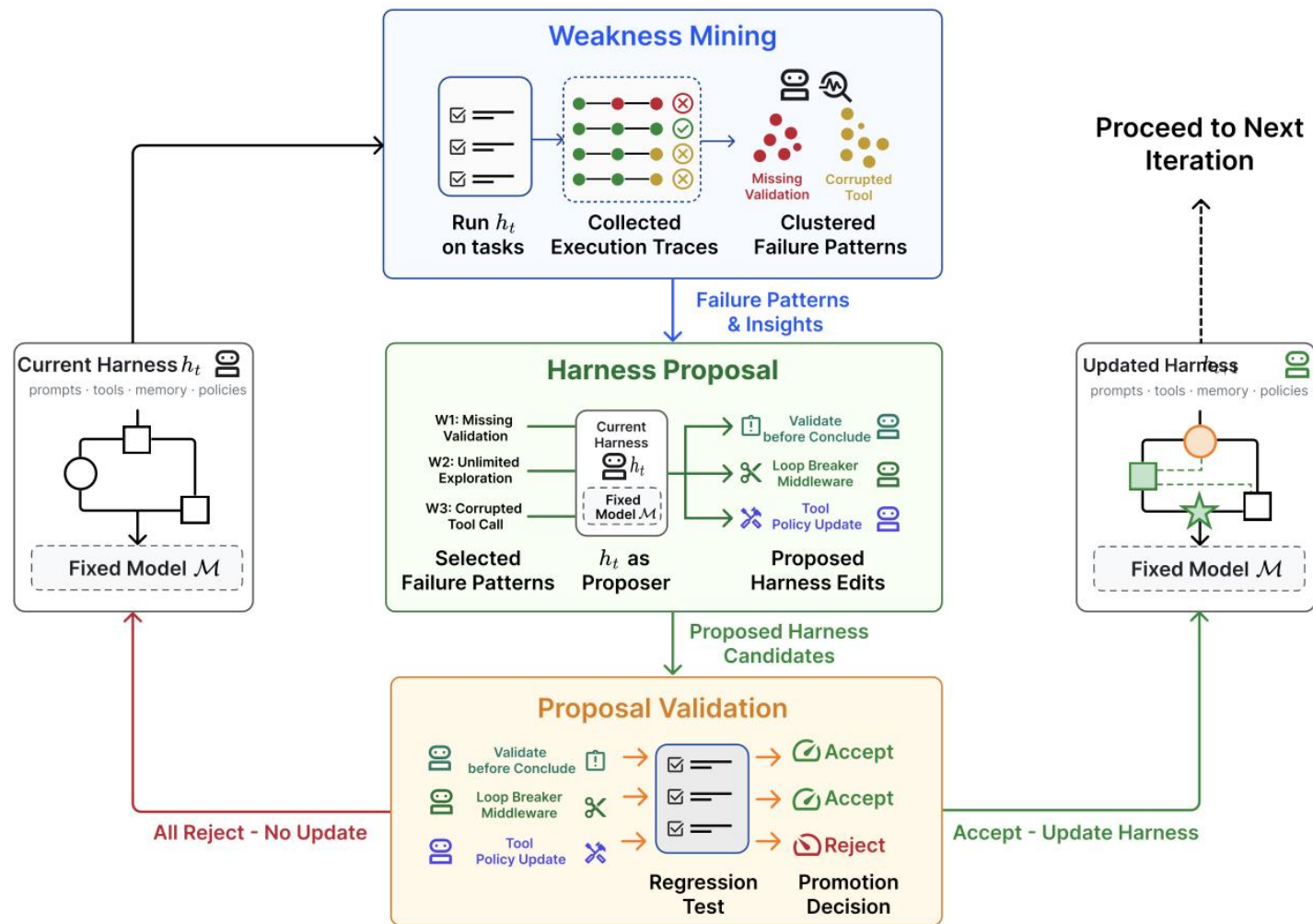
Reduced forgetting — capabilities persist in rules and tests

Regression-testability — tests, replays, golden cases

- **Bi-level Optimization:** HL = System 1 (real-time control) + LLM agent = System 2 (improves the system, distills data for weight updates)

Forgetting becomes an engineering problem: tests + replays + refactoring, not parameter tricks.

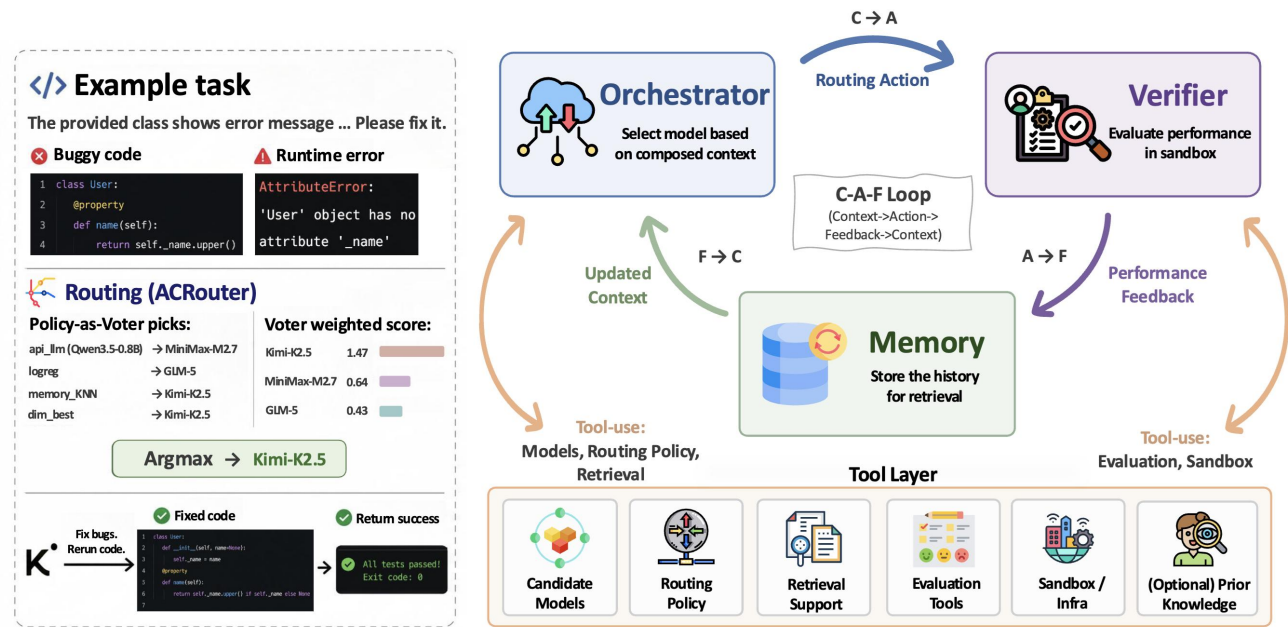
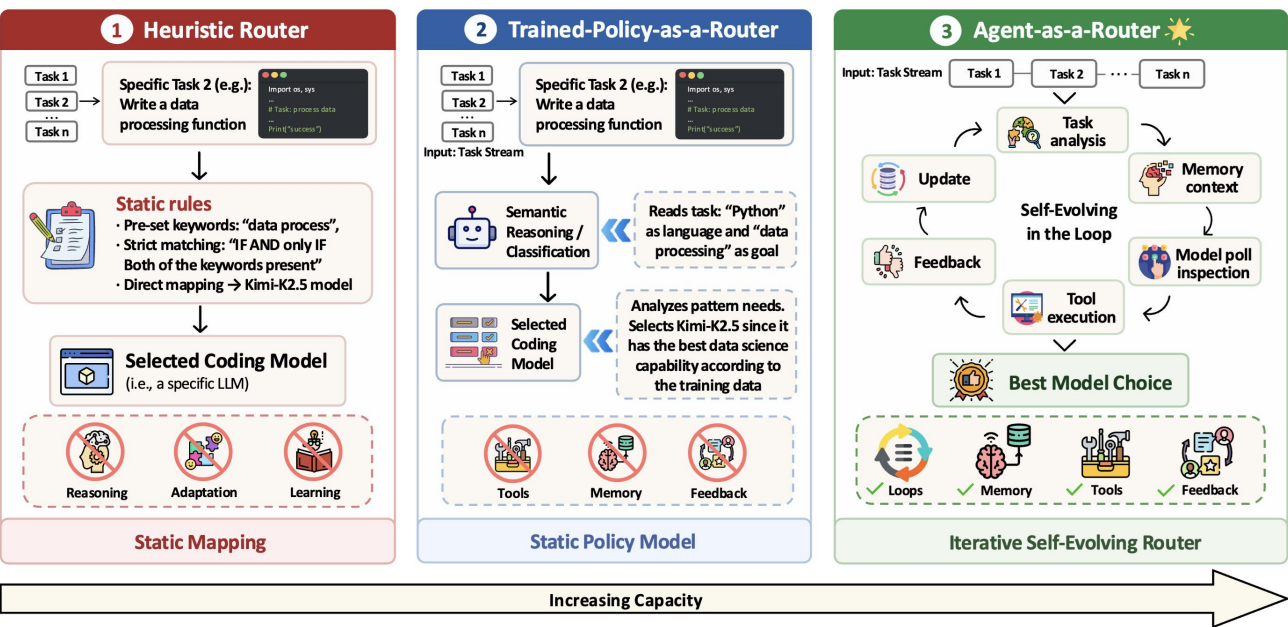
Self-improving Harness: Optimize the Code that Executes Agents



- **Self-Harness: a propose–evaluate–accept loop**
 - **Weakness mining** — cluster failures into verifier-grounded root-cause patterns
 - **Harness proposal** — bounded edits on editable surfaces; target recurrent, addressable patterns
 - **Validation** — accept only edits with no regression on held-in + held-out splits
- **Findings & cautions:**
 - **Boundaries** — permissions & security must live outside the loop; reward hacking remains

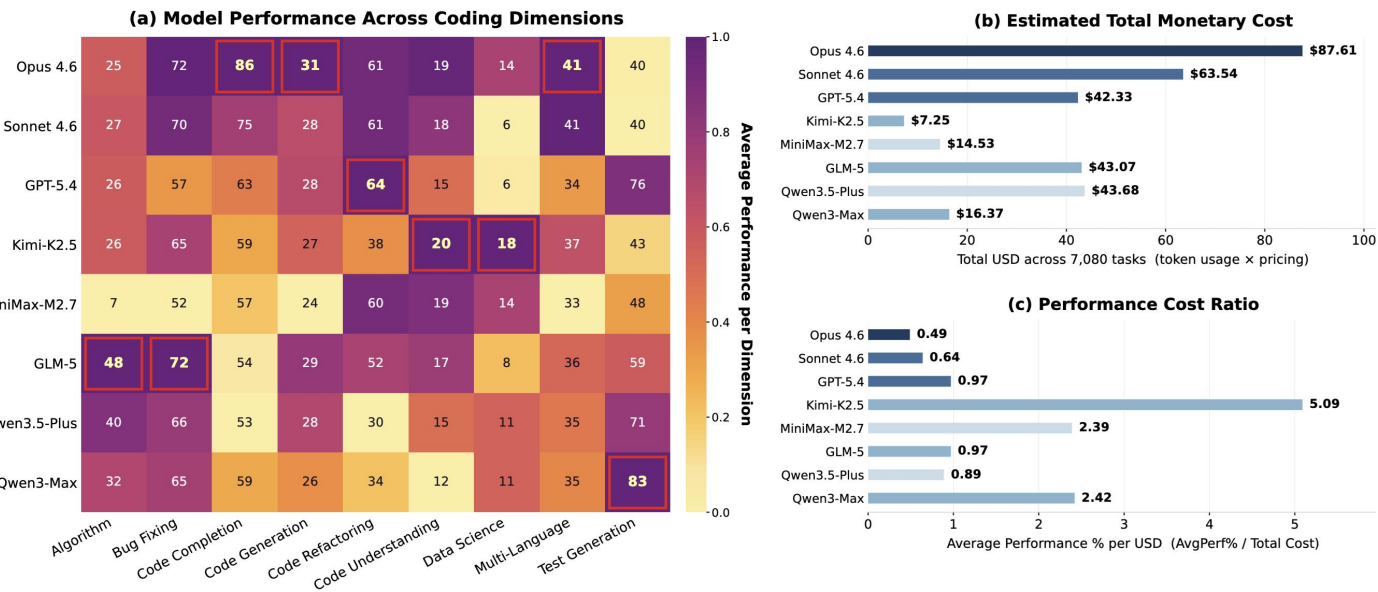
The harness itself becomes a learned artifact

Example: Agent-as-A-Router



Agent-as-a-Router: Agentic Model Routing for Coding Tasks, arxiv 2026

Example: Agent-as-A-Router



在3000个编码任务上测试，**agentic router**的框架的平均任务得分高于**opus**的分数，并且成本不到**opus**的一半。

Table 3 | Routing results, grouped by component-configuration taxonomy. **Left**: in-distribution test across 9 single-turn coding dimensions. **Right**: real-world OOD test on agentic programming. **CumReg**: cumulative regret across all tasks. **Perf/\$**: AvgPerf% per USD. DimensionBest is not applicable to OOD test as it requires predefined dimension-to-model mapping, which is unavailable for unseen agentic-programming tasks. The full breakdown is in the appendix.

Router	In-Distribution (n=2,919)			OOD Test (n=176)		
	AvgPerf%↑	CumReg↓	Perf/\$↑	AvgPerf%↑	CumReg↓	Perf/\$↑
Oracle	57.00	0	8.20	75.89	0	2.32
Agent-as-a-Router						
ACRouter (ours)	49.98	205.5	3.79	62.50	17.0	1.18
Dynamic: Online Bandit						
LinTS	46.48	307.4	4.49	46.43	35.9	0.75
LinUCB	46.84	296.9	4.38	49.82	31.1	0.96
Static: Heuristic						
DimensionBest	47.50	277.4	3.69	—	—	—
kNN Retrieval	47.18	286.7	6.07	14.29	66.7	1.45
Static: Trained Policy						
LogReg	47.26	284.4	6.27	19.64	61.8	1.17
RouteLLM-BERT	47.22	285.5	6.22	21.43	59.4	1.30
TF-IDF+MLP	46.97	292.8	6.11	13.39	67.9	1.17
Qwen3.5-0.8B-Finetuned	46.41	309.1	6.82	55.36	27.2	0.74
RouteLLM-MF	46.16	316.5	6.19	8.93	72.7	0.94
Single-Model Baselines						
Always-Opus 4.6	43.83	387.1	1.29	57.14	26.7	0.64
Always-Kimi-K2.5	36.66	593.3	12.62	18.75	62.3	1.22
Always-Qwen3.5-Plus	37.16	580.2	2.05	2.68	80.1	0.19
Random	38.75	533.6	2.48	31.25	50.4	0.85

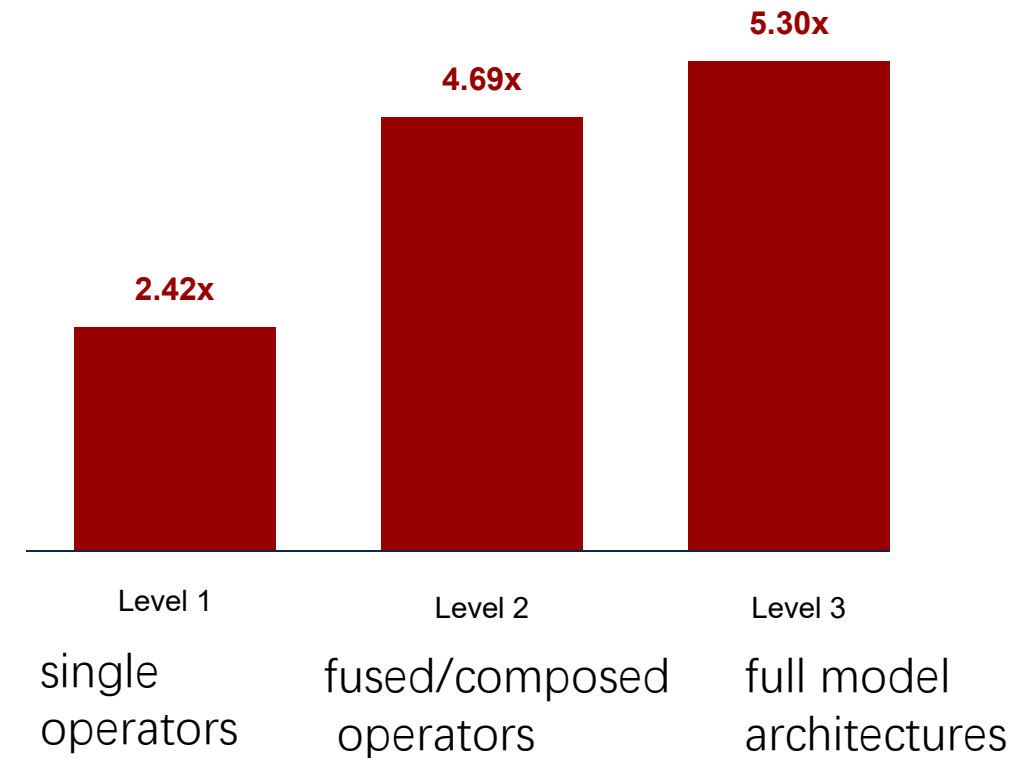
Agent Loops for Kernel, Infra and Hardware Optimization

Kernel Agents: From One-Shot Generation to Measured Search

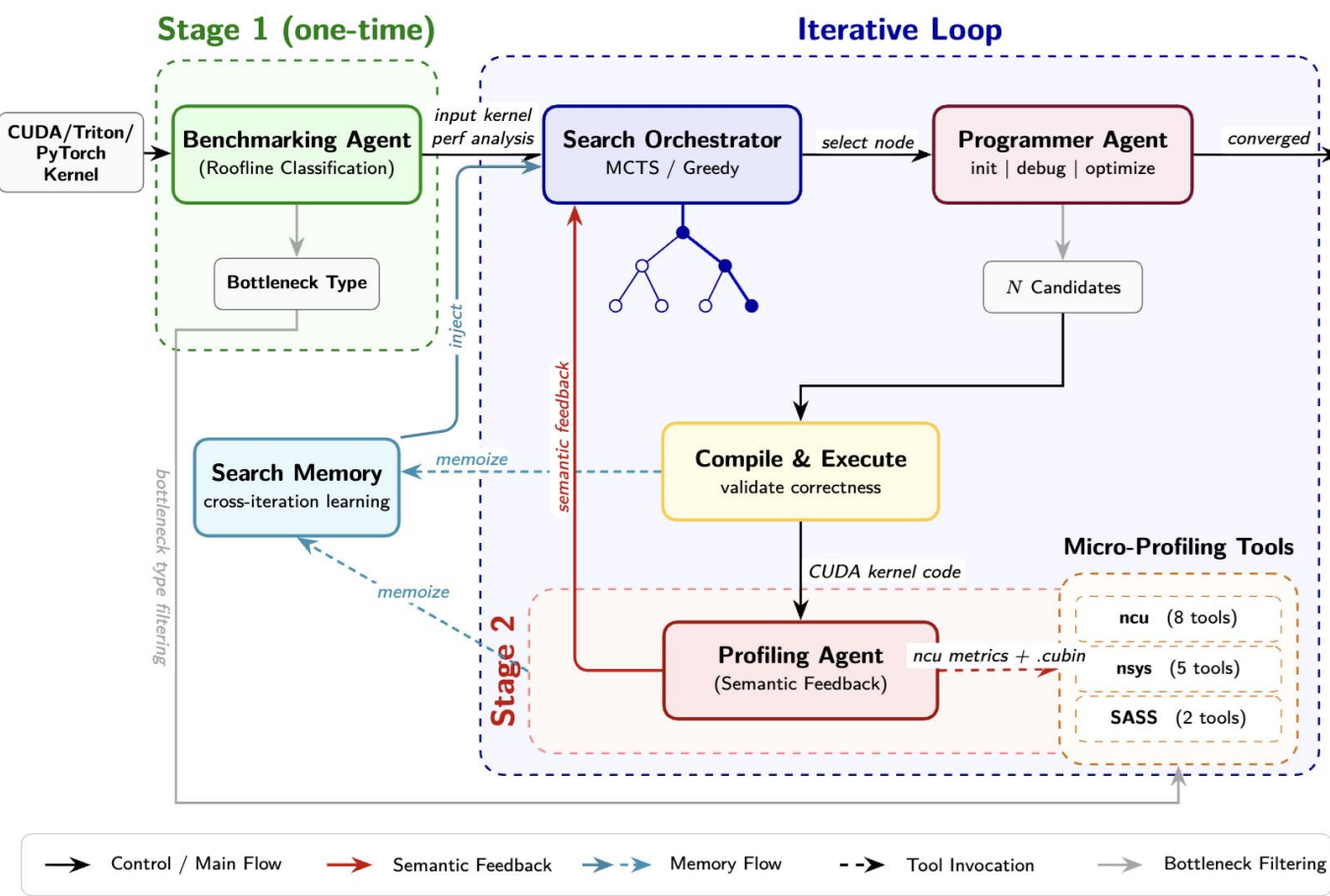
- One-shot LLMs beat PyTorch eager on **< 20%** of KernelBench (250 tasks) — the wins come from the loop.
- **The loop:** correctness gate → benchmark → NCU profile → targeted next edit
 - **KernelFalcon** (PyTorch) — 100% correctness on all 250 KernelBench tasks
 - **KernelPro** — 5.30x geomean on Level 3; -11.6% energy at matched speed
 - **KernelEvolve** (Meta, ISCA 2026) — +60% inference throughput in production

**Profiler evidence → bottleneck diagnosis → next edit:
kernel optimization becomes measured search.**

KernelPro geomean speedup vs. KernelBlaster



Example: KernelPro



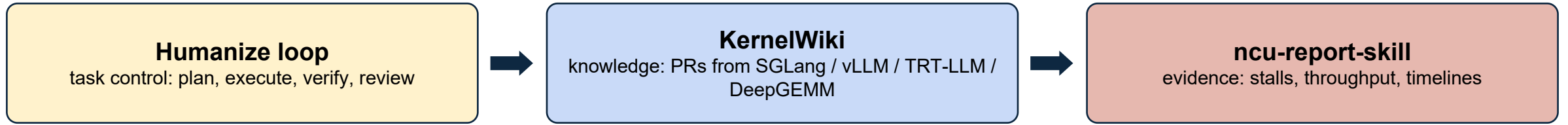
nsys vs ncu: GPU 性能分析工具对比

先用 nsys 找“哪里慢”，再用 ncu 查“为什么慢”

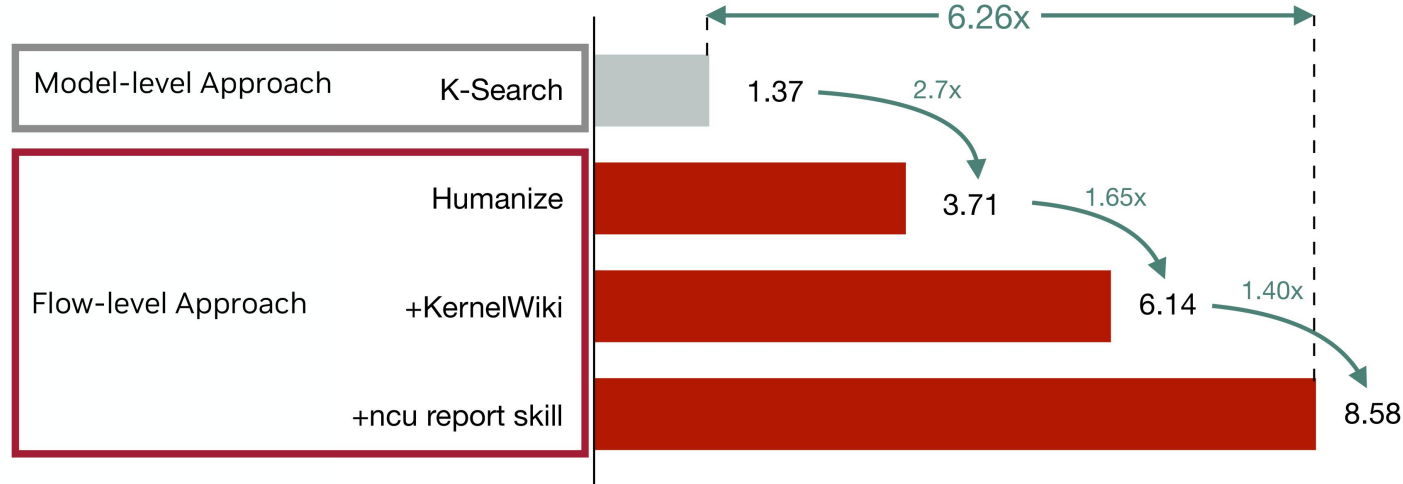
工具	作用	核心指标
 Nsight Systems nsys	全局时间线 定位“哪里慢”	Kernel 时长 · GPU 利用率 CPU - GPU 同步 · Memcpy Launch 开销
 Nsight Compute ncu	深入单个 Kernel 分析“为什么慢”	SM / Tensor Core 利用率 显存带宽 · L1 / L2 命中率 Occupancy · Warp Stall



KDA — Top-3 in Every MLSys'26 FlashInfer Track



DeepSeek Sparse Attention TopK Indexer Kernel



Mean Speedup vs FlashInfer Baseline

Over 200+ teams:

- #1 MoE track — Full-Agent
- #2 DSA track — Full-Agent
- #3 GDN track — Full-Agent

- DSA indexer: **1.37x** → **19.08x**
- DSA attention: **6.15x** → **15.22x**
- FP8 MoE: **0.27x** → **1.31x** (post-contest, vs closed-source TRT-LLM baseline)
- Not magic everywhere: GDN decode 0.92x — verifier-grounded honesty

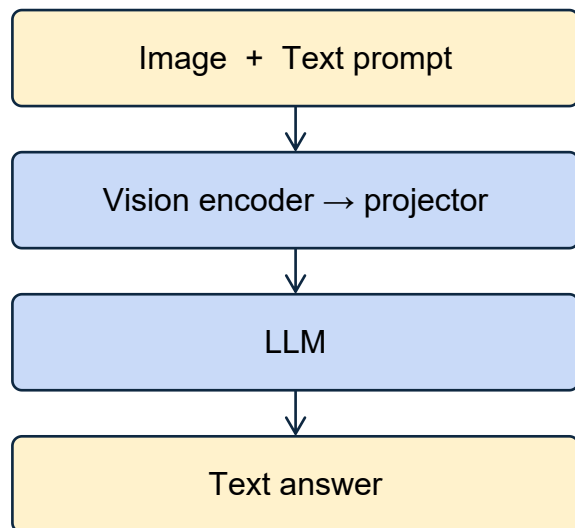
- **No single trick — structure, knowledge, and profiler evidence compound to 6.26x.**
- **Implement: CC, Review: Codex**

Part 5 – Future Work

Evolution of Multimodal Foundation Models

VLM: understanding

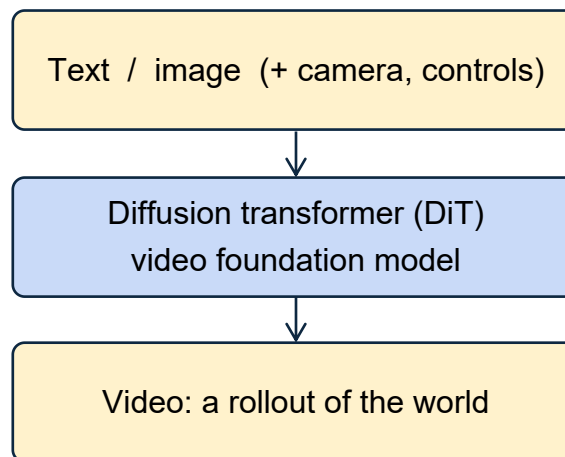
e.g., LLaVA, Qwen-VL



Bolt a vision encoder onto an LLM via visual instruction tuning — perception & reasoning, output is text only.

Image/video generation

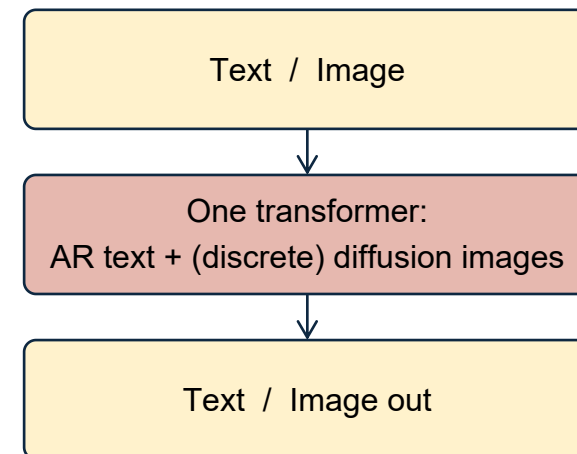
e.g., Wan 2.x, HunyuanVideo



Open video models (Wan 2.7 leads Wan-Bench 2.0; HunyuanVideo-1.5 runs on one 4090) — generation becomes simulation: predicting how the world evolves.

Unified understanding + generation

e.g., Show-o, Janus-Pro



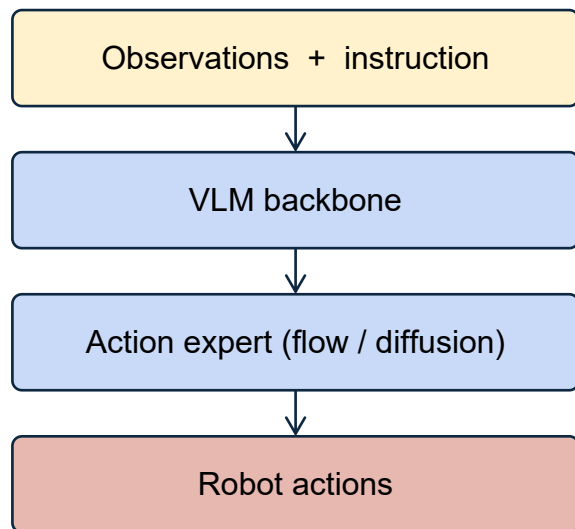
One model both understands and generates; Janus-Pro decouples visual encoding for the two roles.

From perceiving images, to generating them, to simulating how the world evolves.

Adding Action: VLA → World Action Model → Unified Action

VLA: vision–language–action

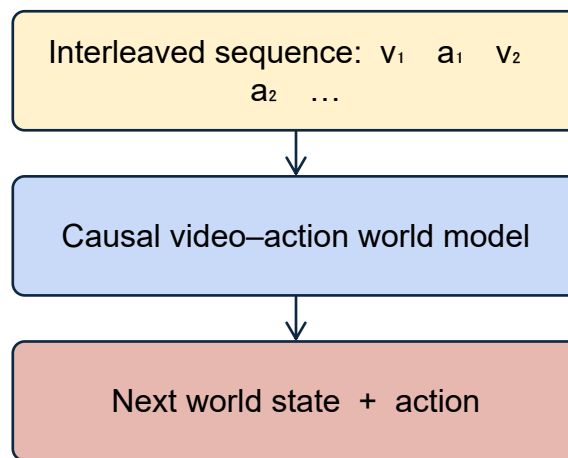
e.g., π -0.5, Qwen-VLA



π -0.5: open-world generalization in unseen homes; Qwen-VLA: one unified action space controls many robot bodies (38k hrs of data).

World action model

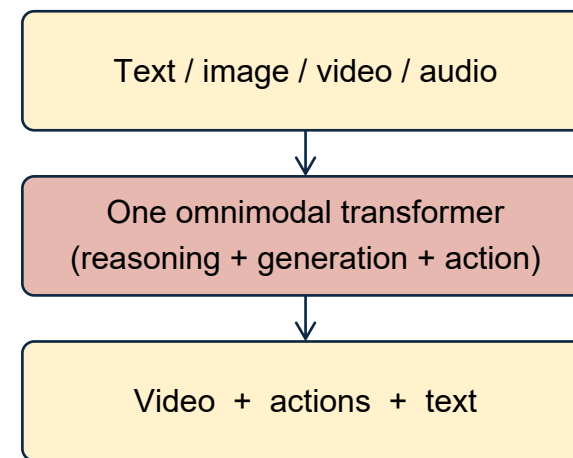
e.g., LingBot-VA



"Think before acting": imagine the next frames, then act. 98.5% on LIBERO-Long; adapts to new tasks with 30–50 demos.

Unified action model

e.g., Cosmos-3, π -0.7



Cosmos-3: two-tower MoT omni-model subsuming VLMs, video generators, simulators & policies; π -0.7: steerable generalist with emergent skill composition.

[1] Physical Intelligence, π -0.5 (2025) & π -0.7 (2026). [2] Qwen Team, Qwen-VLA, 2026. [3] NVIDIA, Cosmos-3: Omnimodal World Models for Physical AI, 2026. [4] Ant Lingbo, LingBot-VA, RSS 2026.

Action closes the loop: from predicting the world to acting in it with one model.

Future Work – 4D World Model

World model properties:

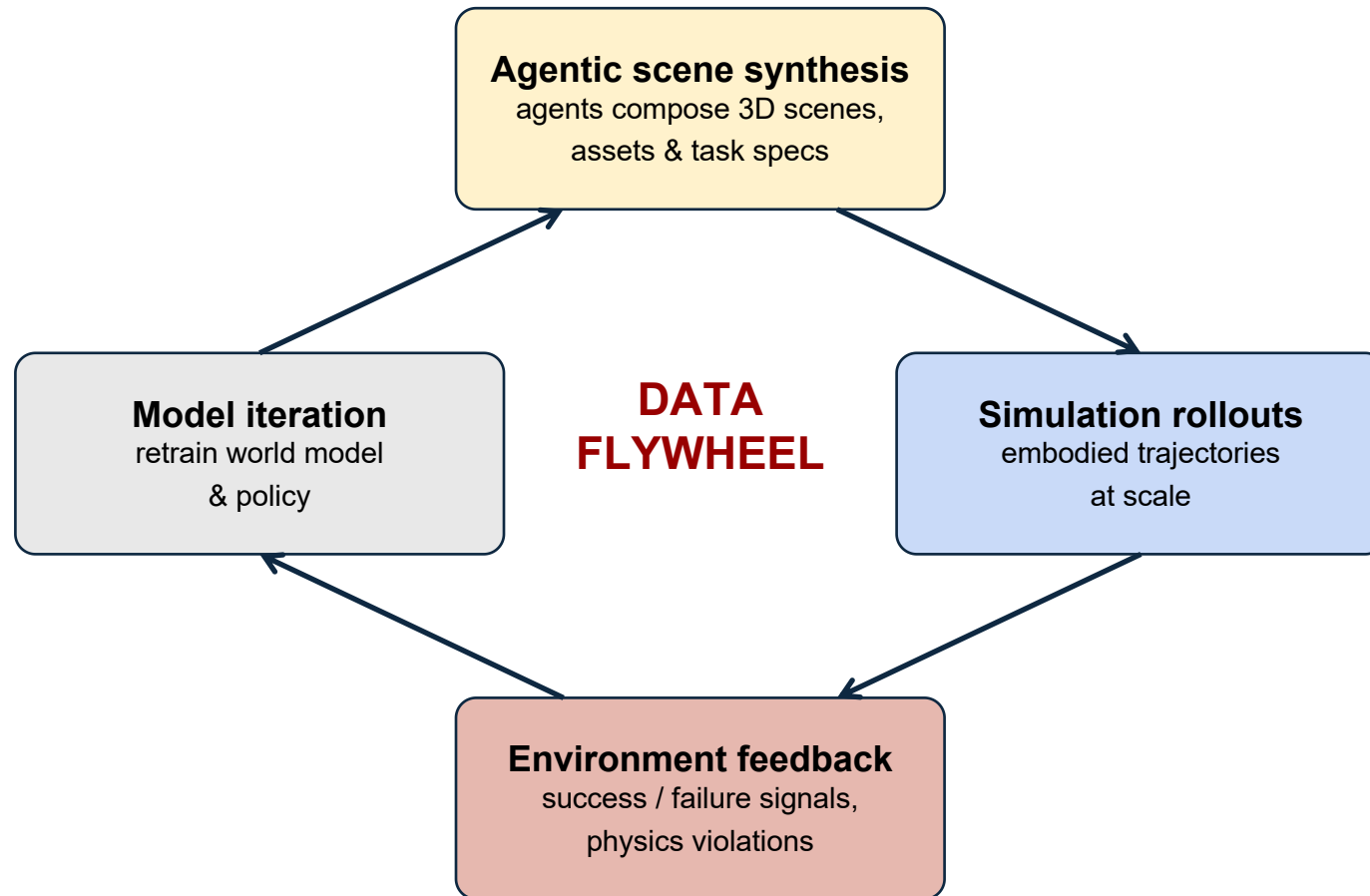
Real-time, interactive, physically plausible, geometry consistent, long, high-fidelity...

Why we need 4D world model?

- Simulator for agentic AI, embodied AI, and gaming.
- Scaling up World Model -- emerging understanding (perception and reasoning) capabilities.

LLM-centric era will be gone! Vision-centric will be the future!

Agentic World Model: What We Are Targeting



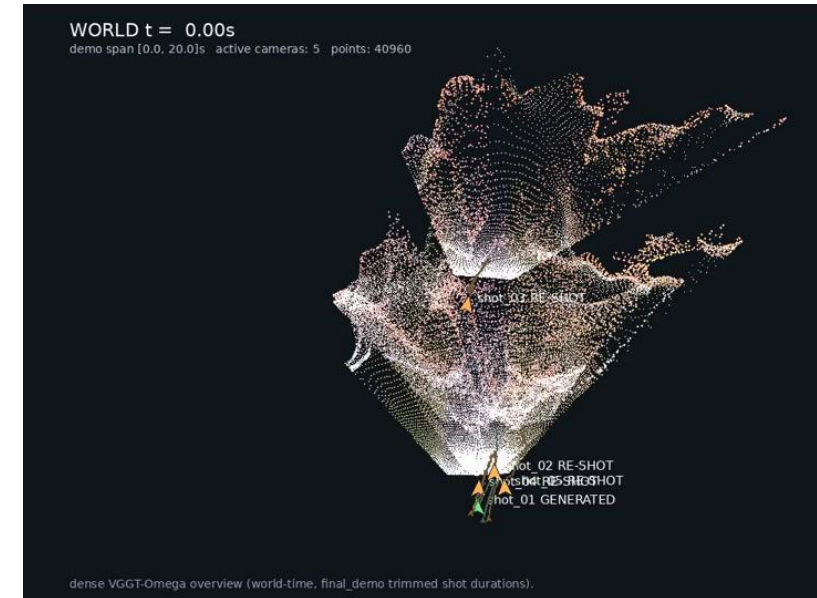
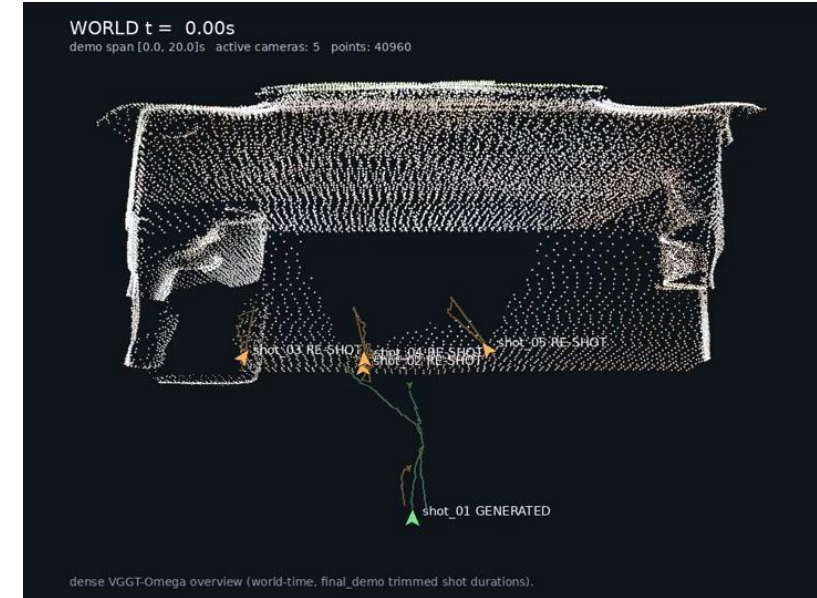
- Embodied data is the bottleneck: real-robot collection is slow, sim scenes are hand-built
- Our approach: agents synthesize simulation-ready scenes & tasks; environment feedback filters and improves them
- Every loop iteration improves both the data and the model

~100× cheaper

our synthesized scenes cost two orders of magnitude less than commercial scene pipelines

An agentic world model turns simulation into a self-improving data flywheel — at 1% of the commercial cost.

Agentic World Model: A Simple Demo



Some Real Industry-level Infra Issues – Training

- **万卡集群动态容错：**避免全集群重启，实现精度无损的故障恢复。
- **长序列通算融合 Attention：**挖掘 B卡新架构算力并实现高效的通信-计算重叠，提升长序列下的 MFU。
- **多模态与短序列优化：**支持任意 Mask 的稀疏计算与负载均衡，弥补开源 Flash Attention 在 B 卡短序列场景的性能差距。
- **Data/Training Server 服务化架构：**数据预处理去中心化，支持多用户 LoRA 任务在线调度。

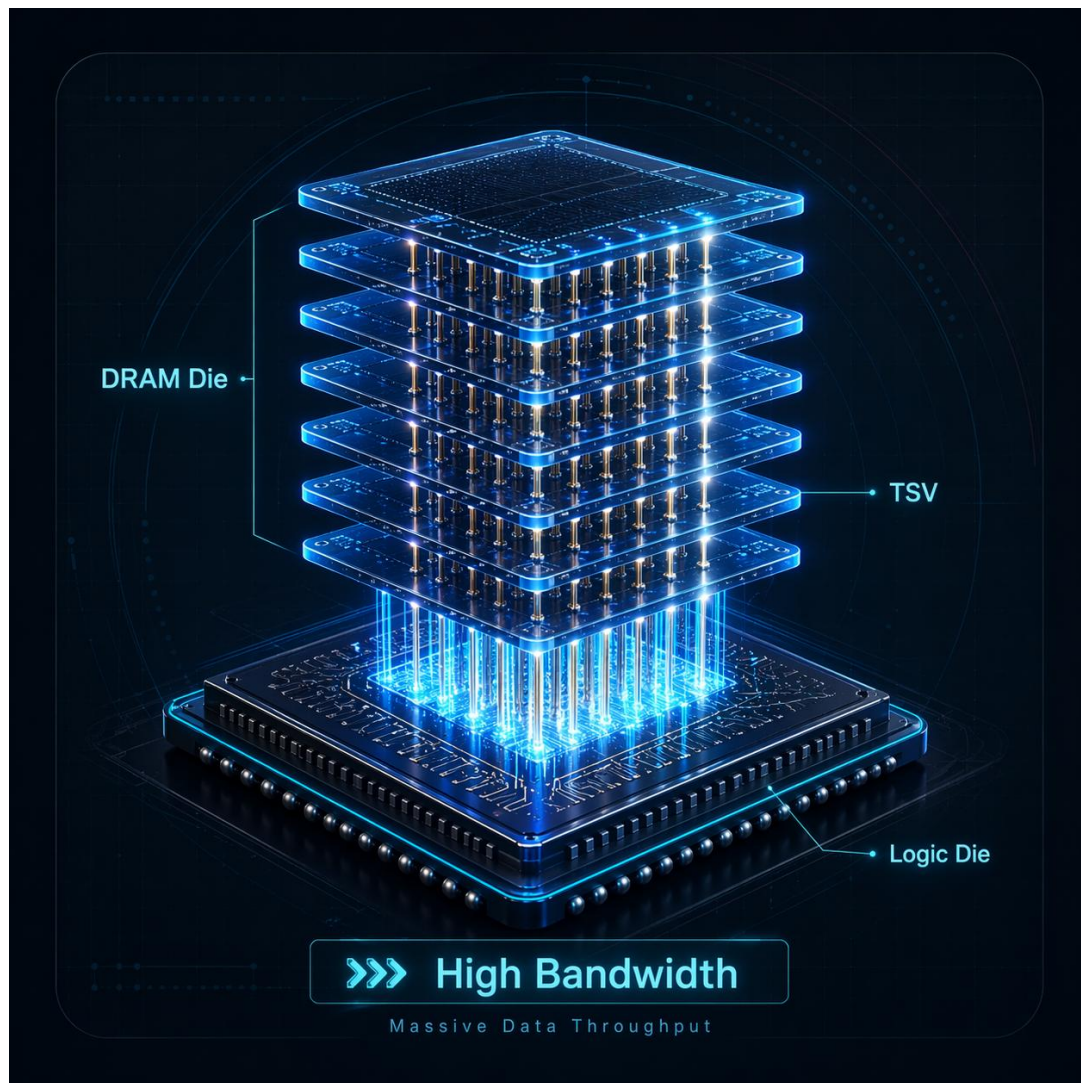
◦ ◦ ◦ ◦ ◦ ◦ ◦

Some Real Industry-level Infra Issues – Inference

- **多级 KV Cache 与存储优化：**构建 HBM-SSD-远端存储四级缓存并结合cache压缩，在保持约 90% 缓存命中率的同时大幅压降存储成本。
- **异构计算统一图优化：**自动构图与算子融合替代低效的人工调优。
- **多模态无损推理：**低比特量化与投机解码须保证生成质量零回退。
- **动态资源调度与新硬件适配：**以负载感知的动态 PD 调度应对训推波峰波谷，并针对 Blackwell 显存小、通信弱的特点实现计算-通信重叠。

◦ ◦ ◦ ◦ ◦ ◦ ◦ ◦ ◦

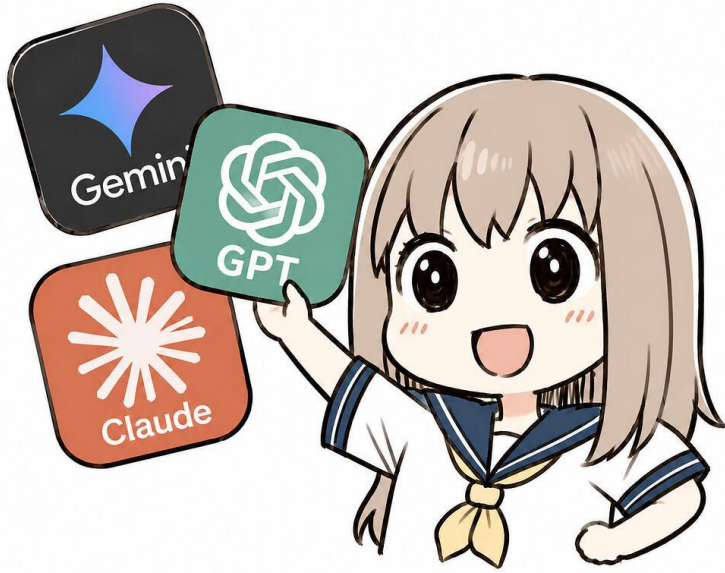
AI Chip for Agent



高带宽存储与大规模 AI 计算能力

- 大模型训练与推理系统
- AI 编译器及自动优化
- 高性能Kernel
- 面向新型芯片架构的软硬件协同优化

e.g., 3D 堆叠, 光通信, 原生多级存储, etc...



你还能有AI聪明？

Vision & perspective
decide how far you can go

Research taste & insight
decide the quality and impact of your work

Agent-native workflow
decides how fast ideas become reality

Think big, stay curious, be agent-native — and take care of yourself along the way.